

Szoftvertchnológia és -technikák

2. Előadás

Bevezetés az UML világába, osztálydiagram



Tartalom

Modellezés - bevezetés

UML

Osztálydiagram



Modellezés

Szoftverfejlesztés

- Készítsünk egy programot, ami kiszámolja és kiírja egy adott szám faktoriálisát!



```
class Faktorialis
{
    public static void main(String args[])
    {
        int i;
        int fact = 1;

        int number = 5;
        for (i = 1; i <= number; i++)
        {
            fact = fact * i;
        }
        System.out.println("A(z) " + number + " faktoriálisa: " + fact);
    }
}
```



Szoftverfejlesztés

- Készítsünk programot, ami kiszámolja a binomiális tételt bekért paraméterekkel!
$$\sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$
- Készítsünk programot, ami kiszámol egy tetszőleges matematikai műveletsort!
(Az elérhető függvények listája előre adott)
- Készítsünk egy tőzsdei trendeket előrejelző, pénzügyi tanácsadó alkalmazást mobilra!



Szoftverfejlesztés

- Tervezés nélkül is lehet építeni házat
 - > Fatárolóhoz elég lehet
 - > Ha egyedül építi az ember
 - > Azonnal összeomlik...
 - > ... vagy csak később?
- Több szintes családi ház
 - > Többen építik
 - Sorrend: A tetőfedő csak az elején ér rá?
 - Értelmezés: Tűzfalra néző ablak
 - > Élni is szeretnénk benne
 - > Sok évig



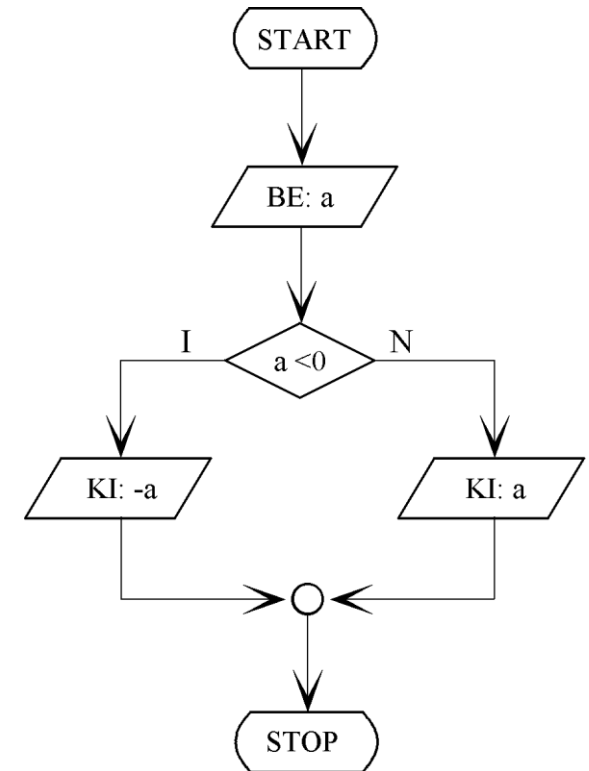
Szoftverfejlesztés nagyban

- Szoftver
 - > Kis feladat → “Csak meg kell írni”
 - > Komplex feladat → Érdemes megtervezni
- Tervezés
 - > Ha a kódméret/feladat komplex
 - > Ha egyeztetni kell a megrendelővel
 - > Ha meg kell becsülni a költségeket
 - > Ha több ember fejleszti, új belépők is lehetnek
 - > Ha segédmunkások is vannak
- Megoldás
 - > Modellezés



Modellezés

- Modell: a valóság egy egyszerűsített nézete
 - > Absztrakciós szint növekedés
 - > Hangsúly azon, ami fontos...
 - > ... elrejtve azt, ami nem
- Modell a szoftverfejlesztéshez
 - > A struktúrához?
 - > A folyamatokhoz?
 - > A viselkedéshez?
 - > ... Vagy mindenhez ezek közül?



Unified Modeling Language

Bevezetés

Szoftvermodellezés

- Igény a szoftverek tervezésére
 - > Magas absztrakciós szint, vizuális ábrázolás
 - > '90-es évektől több modellező nyelv: kaotikus helyzet
 - > 1997. – Unified Modeling Language (UML)
- Unified Modeling Language
 - > Cél: szoftverfejlesztés és dokumentáció modellezése
 - > Általános modellező nyelv
 - > A szabvány modellezés terén
 - > Jelenlegi verzió: 2.5.1.
<https://www.omg.org/spec/UML/2.5.1/PDF>
 - > Jó magyarázatok és példák:
<https://www.uml-diagrams.org>
 - > Modellezőeszköz a félév során:
Modelio (<https://www.modelio.org/>)
 - Részletek gyakorlaton!



UML

- UML használatának előnyei

- > Vázlat

- Implementáció előtt:

- Probléma jobb megértése
 - Hibák/hiányosságok feltárása

- Implementáció után

- Dokumentálás
 - Kommunikáció
 - Megrendelővel
 - Fejlesztőkkel

- > Tervrajz

- Terv alapján könnyebb dolgozni
(architect vs. programozó)



UML diagrammok

- Struktúra

- > Osztálydiagram (Class diagram)
- > Komponensdiagram (Component diagram)
- > Telepítési diagram (Deployment diagram)
- > Objektumdiagram (Object diagram)
- > Csomagdiagram (Package diagram)
- > Összetett struktúradiagram (Composite Structure diagram)
- > Profildiagram (Profile diagram)



UML diagramok II.

- Viselkedés
 - > Aktivitásdiagram (Activity diagram)
 - > Használati eset diagram (Use case diagram)
 - > Állapotgép (State machine)
 - > Interaktivitás
 - Szekvenciadiagram (Sequence diagram)
 - Kommunikációs diagram (Communication diagram)
 - Interakció áttekintő diagram (Interaction overview diagram)
 - Időzítő diagram (Timing diagram)



UML

- Mit ad az UML?
 - > Jelölésrendszer, közös nyelv
 - > Magas absztrakciós szint, tömör leírás
 - > Szabványos keret
 - > Segít szemléltetni a szoftvertervezési módszereket
- Mit nem ad az UML?
 - > Nem oldja meg a problémát, csak segít leírni azt
 - > Nem mutatja meg, *hogyan* tervezzünk jól
 - Ezért tanulunk a Clean code-ról, a SOLID elvekről,...
 - Ezért kellenek a tervezési minták



Osztálydiagram

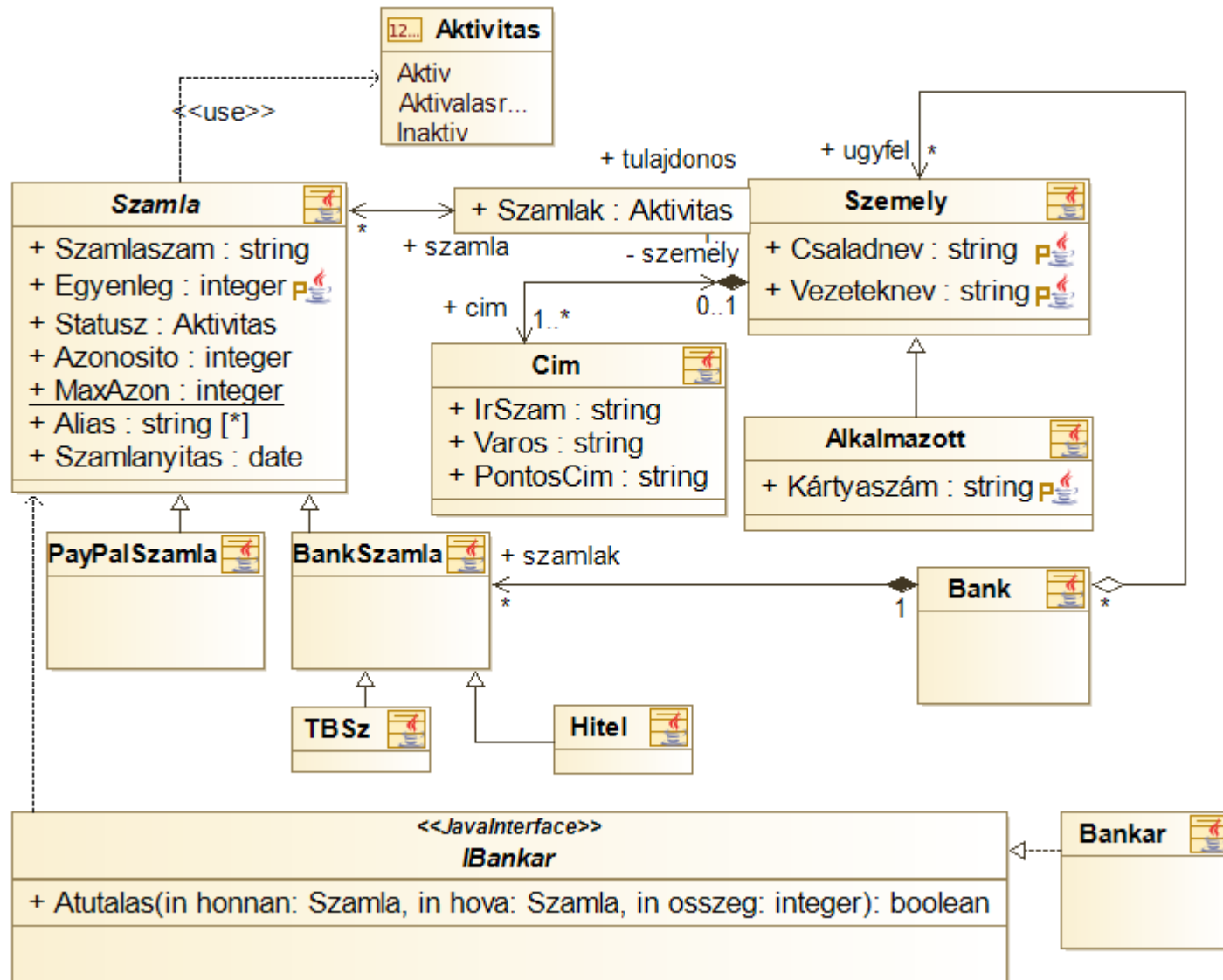
Class diagram

Osztálydiagram

- A leggyakrabban használt, **legfontosabb UML diagram**
- OOP-alapon készült rendszerekhez
- Struktúra megadására (osztályok, tagváltozók, metódusok, kapcsolatok)
- Programozási nyelvtől független!
 - Generálható belőle forráskód (metódusoknál csak a váz)



Osztálydiagram: Banki adminisztrációs rendszer



Modellelemek

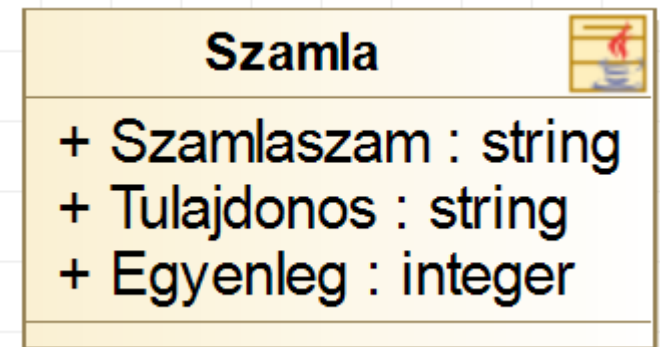
Osztályok



Számlaszám	Tulajdonos neve	Egyenleg (Ft)
11742232-15393276	Nagy Lajos	12 000 343
11742232-75343245	Hunyadi Mátyás	4 556
11742232-35393127	Károly Róbert	443 227 786

- Készítsünk hozzá egy osztályt és tagváltozókat!
 - > Osztály (class) és tagváltozó (attribute)

```
1 public class Szamla {  
2  
3     public String Szamlaszam;  
4     public String Tulajdonos;  
5     public int Egyenleg;  
6  
7 }
```



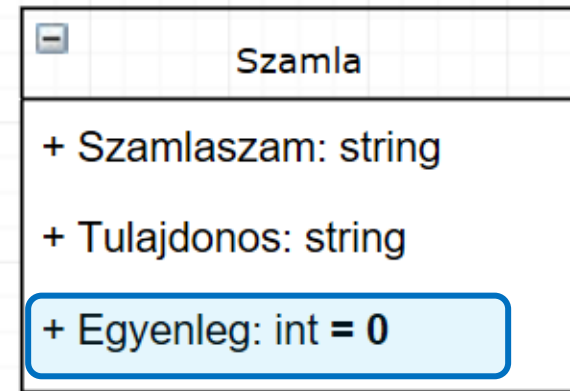
Kezdőérték

- Legyen az egyenleg alapesetben 0 Ft!
 - > Sajnos a Modelio eszköz nem jeleníti meg az ábrán!



> A generált kód:

```
1 import com.modeliosoft.modelio.javadesigner.annota
2
3 @objid ("84fe0cc8-8a45-4874-9b29-6c491370c38b")
4 public class Szamla {
5     @objid ("e2c21051-b938-40b4-b271-48f932146246")
6     public String Szamlaszam;
7
8     @objid ("f15659a4-cc45-45be-bc79-597568508df9")
9     public String Tulajdonos;
10
11     @objid ("d662343e-735f-44fa-8c68-1db19fdf66c2")
12     public int Egyenleg = 0;
13
14 }
15
```



Láthatóság

+ -> **public**
- -> **private**
-> **protected**

- Az egyenleg megváltoztatását és lekérdezését válasszuk szét!



- > Metódus (method)
- > Láthatóság (visibility/member access modifier)
- > Privát tagváltozó + publikus metódusok
- > Tagváltozó → property

Szamla	
+ Szamlaszam : string	
+ Tulajdonos : string	
- Egyenleg : integer	
+ getEgyenleg(): integer	00
+ setEgyenleg(in value: integer)	00

Szamla	
+ Szamlaszam : string	
+ Tulajdonos : string	
+ Egyenleg : integer	P

```
12  
13  
14  
15  
16  
17  
18  
19
```

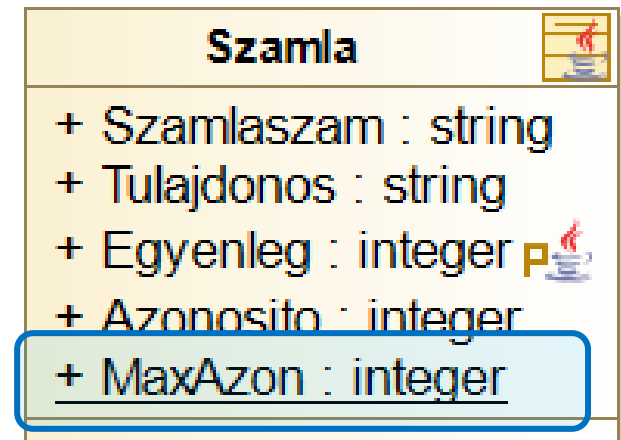
```
private int Egyenleg = 0;  
  
public int getEgyenleg() {  
    return this.Egyenleg;  
}  
  
public void setEgyenleg(int value) {  
    this.Egyenleg = value;  
}
```

Statikus tagváltozók

- Minden számlának legyen egy egyedi, belső azonosítója. Számla létrehozásakor az eddigi maximum azonosítóból számoljuk az új számla azonosítóját!
 - > A számlaazonosító példány szintű adat (minden számlának más)
 - > A max számlaazonosító viszont *osztály* szintű!
 - > Statikus tagváltozó kell (aláhúzás!)
 - > A számolási logika nem a modell része!



```
public class Szamla {  
    ...  
  
    public int Azonosito;  
    public static int MaxAzon;  
  
    ...  
}
```



Multiplicitás

[*] -> korlátlan elemű lista
[1] -> Pontosan 1 elem
[0..*] -> 0 vagy több elem (lista)

- Lehesse megadni a számlához több alias nevet!
 - > Multiplicitás (multiplicity)
 - > Attribútum többes multiplicitással
 - 0..1
 - 1..1 (alapértelmezett)
 - 0..* (lista generálódik belőle!)
 - n..m



Szamla	
+ Szamlaszam :	string
+ Egyenleg :	integer 
+ Statusz :	Aktivitas
+ Azonosito :	integer
+ MaxAzon :	integer
+ Alias :	string [*]

Interfész

- Készítsünk egy interfészt, ami támogatja a számlák közti átutalás műveletét!
 - > **Művelet (operation)**
 - Művelet - definíció vs. Metódus - implementáció
 - > A paraméter típusa nem beépített típus!
 - > Szignatúra: + Atutal(honnan: Szamla,
hova: Szamla,
osszeg: integer): bool



<<JavaInterface>>

IBankar

+ Atutalas(in honnan: Szamla, in hova: Szamla, in osszeg: integer): boolean

Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])([: visszatérési érték])

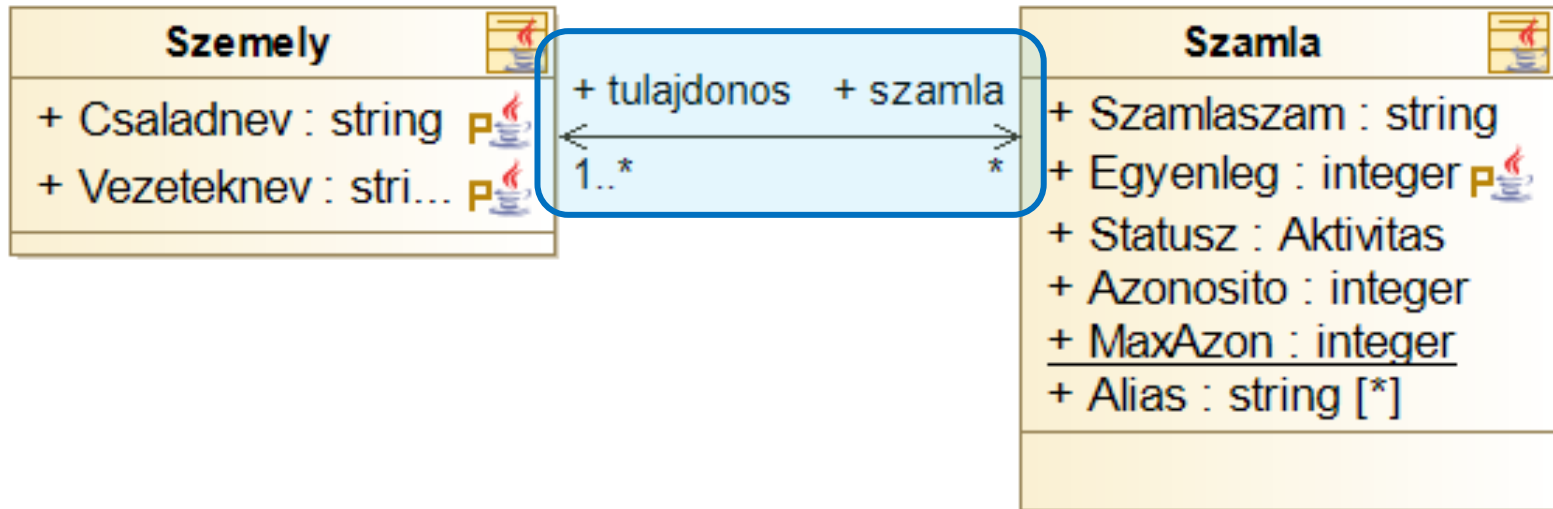
- > Láthatóság

- Public: +
 - Private: -
 - Protected: #
 - Package: ~

Kapcsolatok

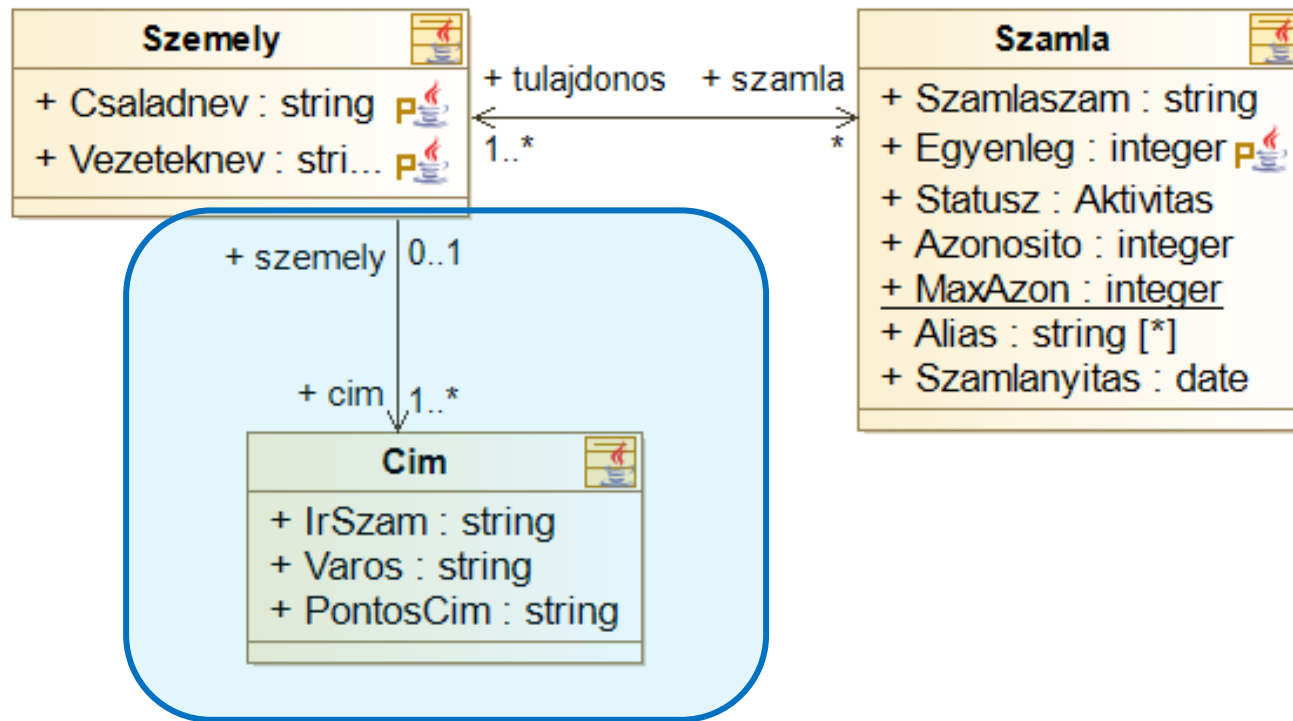
Asszociáció

- Támogassuk, hogy egy személynek több számlája is lehessen!
 - > Asszociáció (association)
 - > Ki kell emelni a személyt az osztályból!
 - > Szerepnév (role name) és multiplicitás (multiplicity)
 - > Kódban: tagváltozó



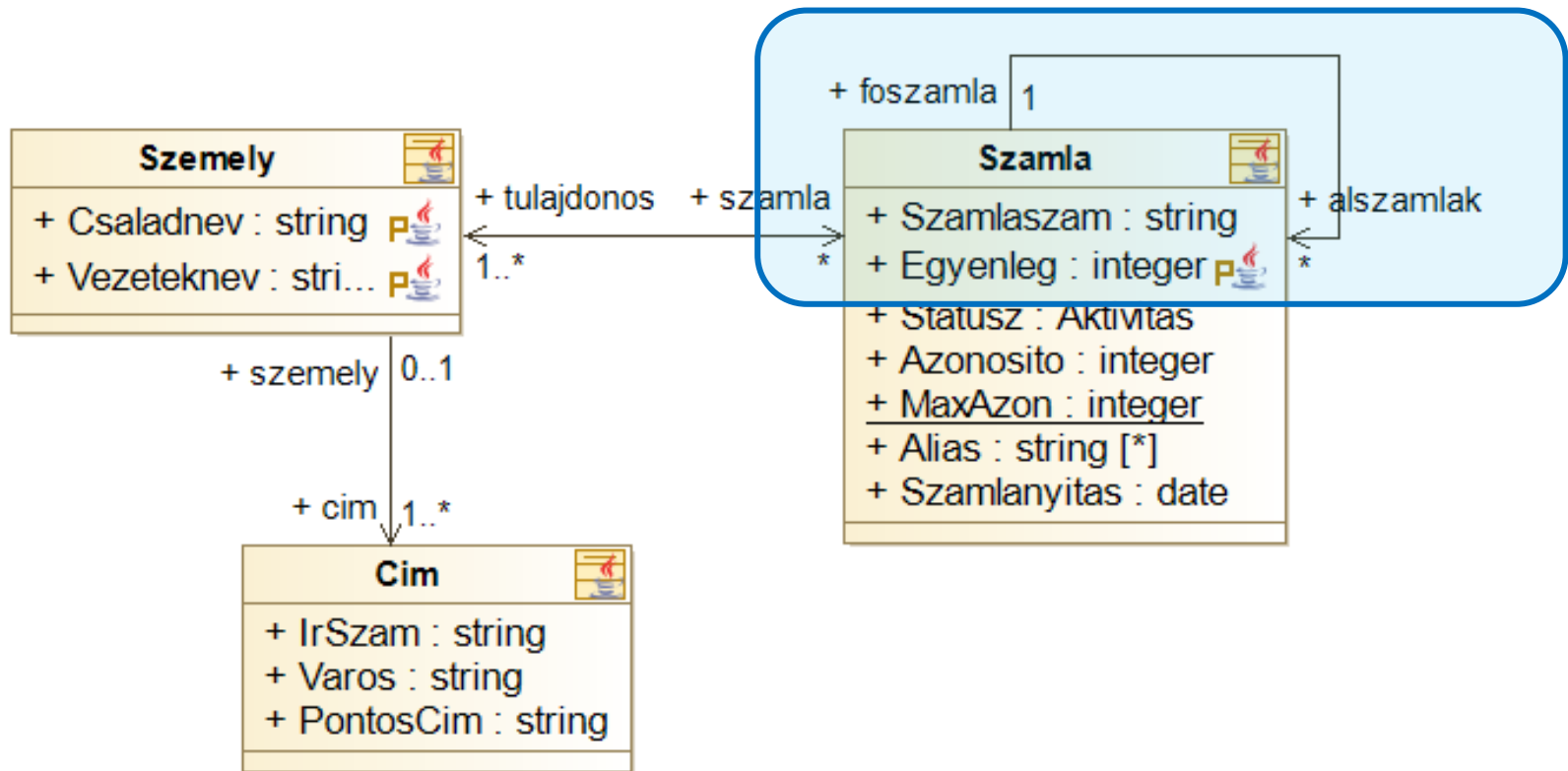
Asszociáció

- Lehessen megadni az egyes emberek lakcímét!
 - > Egy irányban navigálható kapcsolat



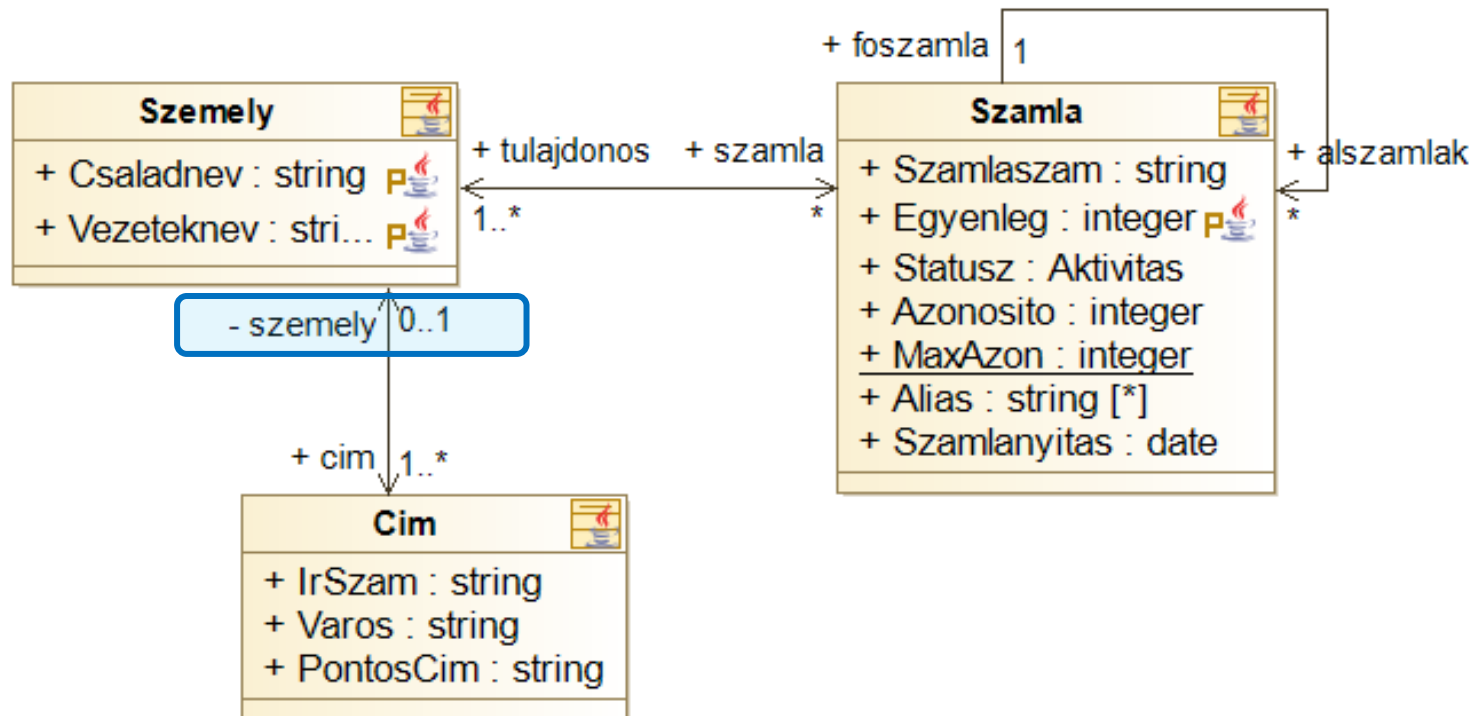
Asszociáció

- Lehessen megadni alszámlákat!



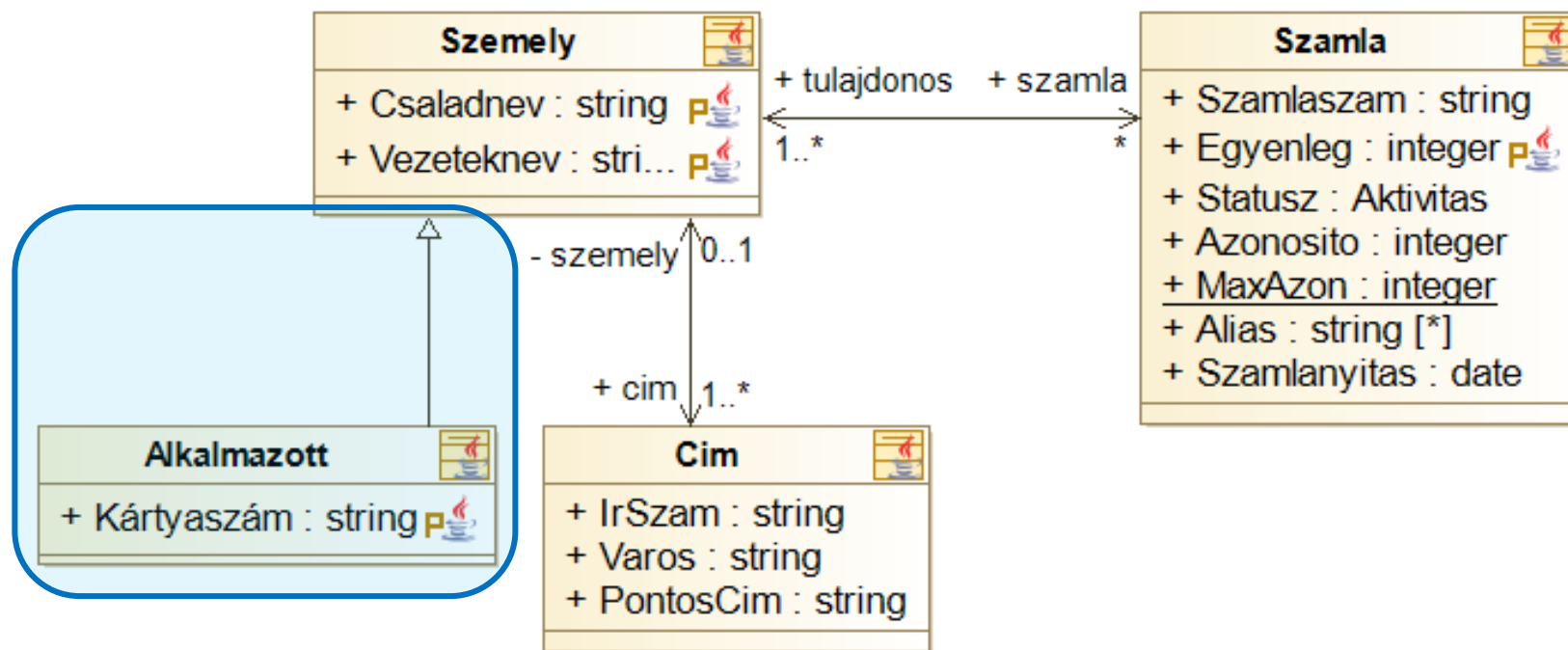
Asszociáció

- A címtől lehessen lekérdezni, melyik személyhez tartozik, de mindezt csak a cím entitáson belülről!
 - > Láthatóság a navigáción!



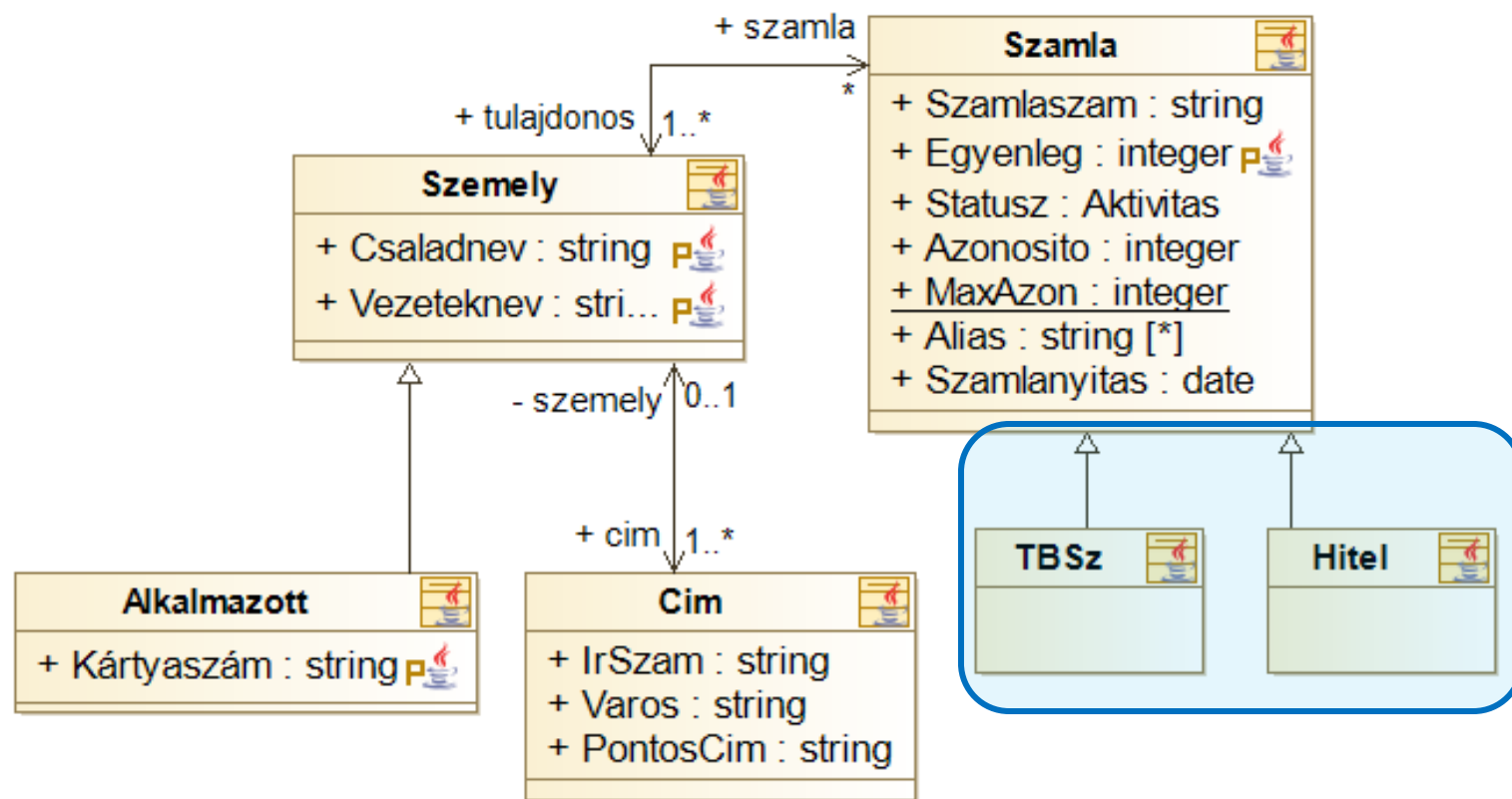
Általánosítás/öröklés

- Tartsuk nyilván a banki dolgozókat is! Nekik is lehet számlájuk, címük, de van azonosítókártyájuk is!
 - > Általánosítás/öröklés (generalization/inheritance)
 - > Kódban: öröklés a két osztály közt



Általánosítás/öröklés

- Legyenek külön számlatípusaink! (TBSz, Hitel)
 - > Általánosítás/öröklés?



Tartalmazás

- Minden számla tartozzon egy bankhoz!
 - > Egy számla nem tartozhat két bankhoz
 - > Ha a bankot töröljük, a számlát is törölni kell!
 - > Kompozíció
(composition)
 - > Kódban: tagváltozó



ERŐS kapcsolat!

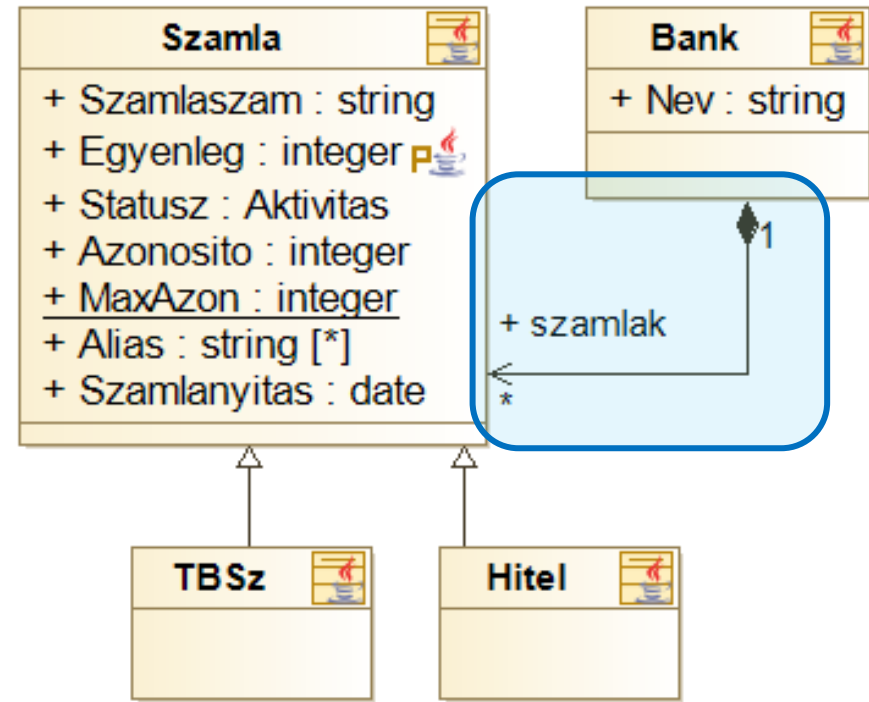
A rész (Számla) nem létezhet az egész nélkül (Bank).

Másik pl: Ház —♦— Szoba

-> Ha a ház megszűnik a szoba is.

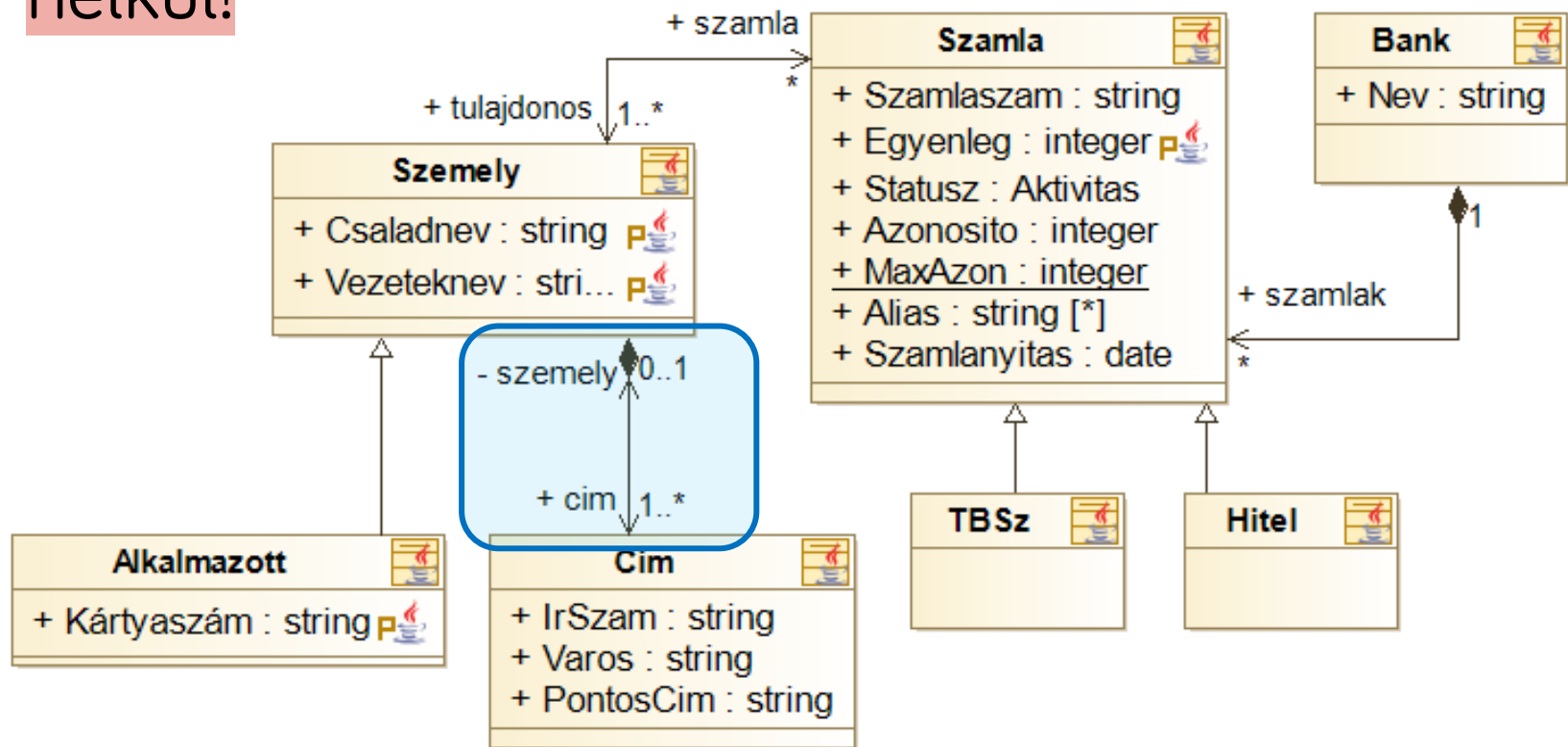
-> A rombusz ott van, ahol a TULAJDONOS!

EGÉSZ ♦ — RÉSZ | eset



Tartalmazás

- Egy cím csak egy személyhez tartozhasson és ne tároljunk feleslegesen címeket, személy nélkül!



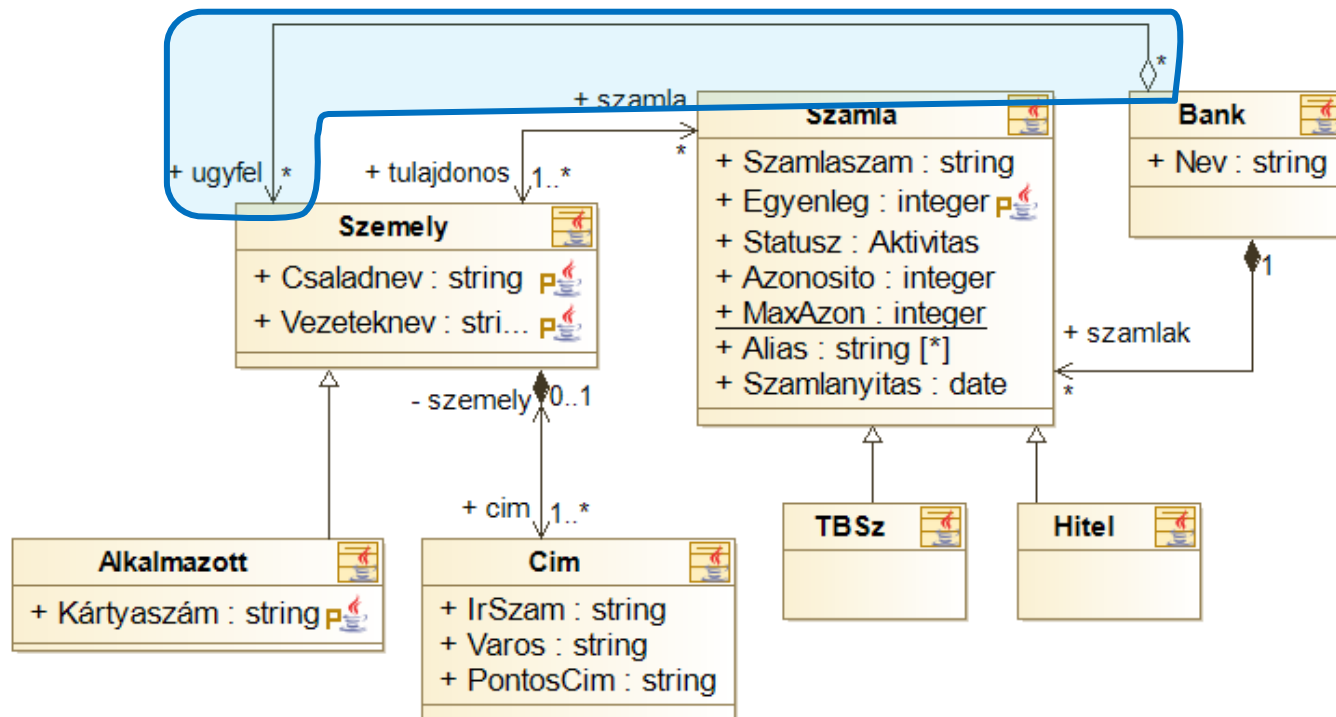
Tartalmazás

Aggregáció - gyenge tartalmazás
A RÉSZ létezhet az EGÉSZ nélkül is!

Tanszék ◇ — Tanár

- A bank tartsa nyilván az ügyfeleit, akkor is, ha azok megszüntetik a számlájukat! Egy ügyfél több bank ügyfele is lehet.

- > Megosztott tartalmazás (aggregation)
- > Kódban: tagváltozó

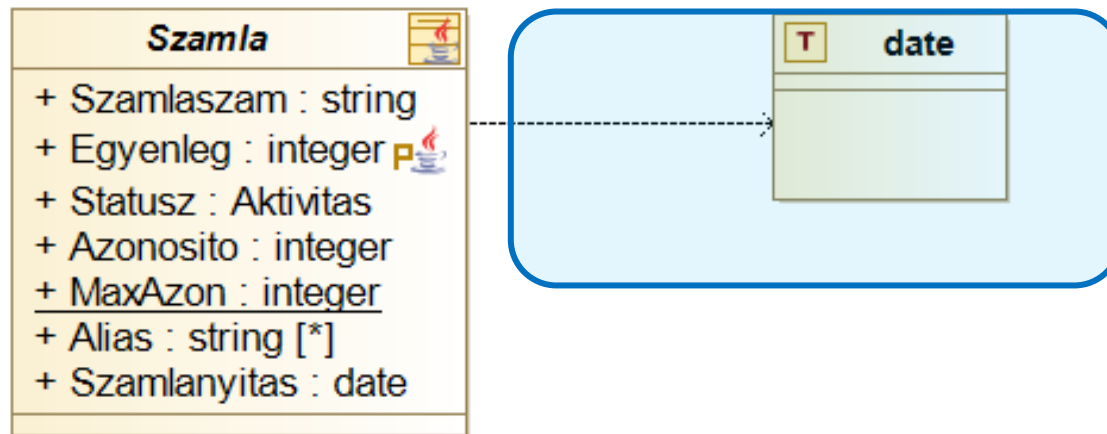


Függőség

Gondoljunk itt a Dependency Injection-re (DI)
Az osztály HASZNÁLJA a másikat, de nem TÁROLJA el!

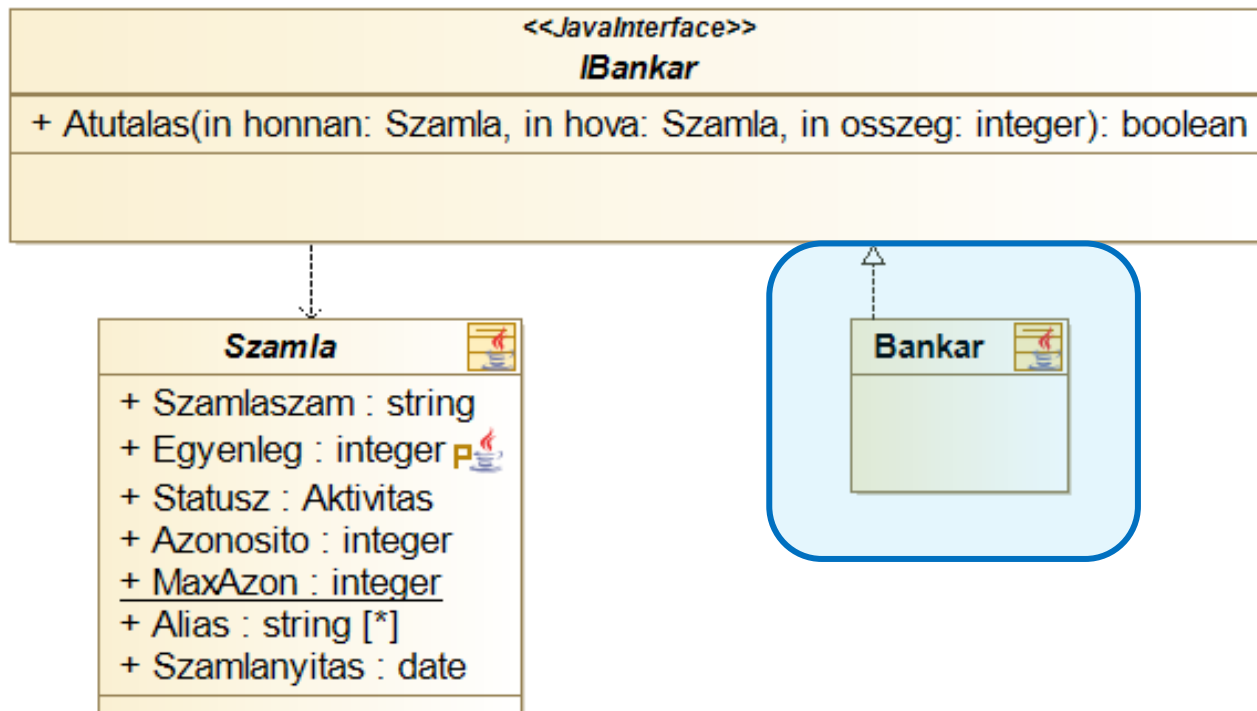
```
class garage {  
    void megszerel(Auto auto)  
        auto.indit();  
}
```

- Lehesse megadni a számlanyitás dátumát!
 - > A típus itt egy saját típus (modellelem)
 - > Függőség (dependency) kapcsolat
 - > Jellemzően nem szoros, hanem ideiglenes kapcsolat
 - > A függőből mutat afelé, amitől függünk
 - > Kódban: import/using



Interfészek II.

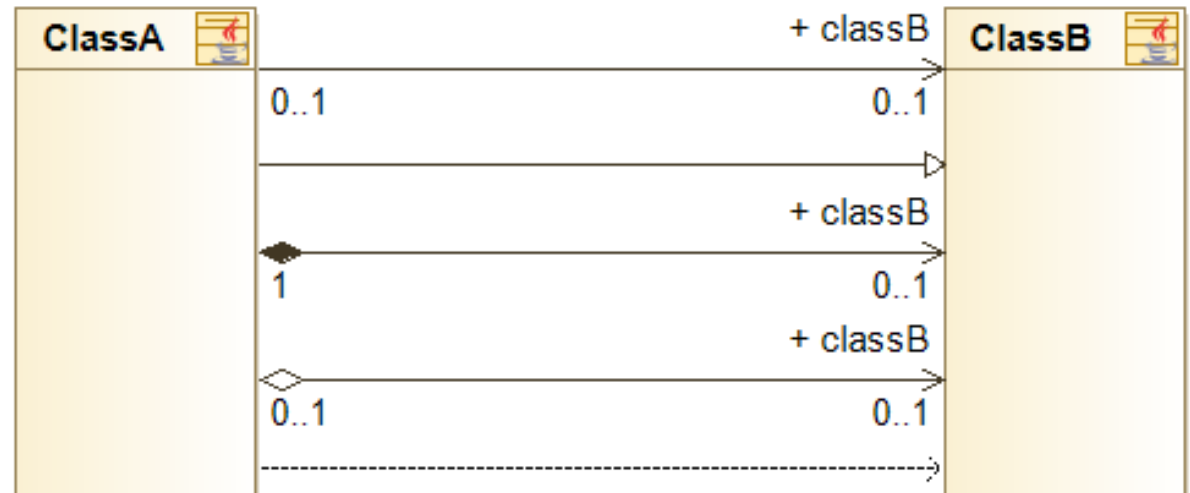
- Térjünk vissza az IBankar interfészhez:
 - > Jelöljük a kapcsolatot a számlával!
 - > Valósítsuk meg egy osztályban az interfészt!
 - > **Interfész megvalósítás (interface realization)**
 - > **Kódban: implements/”:**



Kapcsolatok

- **Kapcsolat típusok**

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



- Multiplicitás
- Navigálás
 - > Szerepnév
 - > Navigálhatóság

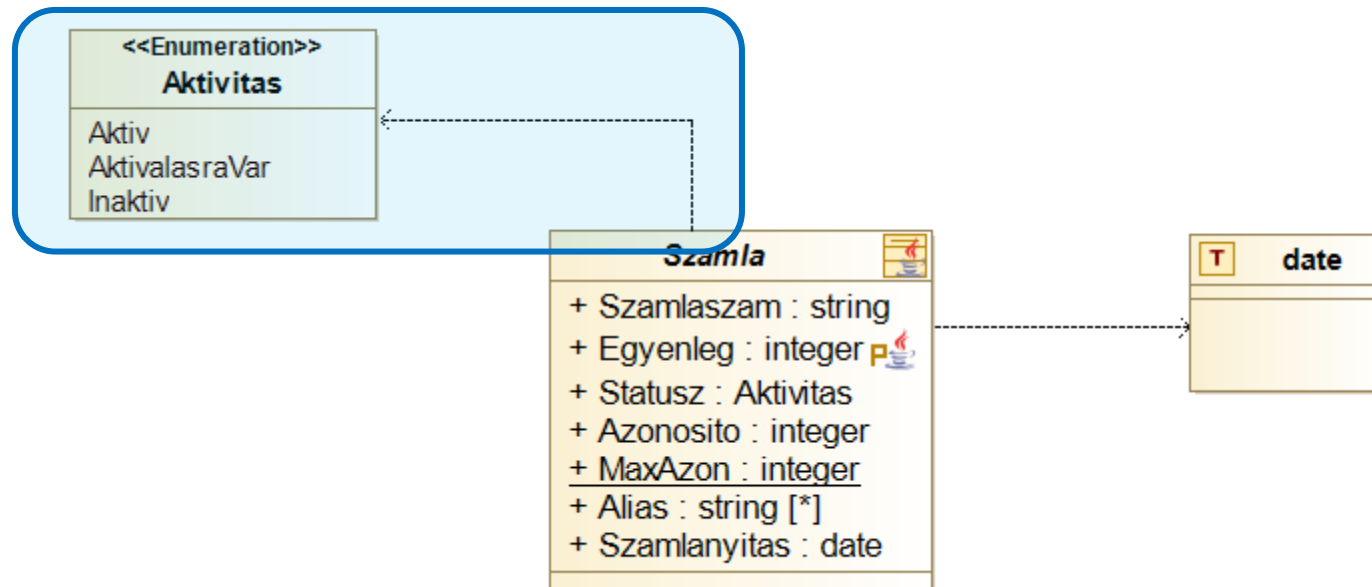
A kapcsolatok szemantikája

- „Is-a” kapcsolat
 - > Általánosítás/interfész megvalósítás (generalization/interface realization)
 - > Közös funkcionalitás
 - > Ősosztállyal felcserélhetőség (Liskov)
 - > Kibővített működés
- „Has-a” kapcsolat
 - > Kizárólagos: kompozíció (containment)
 - Tartalmazó törlése törli a tartalmazottat
 - Tartalmazó egyedi
 - > Megosztott: aggregáció (aggregation)
- Nem tartalmazás, nem öröklés, de (részben) navigálható
 - > Asszociáció (association)
 - > Kódban hasonlít a tartalmazáshoz, szemantikai különbség
- Laza/ideiglenes kapcsolat
 - > Függőség (dependency)

További elemek

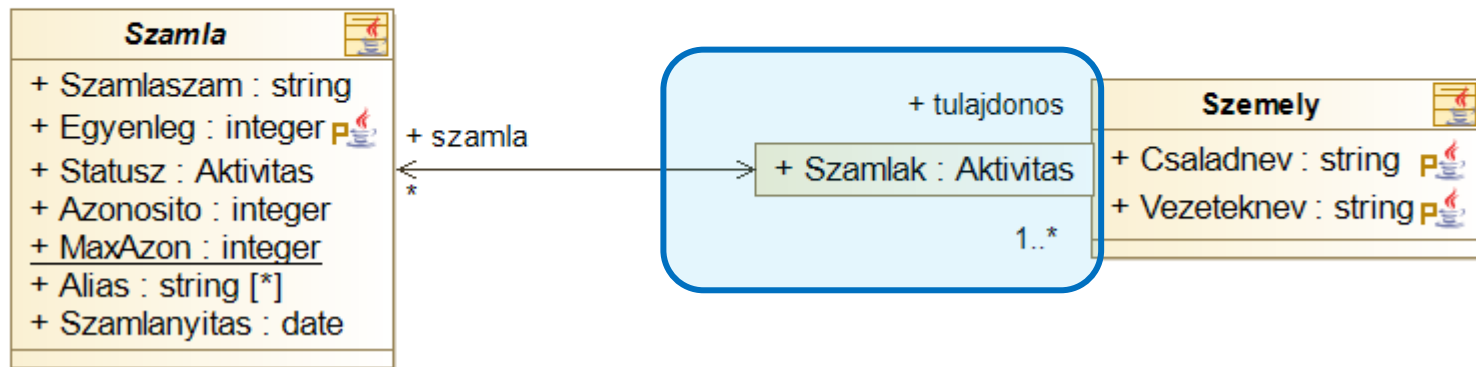
Enumerációk

- Tároljuk el a számla státuszát!
(aktív/aktiválásra vár/inaktív)
 - > Enumeráció (enumeration)



Minősítő (Qualifier)

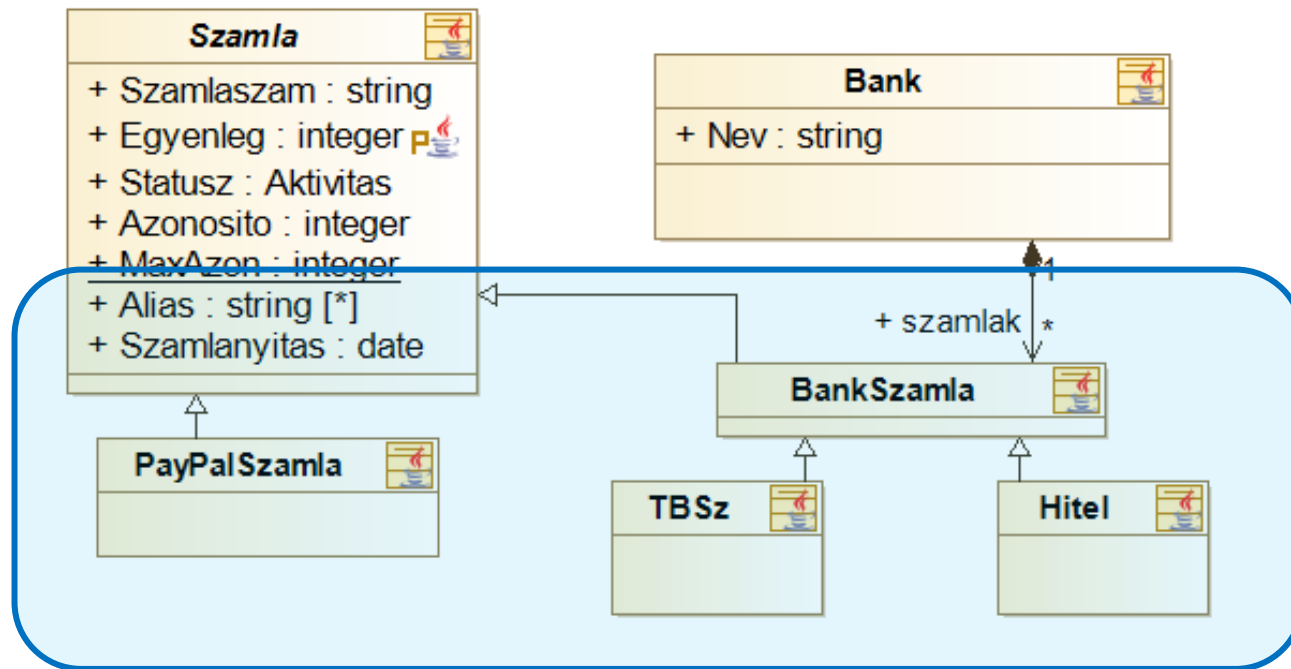
- Csoportosítsuk a számlákat aktivitás szerint!
 - > Minősített asszociáció (qualified association)
 - > Általános lista helyett egyedileg kiválasztott érték



```
public class Szemely {  
  
    ...  
    public Map<Aktivitas, Szamla> szamla =  
        new HashMap<Aktivitas, Szamla> ();  
    ...  
}
```

Absztrakt őszosztály

- Vezessük be a PayPal számlákat! Ezeknél nincs bank, nincsenek speciális számlatípusok, de egyébként bankszámlaként viselkedik minden tekintetben!
 - > Absztrakt őszosztály (abstract base class) (dőlt betű!)
 - > Számla + öröklés



Sztereotípiák

- Sztereotípia(stereotype)
 - > Meglévő elem jelentésének kiegészítése
 - > Jelölés: `<<stereotype_name>>`
 - > Példák
 - `<<interface>>`
 - `<<enumeration>>`
 - Dependency esetén `<<use>>`, `<<create>>`, `<<destroy>>`, ...
 - > Alkalmazástól/szakterülettől függő is lehet

Gondolatok: közös viselkedés?

- A Paypal és a hagyományos bankszámlák közt kell tudni átutalni bármilyen kombinációban.
 - > Mind a négy (2x2) fajta kombináció kell...
 - > ...de valójában ugyanazt ellenőrizzük!
- Megoldás 1. – Közös őosztály (Számla)
- Mi van, ha nem vonható közös őosztály alá?
 - > Megoldás 2. – Közös interfész



Gondolatok: asszociáció, vagy tagváltozó?

- Asszociáció – tartalmazás – attribútum
 - > Mindegyikből tagváltozót generálunk
 - > Mi a különbség?
 - Navigálhatóság, szerepnevek (pl. hurokél)
 - Minősítés (qualifier)
 - Vizualitás, átláthatóság

Köszönöm a figyelmet!

Szoftvertechnológia és -technikák

3. Előadás

Szekvenciadiagram



Tartalom

Osztálydiagram (ismétlés)

Szekvenciadiagram



Modellezés – UML - Osztálydiagram

Ismétlés

Szoftverfejlesztés nagyban

- Szoftver
 - > Kis feladat → “Csak meg kell írni”
 - > Komplex feladat → Érdemes megtervezni
- Tervezés
 - > Ha a kódméret/feladat komplex
 - > Ha egyeztetni kell a megrendelővel
 - > Ha meg kell becsülni a költségeket
 - > Ha több ember fejleszti, új belépők is lehetnek
 - > Ha segédmunkások is vannak
- Megoldás
 - > Modellezés



Szoftvermodellezés

- Igény a szoftverek tervezésére
 - > Magas absztrakciós szint, vizuális ábrázolás
 - > '90-es évektől több modellező nyelv: kaotikus helyzet
 - > 1997. – Unified Modeling Language (UML)
- Unified Modeling Language
 - > Cél: szoftverfejlesztés és dokumentáció modellezése
 - > Általános modellező nyelv
 - > A szabvány modellezés terén



Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])([: visszatérési érték])

- > Láthatóság

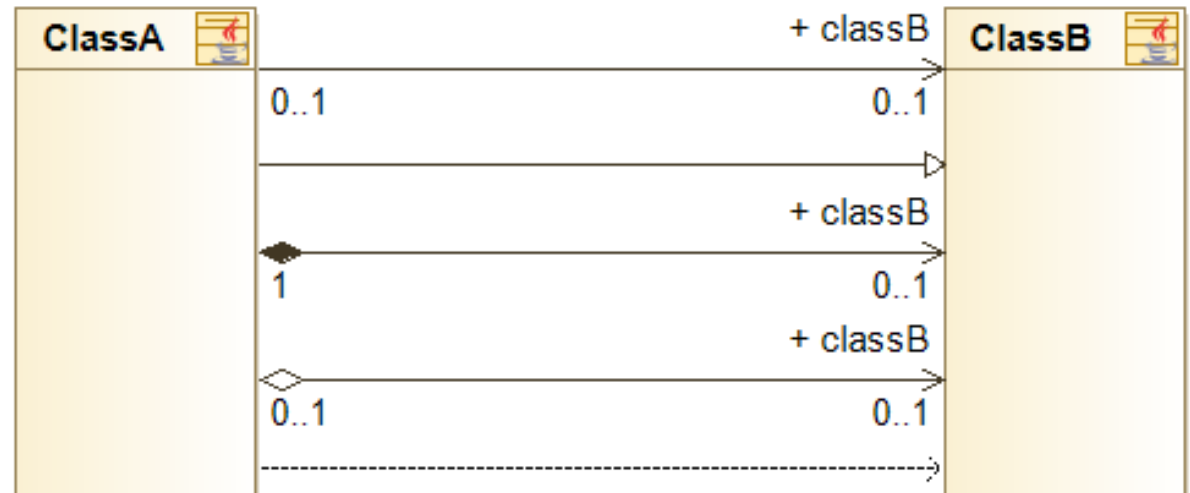
- Public: +
 - Private: -
 - Protected: #
 - Package: ~



Kapcsolatok

- **Kapcsolat típusok**

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



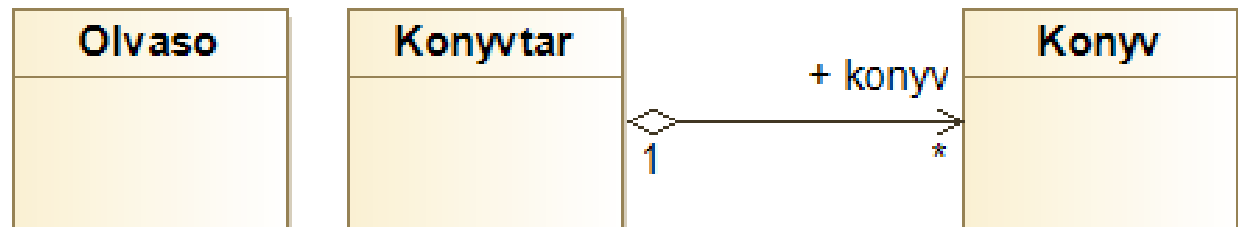
- Multiplicitás
- Navigálás
 - > Szerepnév
 - > Navigálhatóság



Osztálydiagram: könyvtár

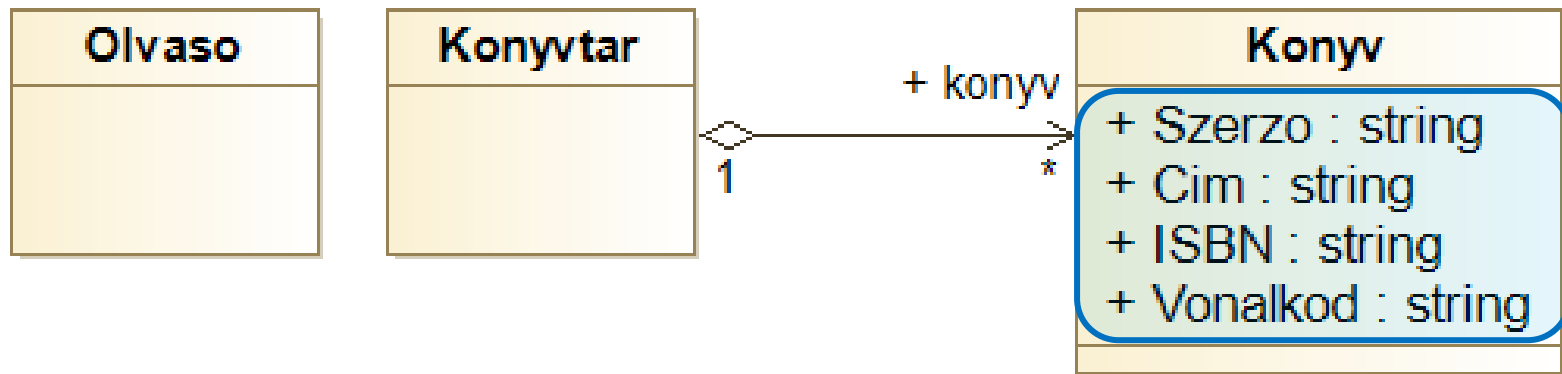
Könyvtár

- Hozzunk létre egy nyilvántartó rendszert könyvtárak számára!
 - > Legyenek benne olvasók és könyvek!
 - > ... természetesen a könyvtárat is modellezni kell
 - > ... a könyvtár tartalmaz könyveket



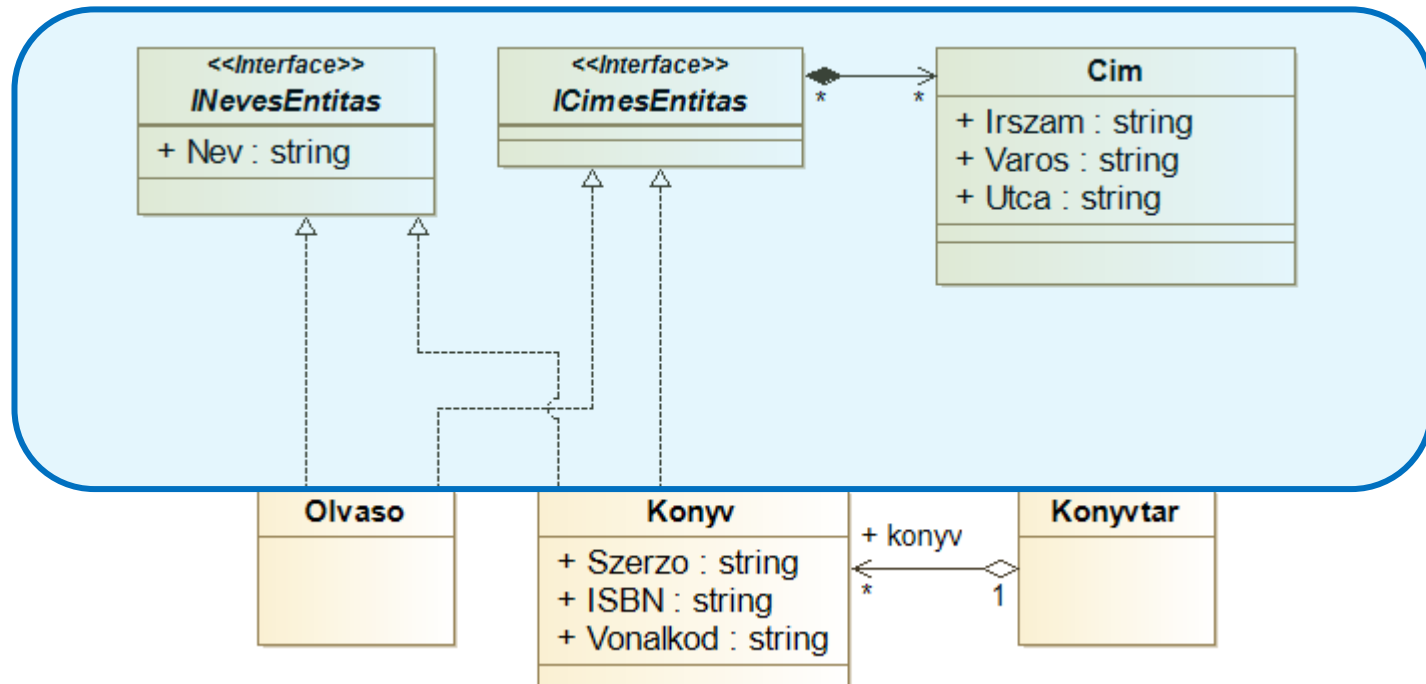
Könyvtár - adatok

- A könyveknek van szerzője, címe és ISBN száma
 - > Mi a könyv példány? Mi a könyv osztály?
 - > ISBN szám vs. egyedi azonosító
 - ISBN: Statikus?



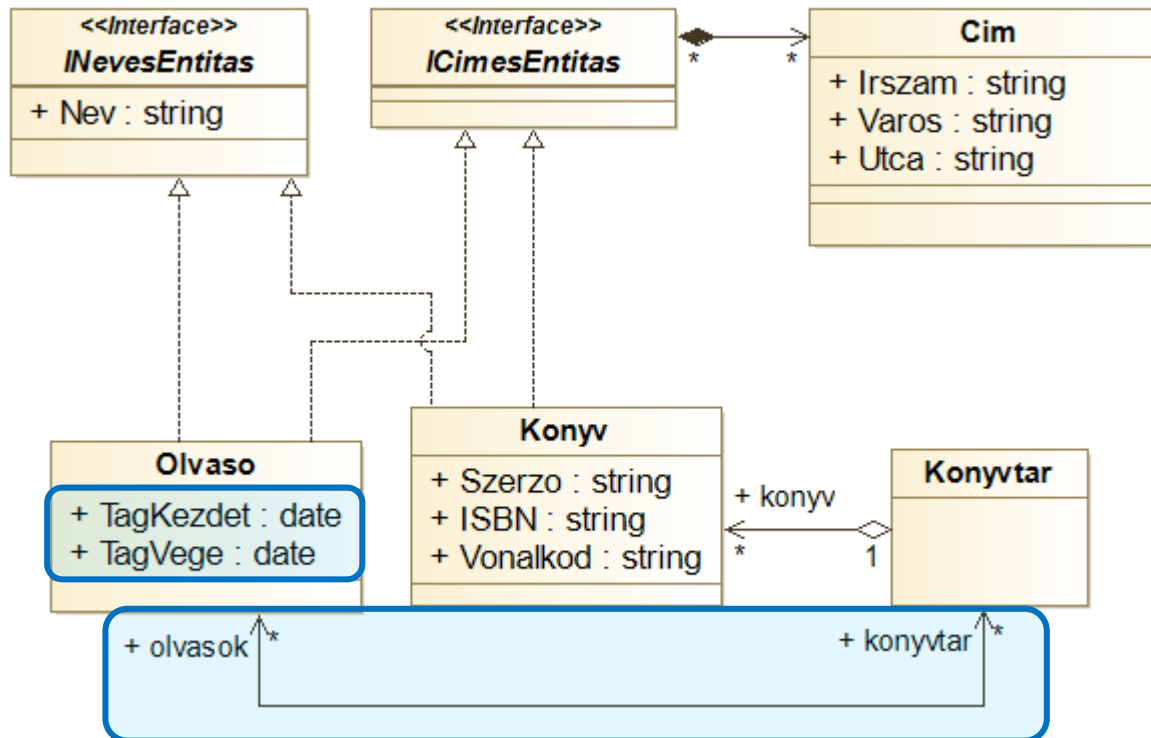
Könyvtár - adatok

- A könyvtárnak és az olvasónak van neve és címe
 - > Közös ős?
 - > Vagy... közös interfész? Egy interfész, vagy kettő?
 - > Vagy... csak egy saját, összetett típusú attribútum?
 - > Vagy... összetett attribútum + interfész?



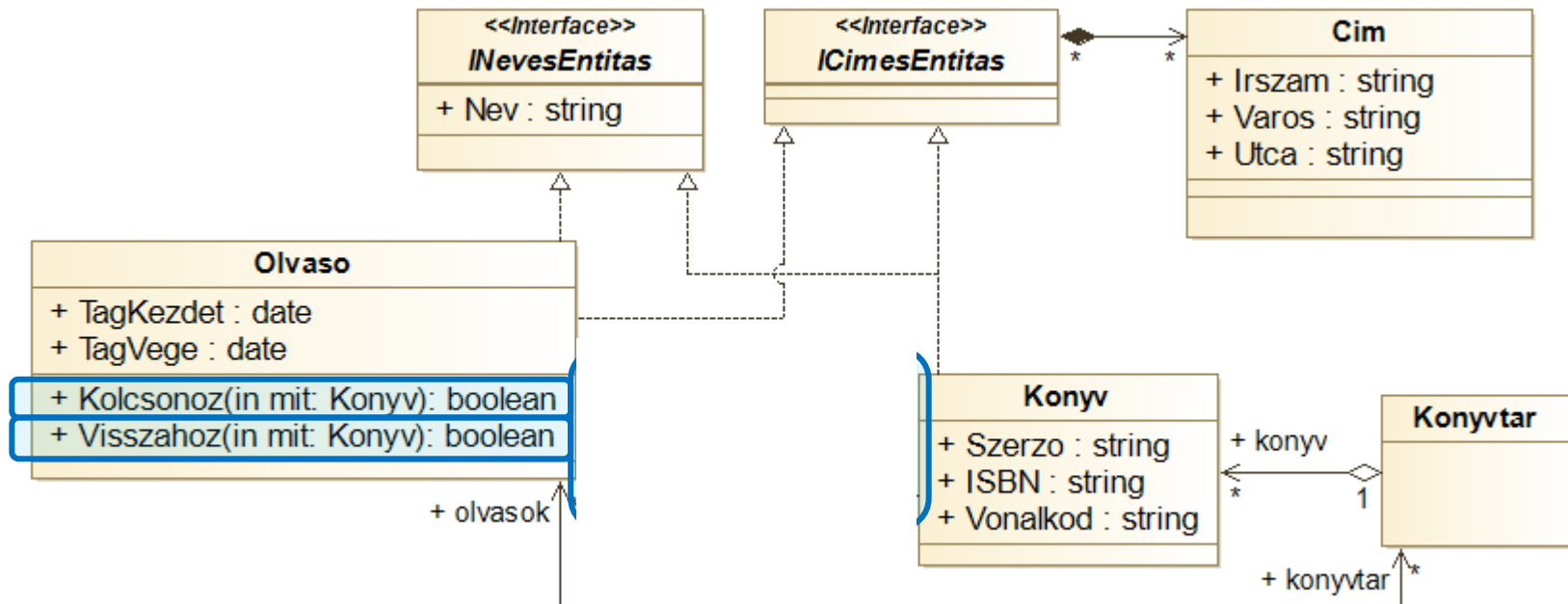
Könyvtár - adatok

- Az olvasó tagságának van kezdete és vége
 - > A könyvtár-olvasó kapcsolatot is érdemes lenne számontartani!



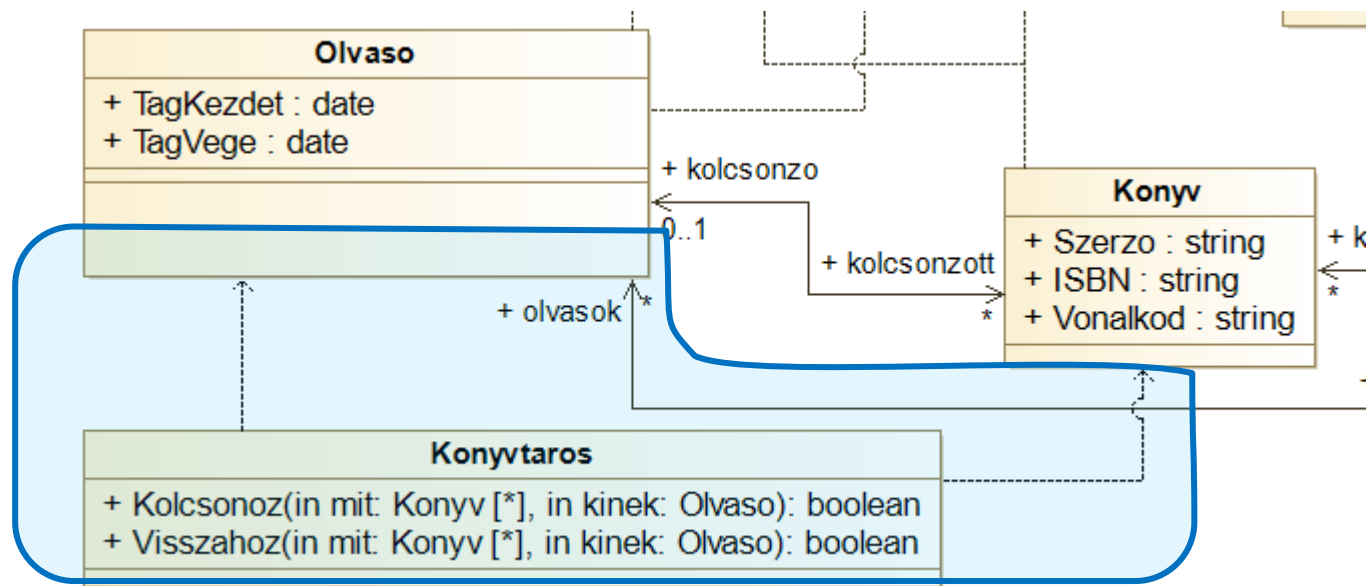
Könyvtár - kölcsönzés

- Az olvasó kölcsönözhet könyveket.
 - > Meg kell jegyezni, kinél van az adott könyv
 - > Tartalmazás vagy asszociáció?
 - > Vissza is kell tudni hozni



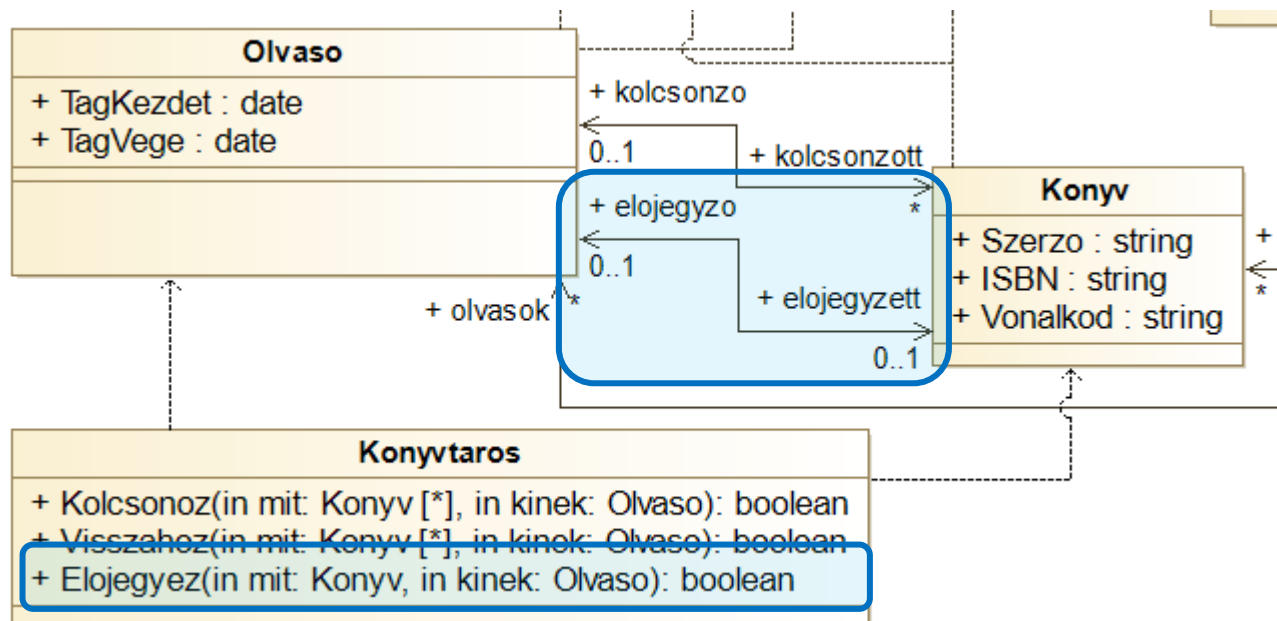
Könyvtár – könyvtáros

- Az olvasó kölcsönözhet könyveket.
 - > Az olvasó kölcsönzi?
 - > ... vagy a könyvtár?
 - > ... vagy a könyvtáros?



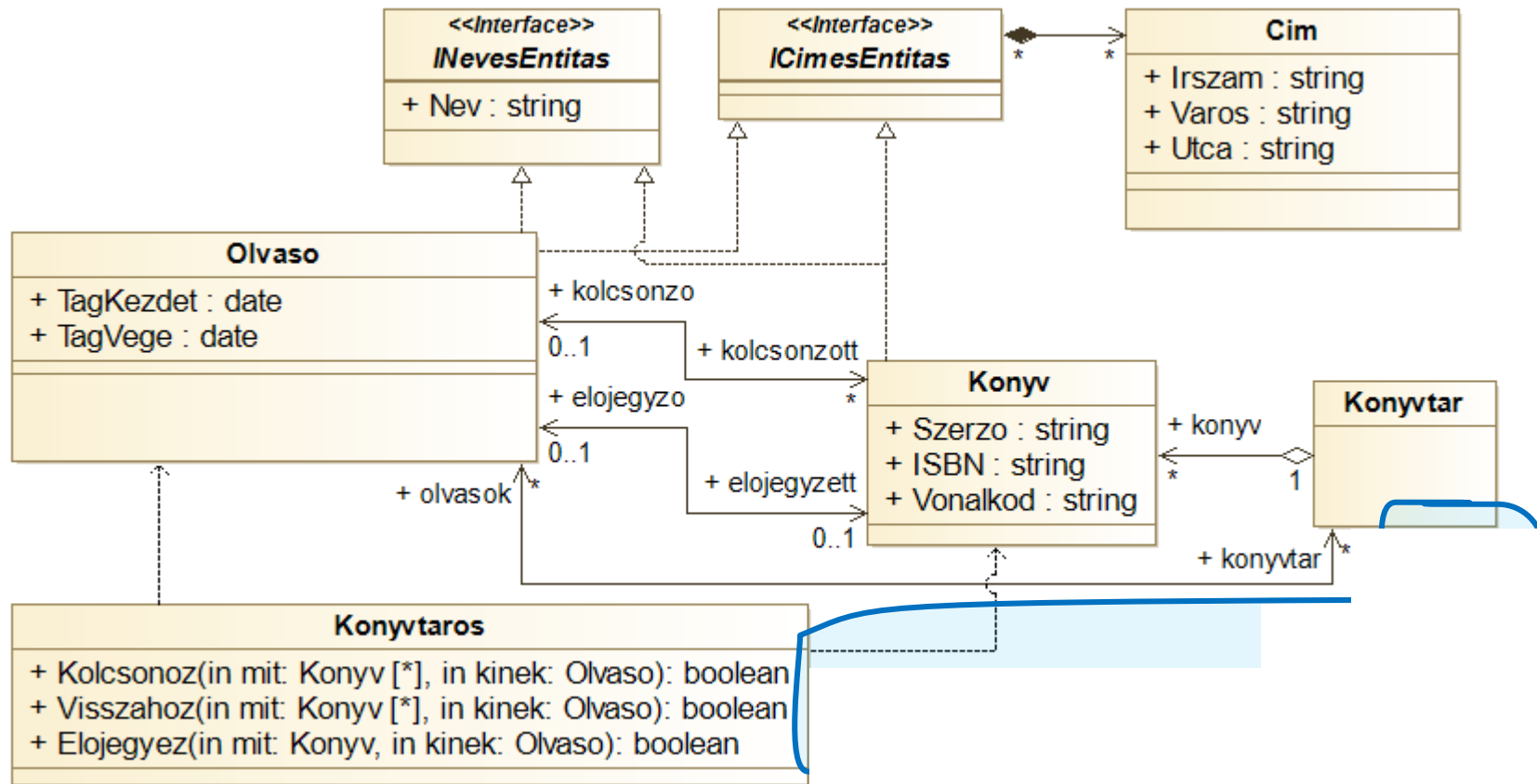
Könyvtár - előjegyzés

- Az olvasó elő is jegyezhet könyvet, de csak egyet!
 - > Az előjegyzés műveletet is támogatni kell
 - > Kiemeljük a könyv-olvasó adatpárost külön típusba?



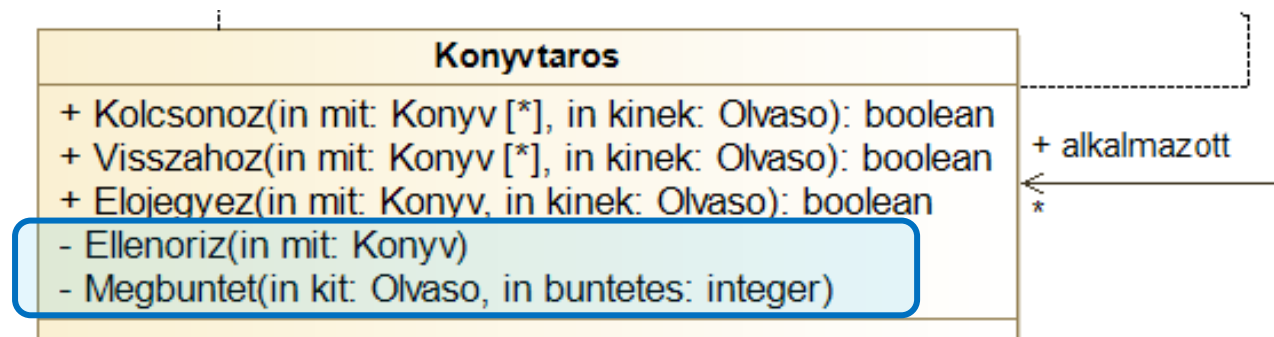
Könyvtár – könyvtáros

- A könyvtáros munkahelye a könyvtár



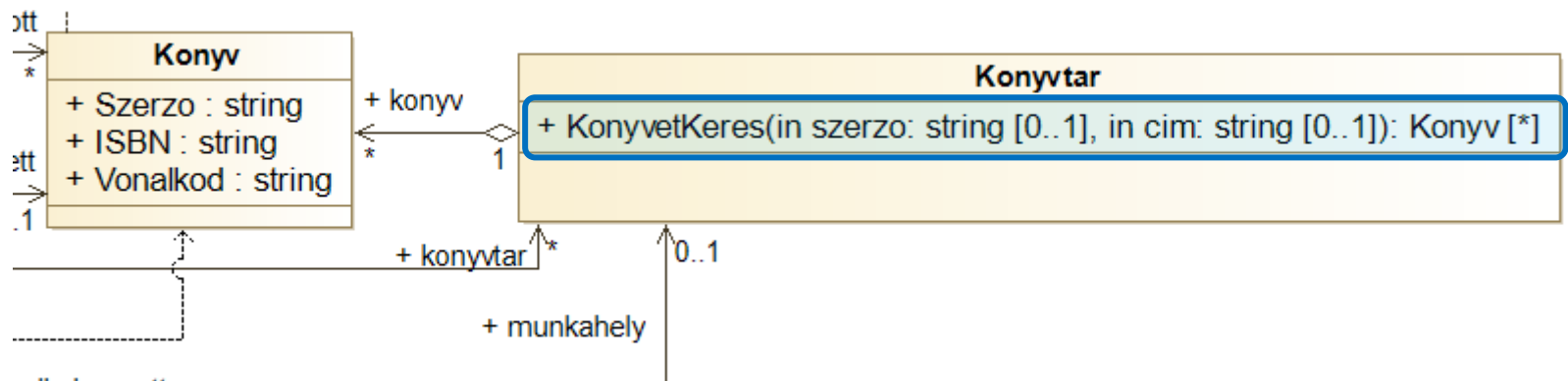
Könyvtár – könyvtáros

- A könyvtáros tudja a könyvet ellenőrizni és büntetést kiszabni, ha szükséges.
 - > Publikus művelet?



Könyvtár - keresés

- Lehesse könyvet keresni a könyvtárban szerző és cím szerint (mindkét parameter elhagyható)!
 - > Mi legyen a visszatérési érték?
 - > Az olvasó, a könyvtáros, vagy a könyvtár keres?



Szekvenciadiagram

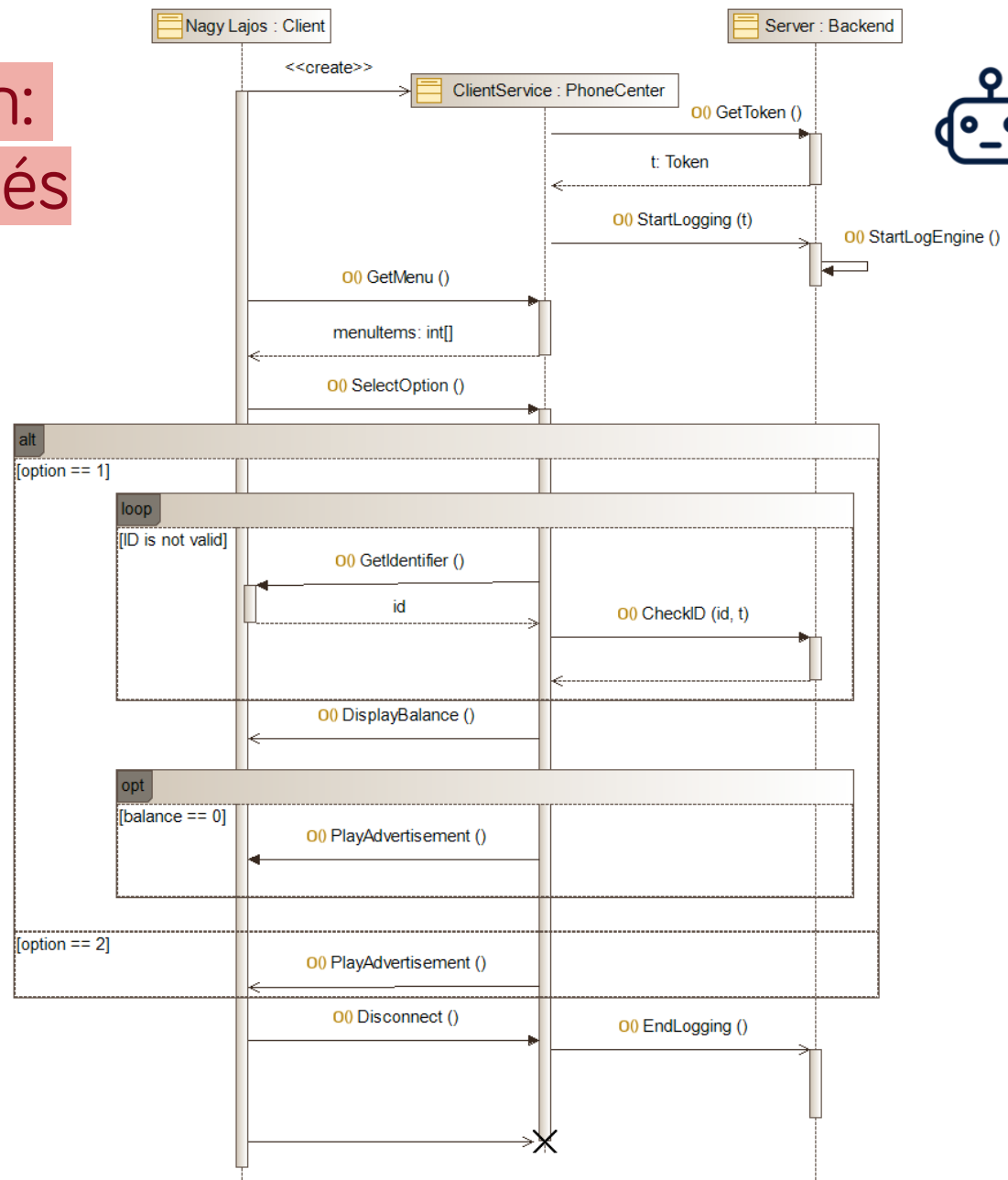
Struktúra vs. viselkedés

- Osztálydiagram
 - > Struktúra, adatszerkezet
 - > Metódusok
 - > Statikus
- Hogyan írjuk le a dinamikus működést?
 - > Szekvenciadiagram
 - > Állapotgép
 - > Aktivitásdiagram
 - > Használati eset diagram

Szekvenciadiagram

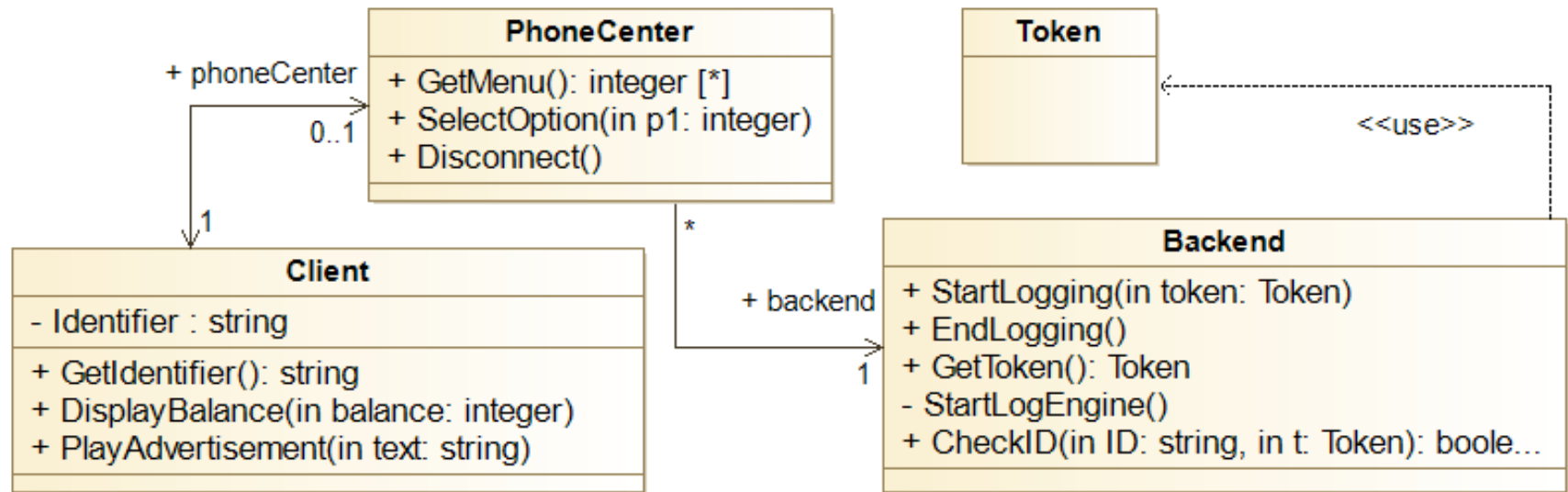
- A folyamatok, műveletek sorrendjére fókuszál
 - > Üzenetek, válaszüzenetek
 - > Interakciók
 - > Információcsere
- Rendszer-, és metódusszinten is alkalmazható
- Egy adott lefutást ábrázol
 - > Ugyanazon résztvevők közt több lefutás is lehetséges (több szekvenciadiagram ábrázolható)
- Résztvevők: osztályok példányai és *aktorok*
 - > Nem “független”

Szekvenciadiagram: Telefonos ügyintézés



Osztálydiagram

- Értelmezzük az alábbi struktúrát!



Életvonalak és üzenetek

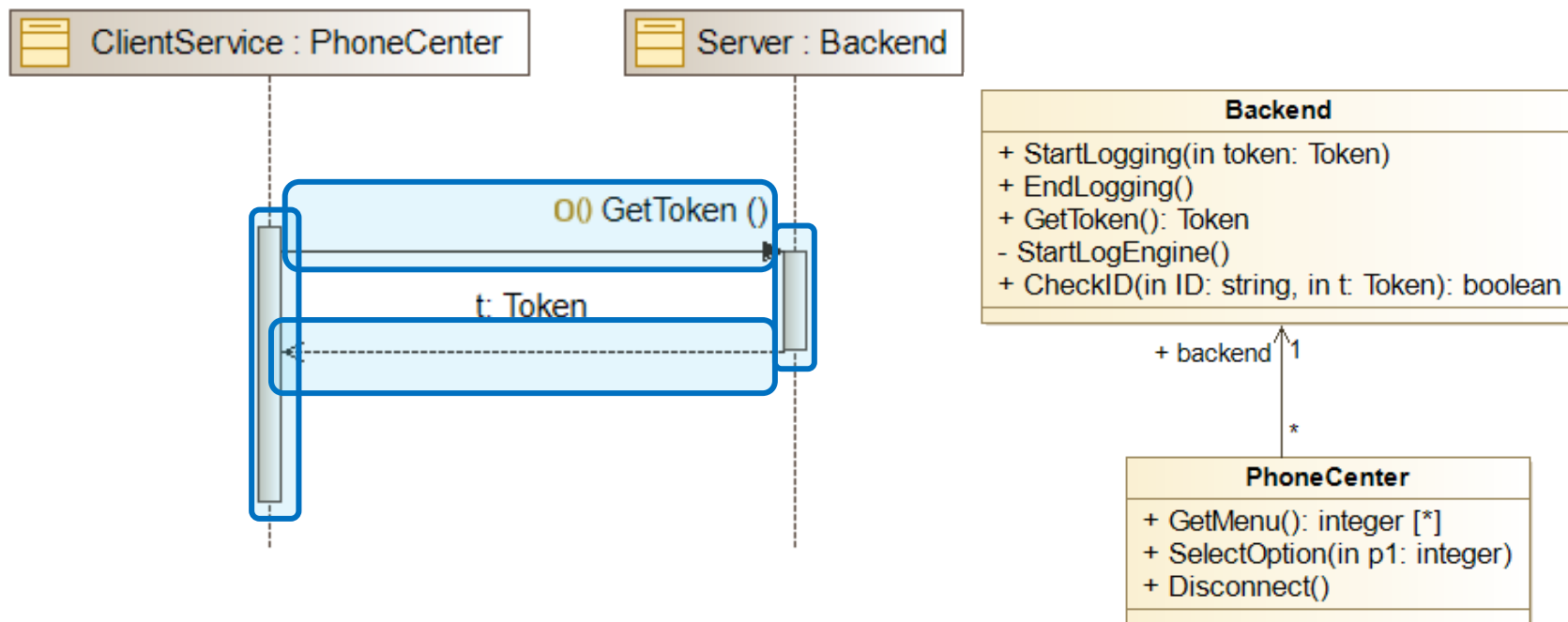
Életvonal

- Vegyünk egy konkrét ügyfelet, aki egy konkrét telefonos szervízzel kommunikál majd a server jóváhagyásával!
 - > Életvonal (Life line)
 - > Nem osztályok szerepelnek, hanem példányok!
 - > A diagram épít az osztálydiagramra!
 - > Idő fentről lefelé telik



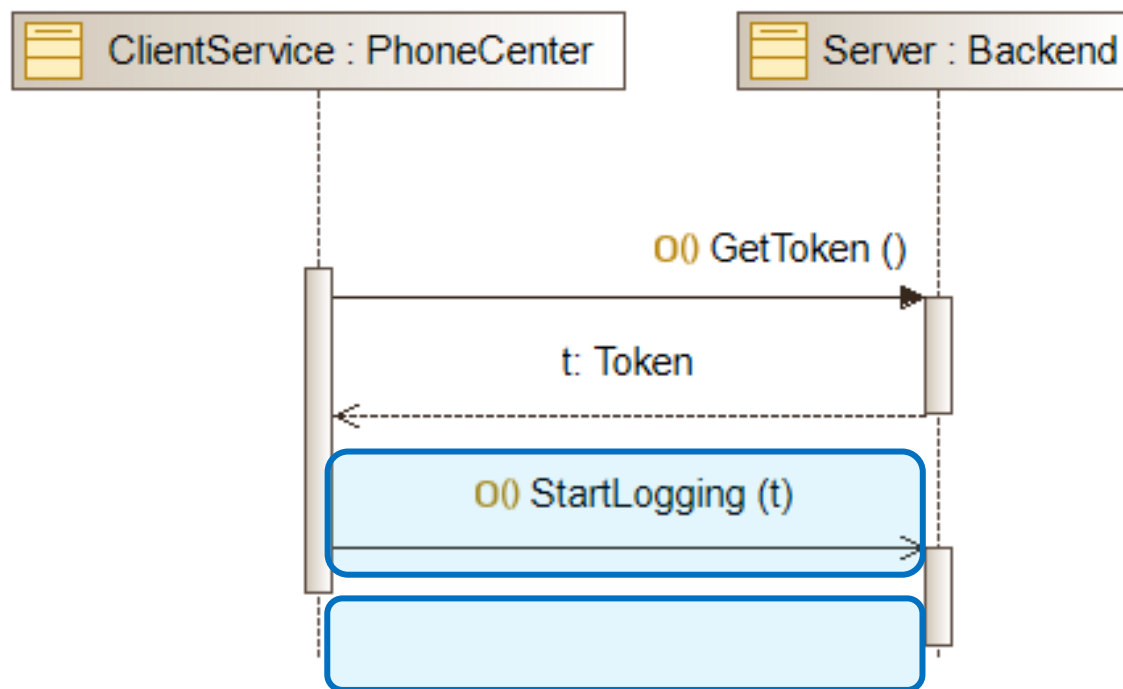
Üzenetek

- A szervíz kérjen el egy token a backendtől!
 - > Szinkron függvényhívás, visszatérési értékkel
 - > Futási specifikáció (execution specification)
 - > Üzenet (message)
 - > Válaszüzenet (return message)



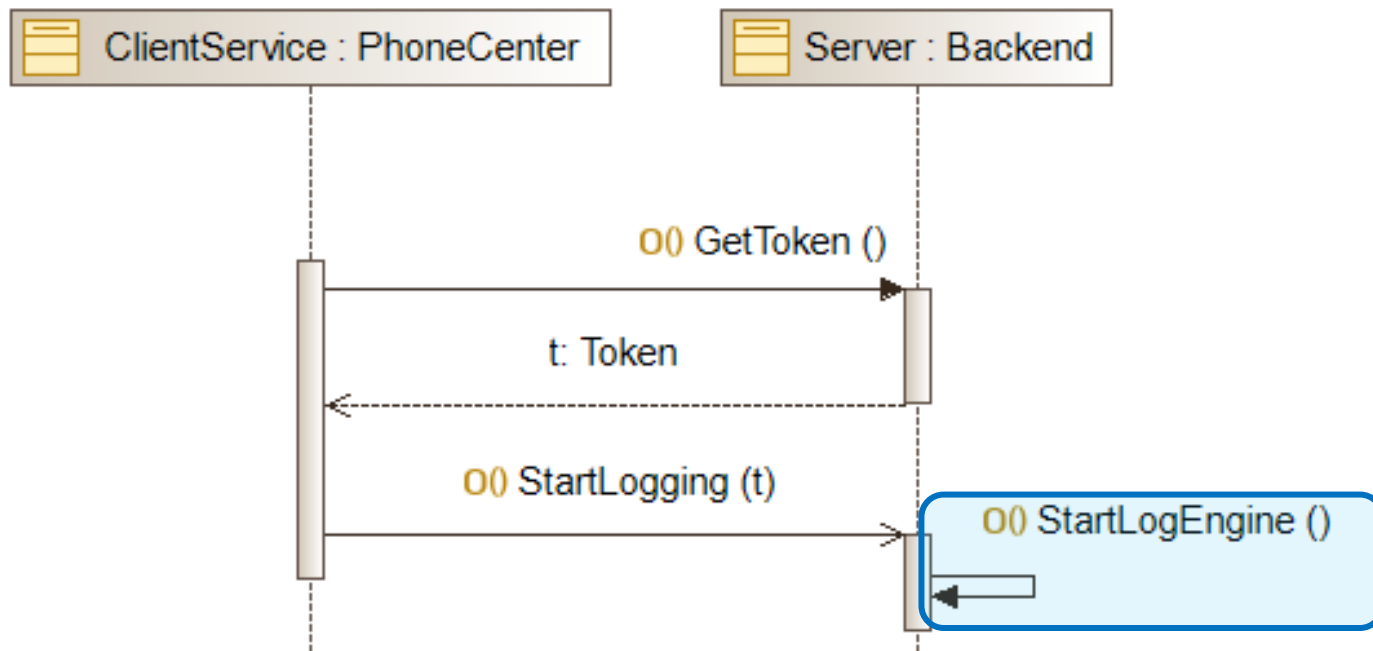
Üzenetek

- A szervíz indítsa el a loggolást a szerveren a token segítségével!
 - > Aszinkron függvényhívás, paraméterrel
 - > Nincs válaszüzenet!



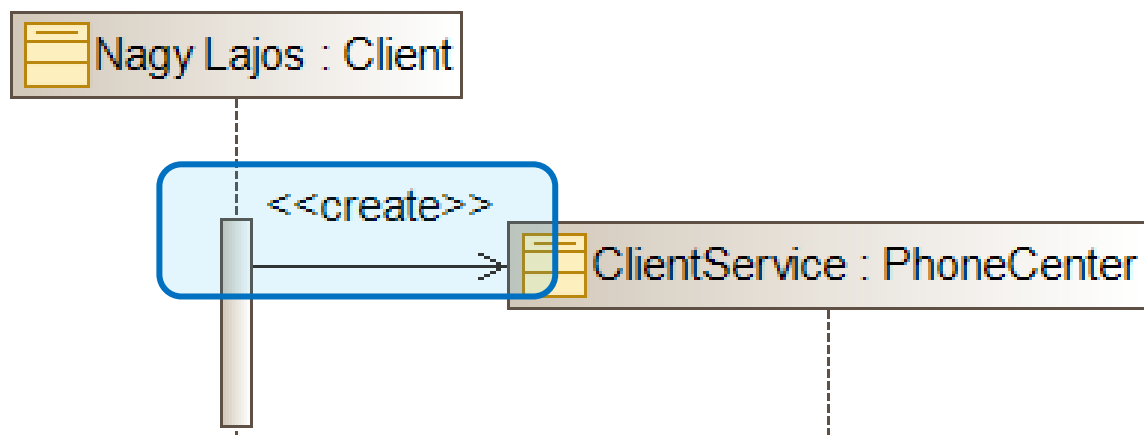
Üzenetek

- A szerver belső folyamatként indítsa el a loggoló metódusát, amikor felkérés jön a szervíztől!



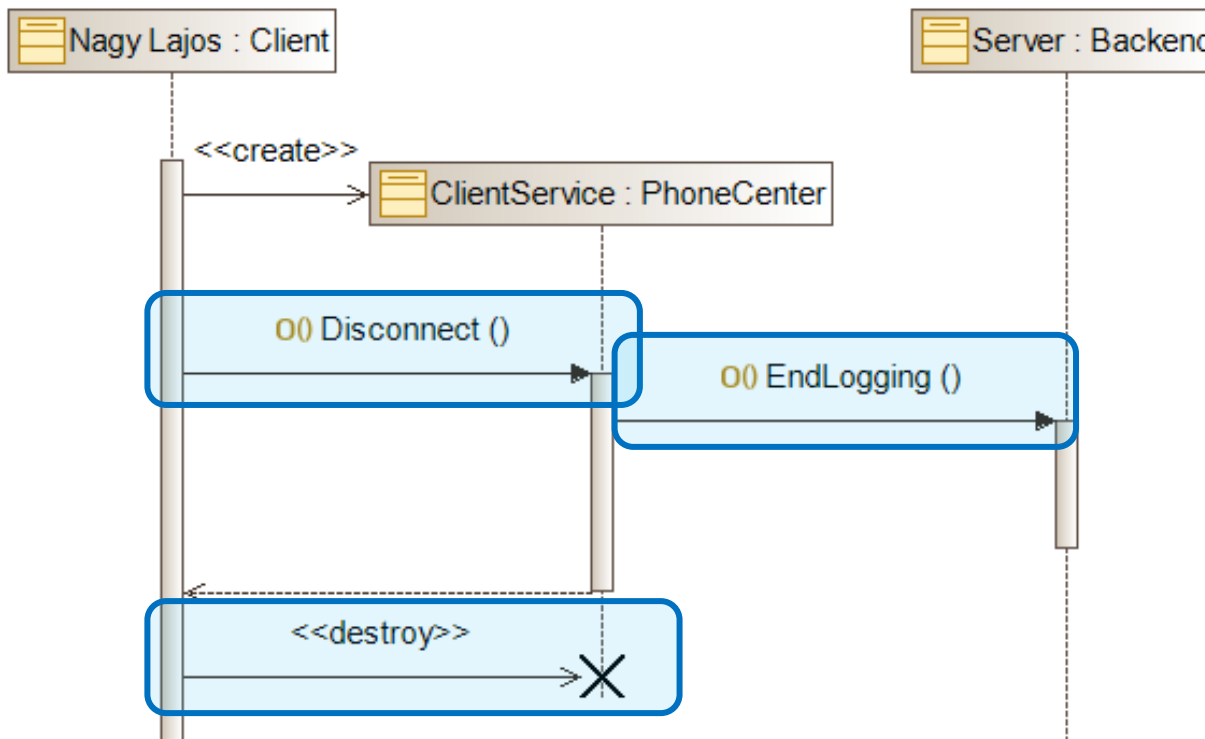
Üzenetek

- A szervíz csak a hívás elején jön létre!
 - > Létrehozási üzenet, konstruktor
 - > “Create” sztereotípia
 - > Később létrejövő életvonal



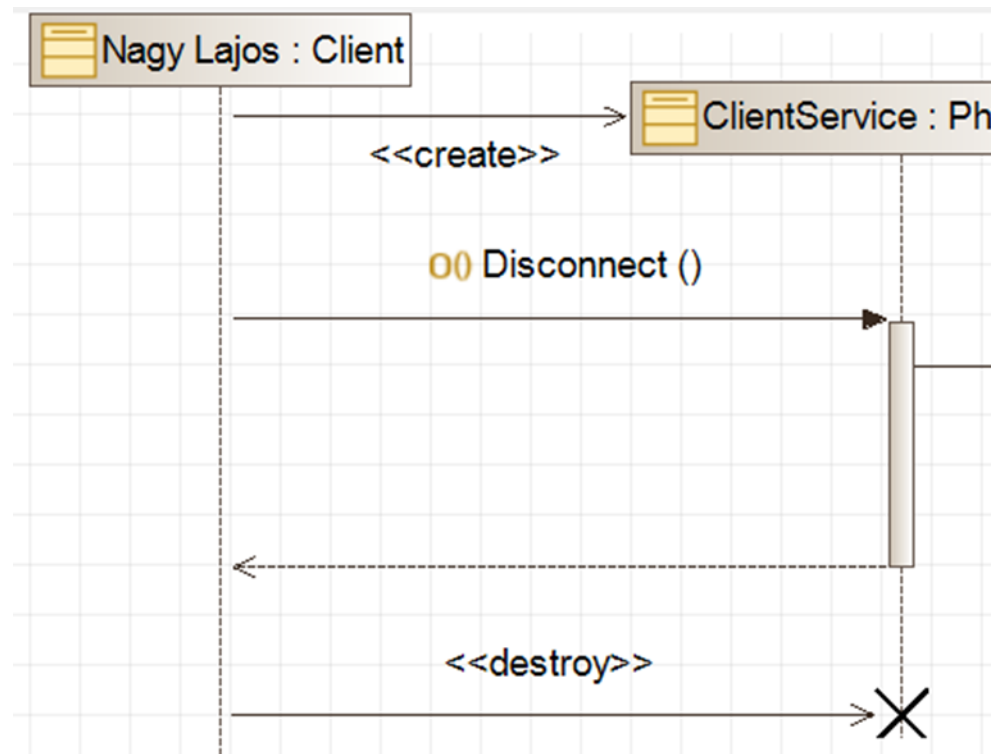
Üzenetek

- Modellezük le a hívás végét
 - > Hívó értesíti a szervízt, ami leállítja a loggolást
 - > Majd megszünteti a szervízt (destruktor)



Életvonalak és üzenetek

- **Életvonal**
 - > [Név]: [típus]
- **Üzenet**
 - > Név([params])
 - > Szinkron/aszinkron
 - > Létrehozás/
megszűntetés
- **Futási/végrehajtási
specifikáció** (execution specification)

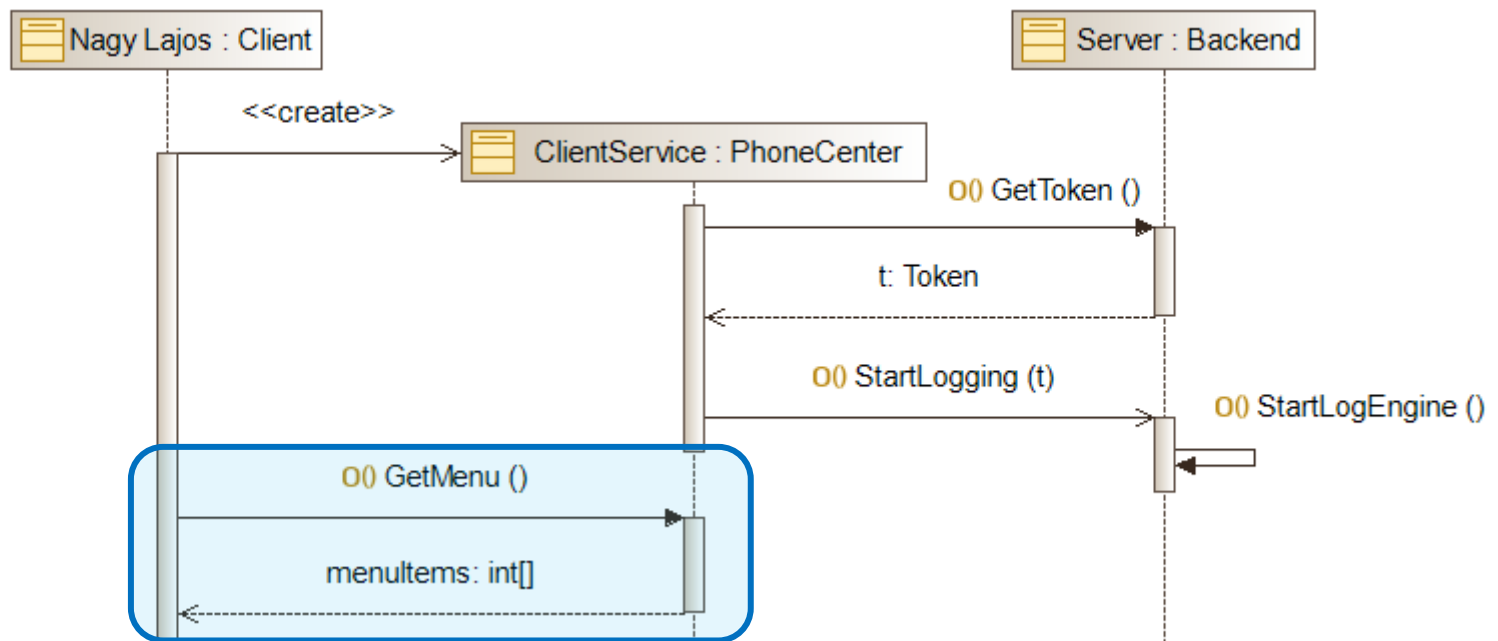


Blokkok

Combined fragments

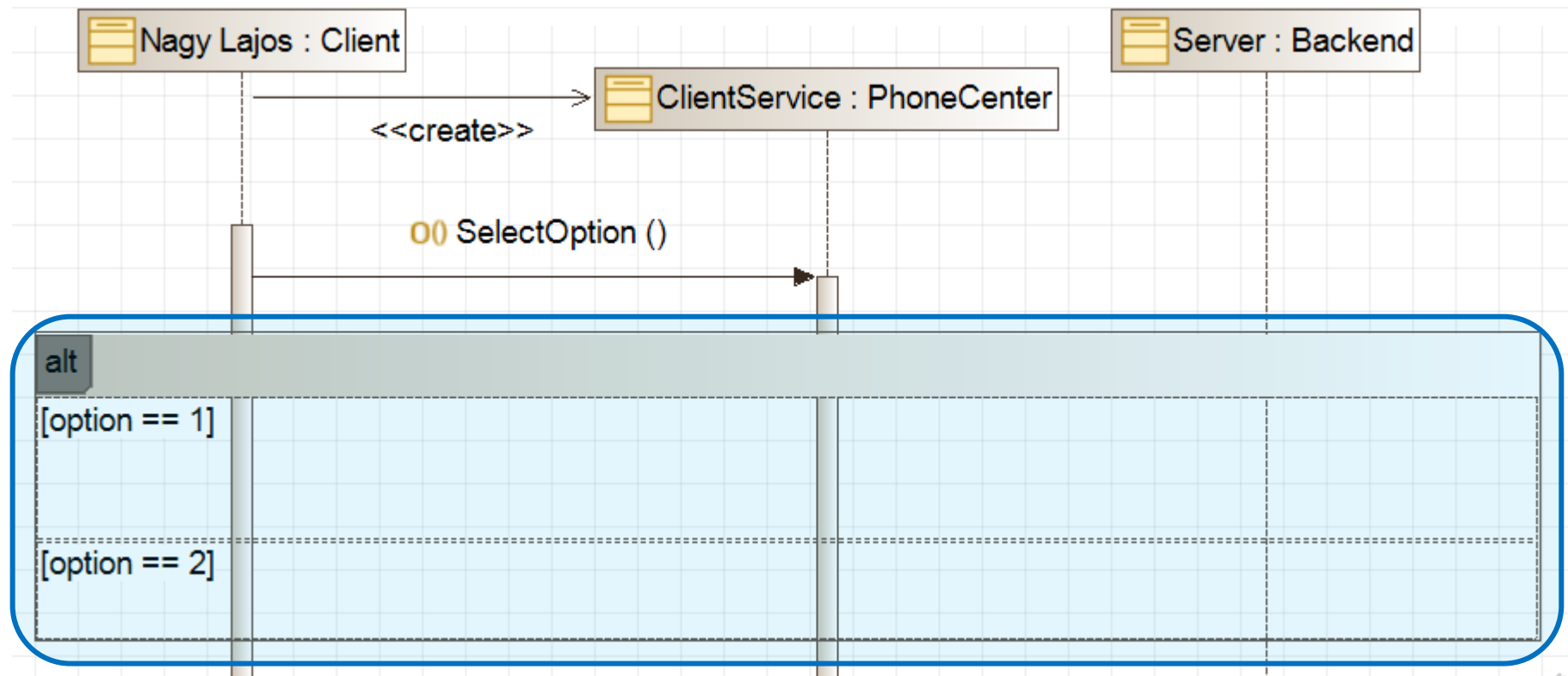
Üzenetek

- A hívó kérhesse le a menüpontokat a szervíztől!



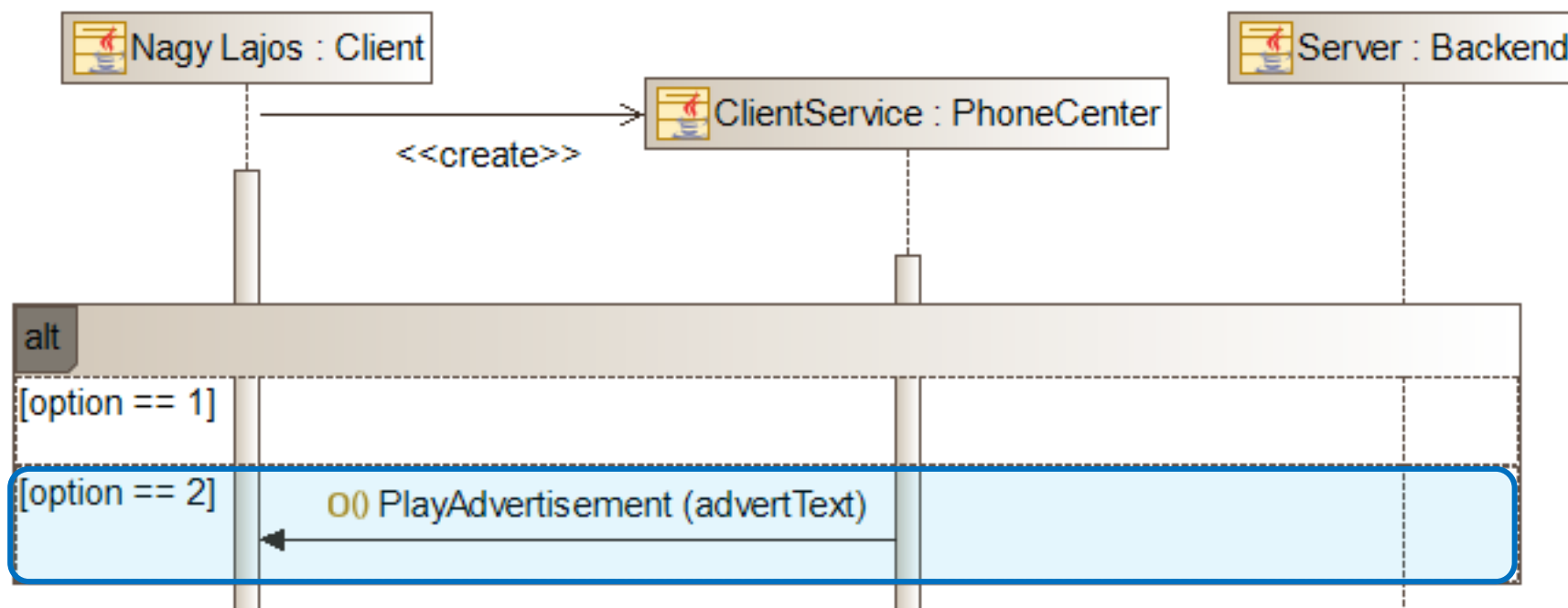
Üzenetek

- Válasszuk szét a műveleteket a hívó választásától függően!
 - > Összetett részlet (combined fragment)
 - > Alternatív blokk (alt) (if – then – else jellegű kompozíció)



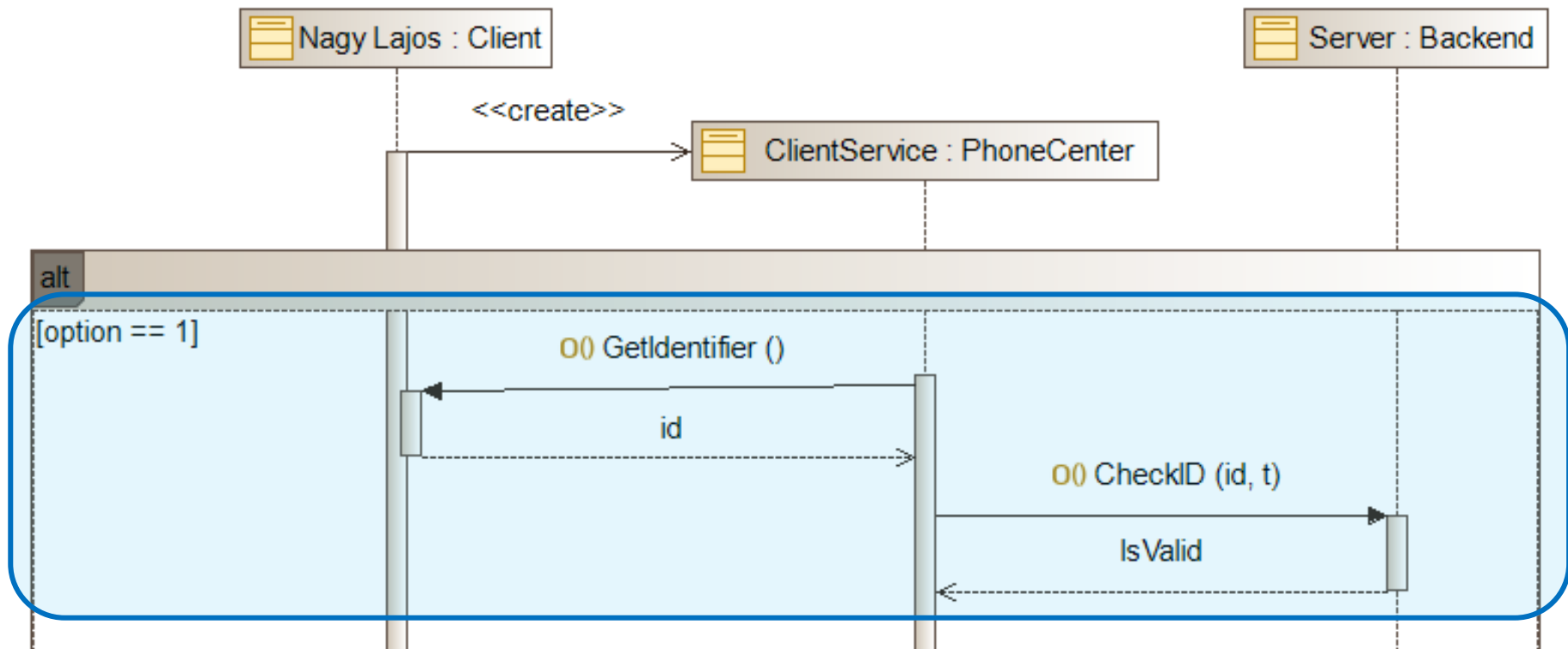
Üzenetek

- A “2-es” menüpont játszon be reklámokat!



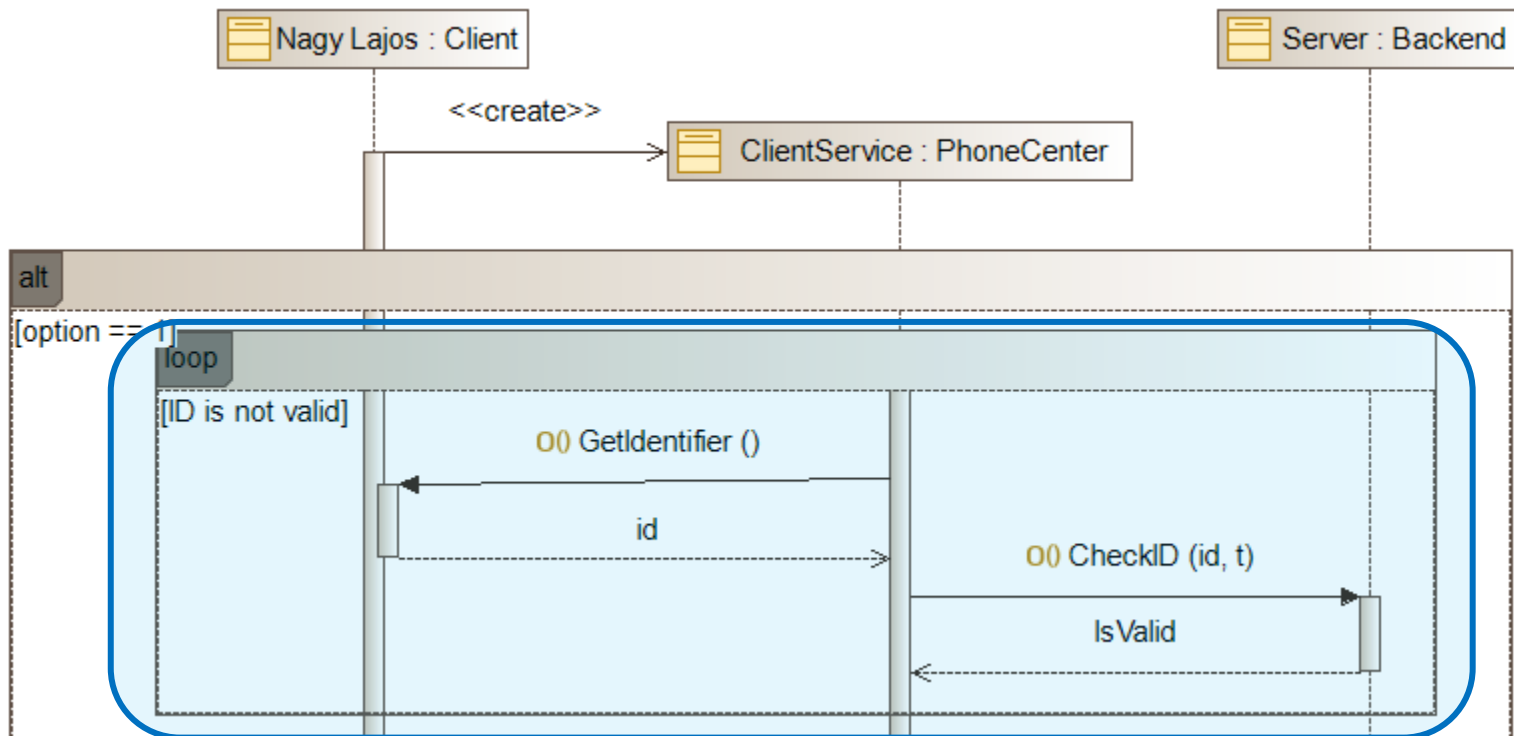
Üzenetek

- Az “1-es” menüpontban kérjük be a hívó azonosítóját és ellenőrizzük is le!



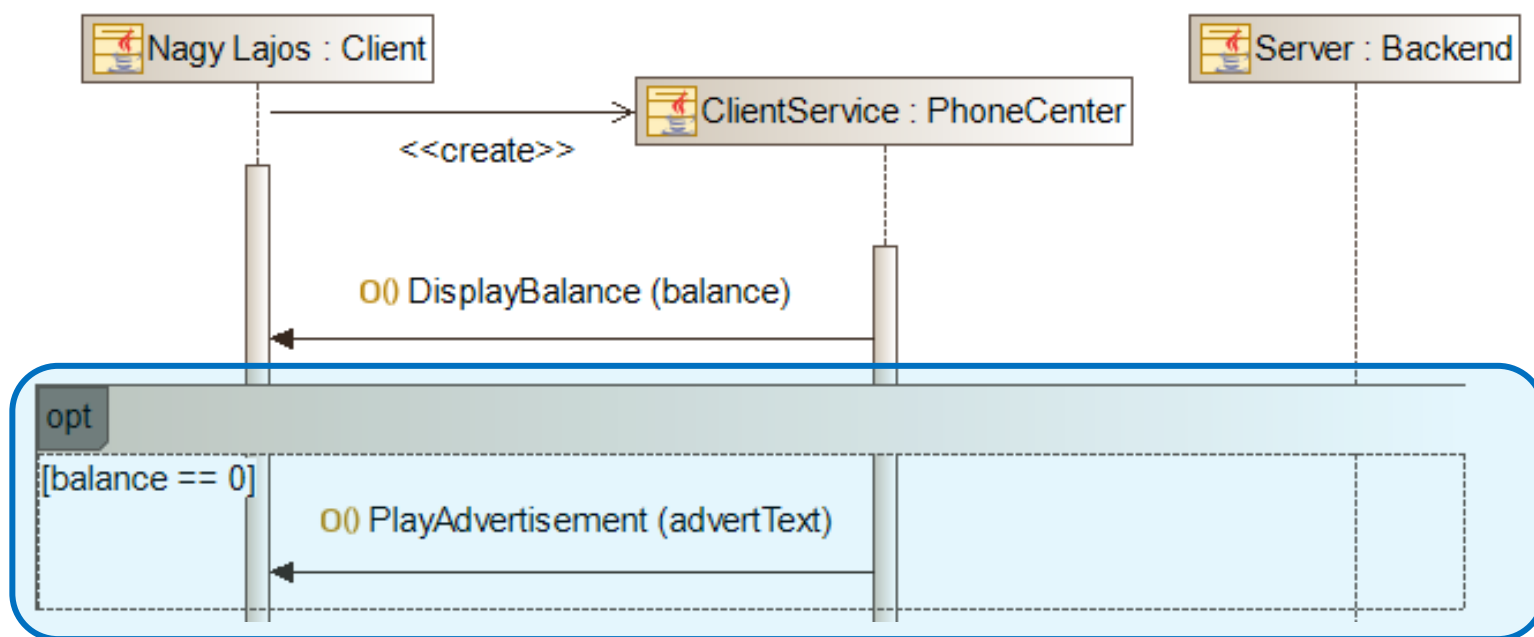
Üzenetek

- Az azonosítót mindaddig újra bekérjük, amíg az nem helyes!
 - > **Ciklus blokk (loop)** (feltétel szerinti ismételés)



Üzenetek

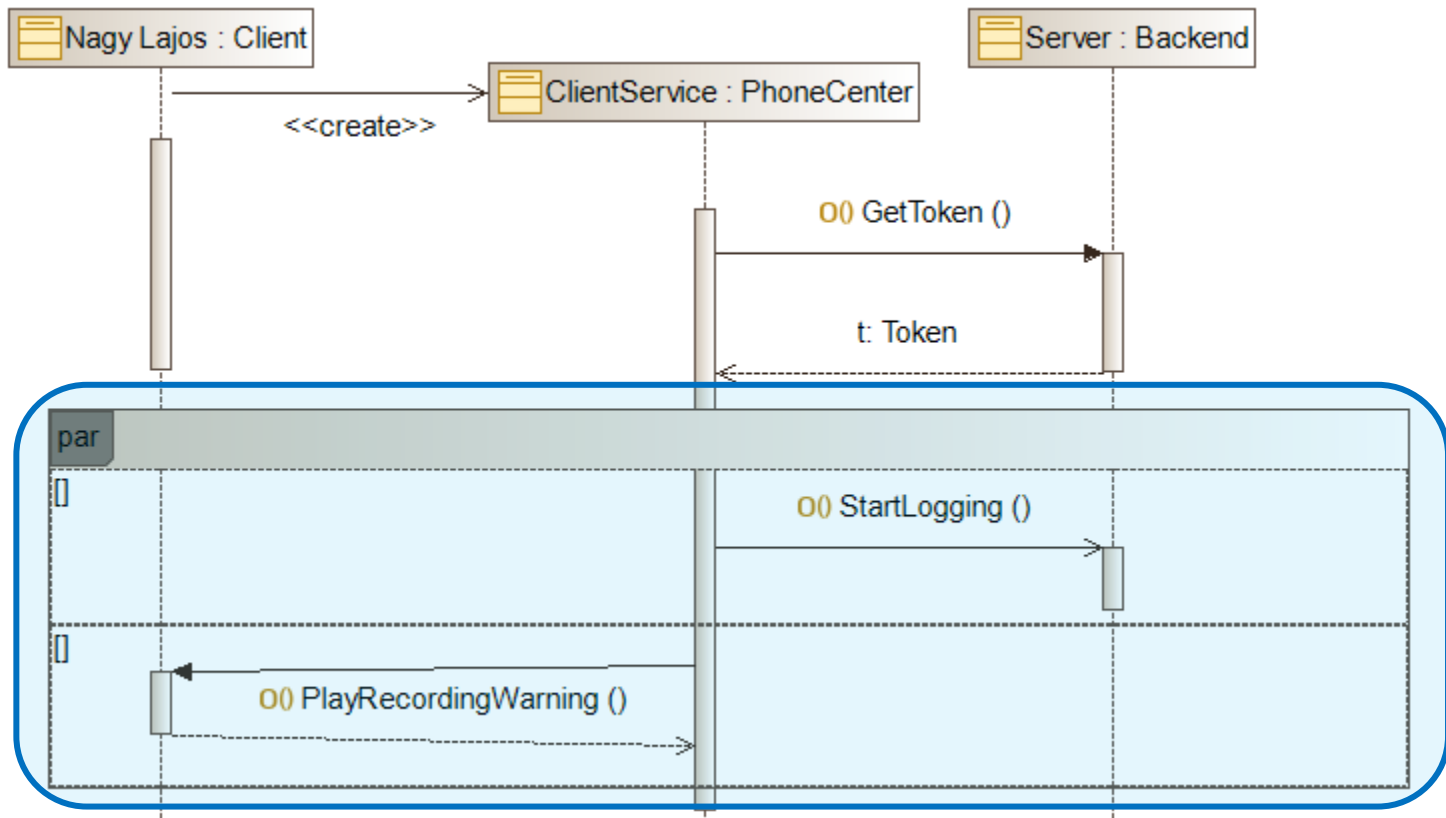
- Mutassuk meg a hívónak az egyenlegét és ha üres, automatikusan játszunk le neki reklámot!
 - > Opcionális blokk (opt) (if else nélkül)



Párhuzamosság

- A beszélgetés kezdetén a loggolás elindításával egyidőben játszunk le egy figyelmeztetést a hívónak, hogy rögzítjük majd a beszélgetést!

> Párhuzamos blokk (par)



Blokkok

- Kombinált részletek (Combined fragments)
 - > Alt: Feltételes elágazás (if – else if – else szerkezet)
 - > Loop: Feltétel szerinti előltesztelő ciklus
 - > Opt: Opcionális utasítások (else ág nélküli if)
 - > Par: Párhuzamosan végrehajtandó utasítások
 - > Seq, Critical, Assert, ... (nem tanuljuk)

Összefoglalás

- Modellezés
 - > Statikus nézet, szerkezet, interfészek -> Osztálydiagram
 - > Metódusok egymás utáni lefutása -> Szekvenciadiagram
- Szekvenciadiagram
 - > Életvonalak
 - > Üzenetek
 - > Blokkok

Köszönöm a figyelmet!

Szoftvertechnológia és -technikák

4. Előadás

Aktivitásdiagram, Állapotgép

Osztálydiagram és szekvenciadiagram

Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])(: visszatérési érték)

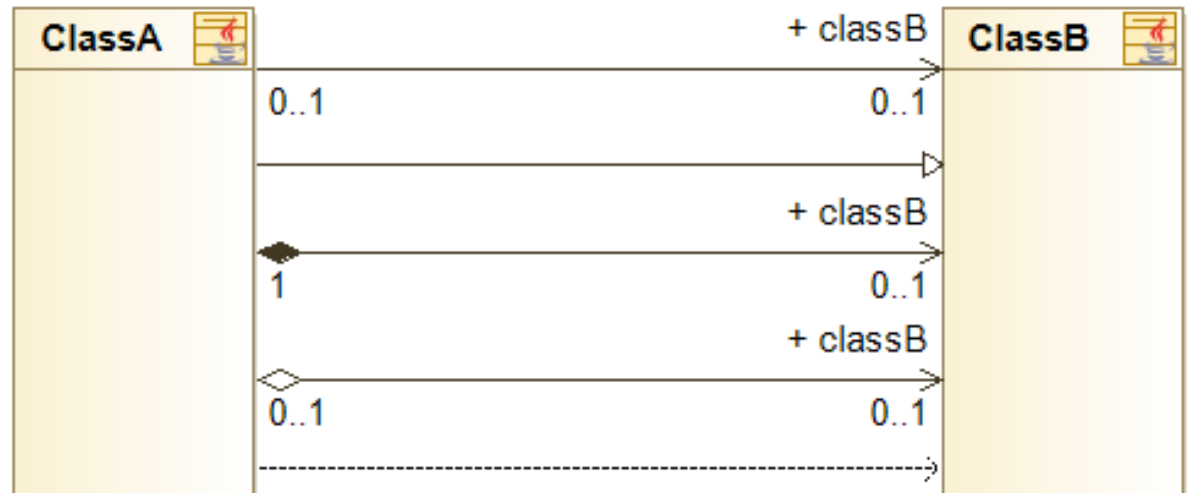
- > Láthatóság

- Public: +
 - Private: -
 - Protected: #
 - Package: ~

Kapcsolatok

- Kapcsolat típusok

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



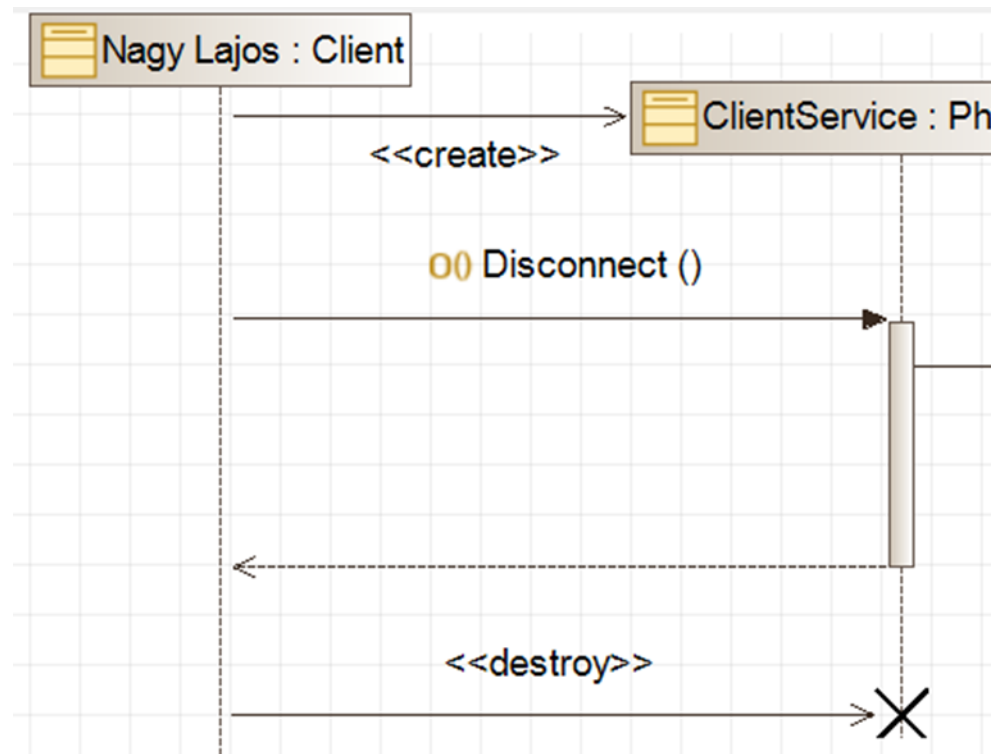
- Multiplicitás

- Navigálás

- > Szerepnév
- > Navigálhatóság

Életvonalak és üzenetek

- Életvonal
 - > [Név]: [típus]
- Üzenet
 - > Név([params])
 - > Szinkron/aszinkron
 - > Létrehozás/
megszűntetés
- Futási/végrehajtási
specifikáció (execution specification)



Blokkok

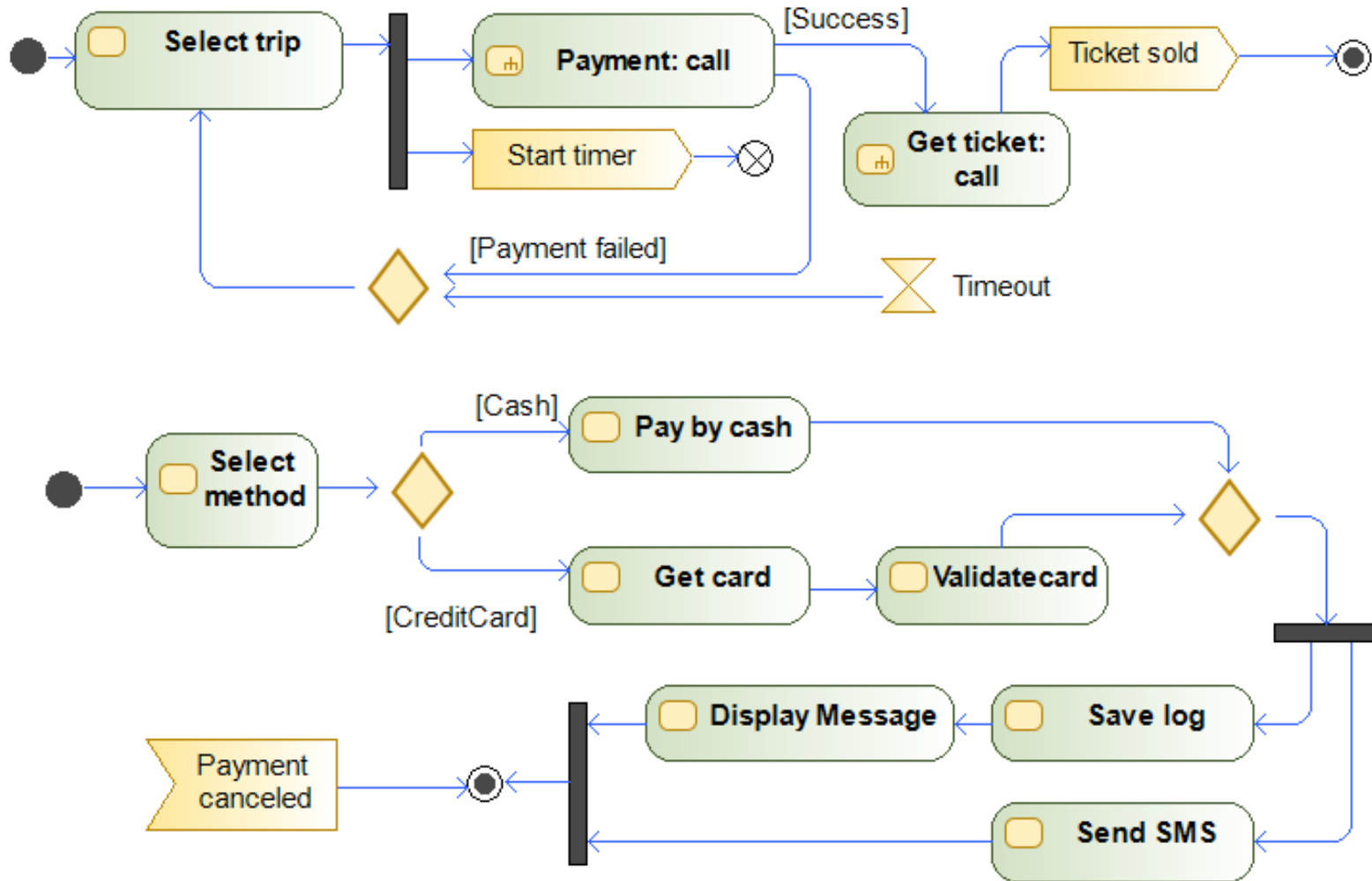
- Kombinált részletek (Combined fragments)
 - > Alt: Feltételes elágazás (if – else if – else szerkezet)
 - > Loop: Feltétel szerinti előltesztelős ciklus
 - > Opt: Opcionális utasítások (else ág nélküli if)
 - > Par: Párhuzamosan végrehajtandó utasítások
 - > Seq, Critical, Assert, ... (nem tanuljuk)

Aktivitás diagram

Aktivitásdiagram

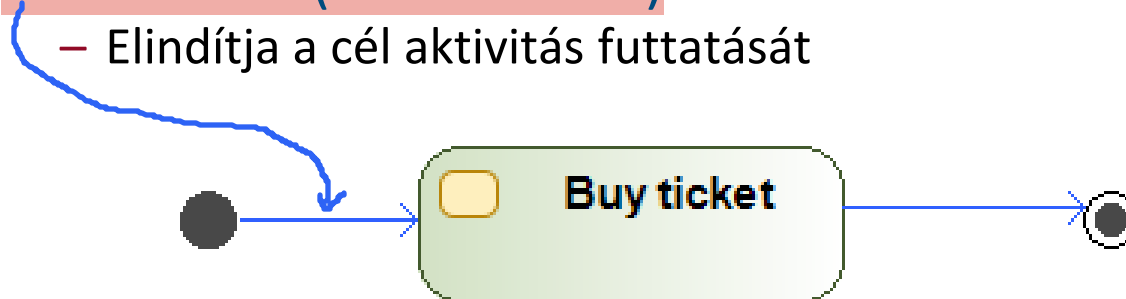
- Struktúra – osztálydiagram
- Műveletek sorrendje – szekvenciadiagram
 - > Szekvenciális végrehajtás
 - > Jellemzően függvényhívások
- Munkafolyamatok leírása – Aktivitás diagram
 - > Workflow, control flow
 - > Elágazások, párhuzamos ágak, feltételek
 - > Fókusz a funkciókon, a folyamatokon
 - > Magasabb szintű, több használati esetet is összefoghat
 - > <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/>

Aktivitásdiagram: BKV jegyvásárlás



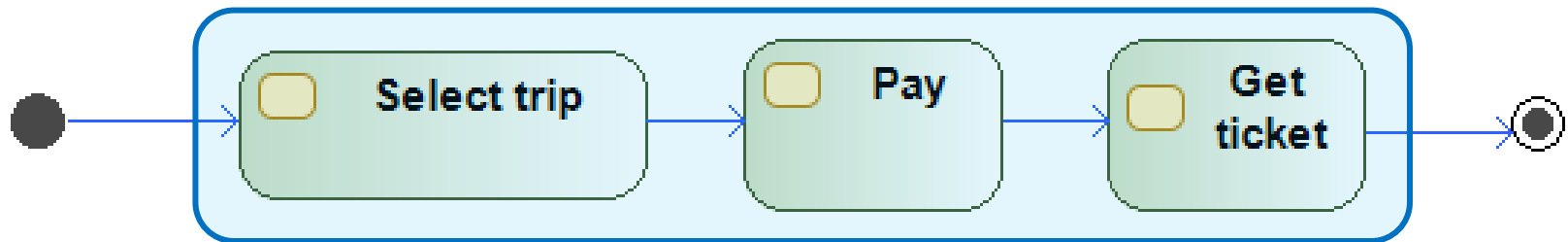
Jegyvásárlás

- Modellezzük a jegyvásárlást!
 - > **Kezdő csomópont (initial node)**
 - Folyamat kezdése
 - Nincs bemenő él
 - Több kezdőpont: párhuzamos kezdés
 - > **Aktivitás (Activity)**
 - > **Végső csomópont (final node)**
 - Folyamat befejezése
 - Nincs kimenő él
 - > **Vezérlési él (control flow)**
 - Elindítja a cél aktivitás futtatását



Több lépésben

- Bontsuk több lépésre a vásárlást!
 - > Lépésről-lépésre finomíthatjuk a folyamat leírását



Őrfeltétel és dekompozíció

- Csak sikeres fizetés esetén adjuk ki a jegyet!



- > Sikeres – sikertelen fizetés

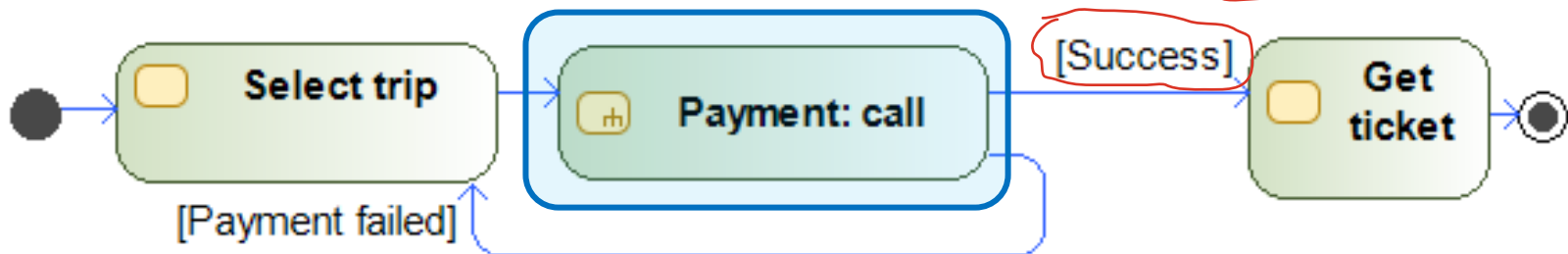
- Őrfeltétel (guard condition):

- Szabályozza melyik vezérlési él válhat aktívvá

- > A fizetési folyamatot bontsuk lépésekre!

- Hívás aktivitás (call activity):

- Hierarchikus dekompozíció



Fizetés

- Lehessen kártyával és készpénzzel fizetni!



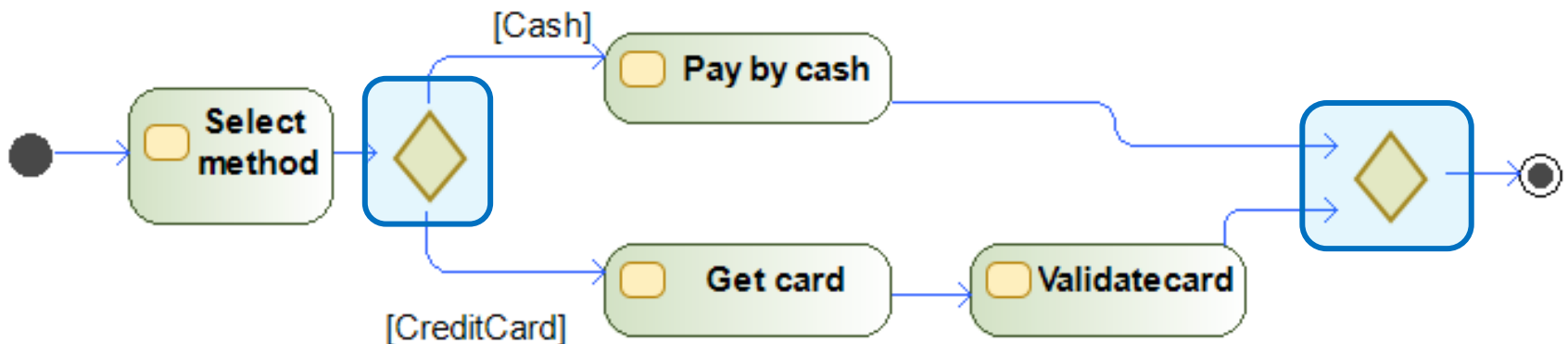
- > **Döntés (decision):**

- Legfeljebb egy kimenő él aktiválható

- > **Összevonás (merge):**

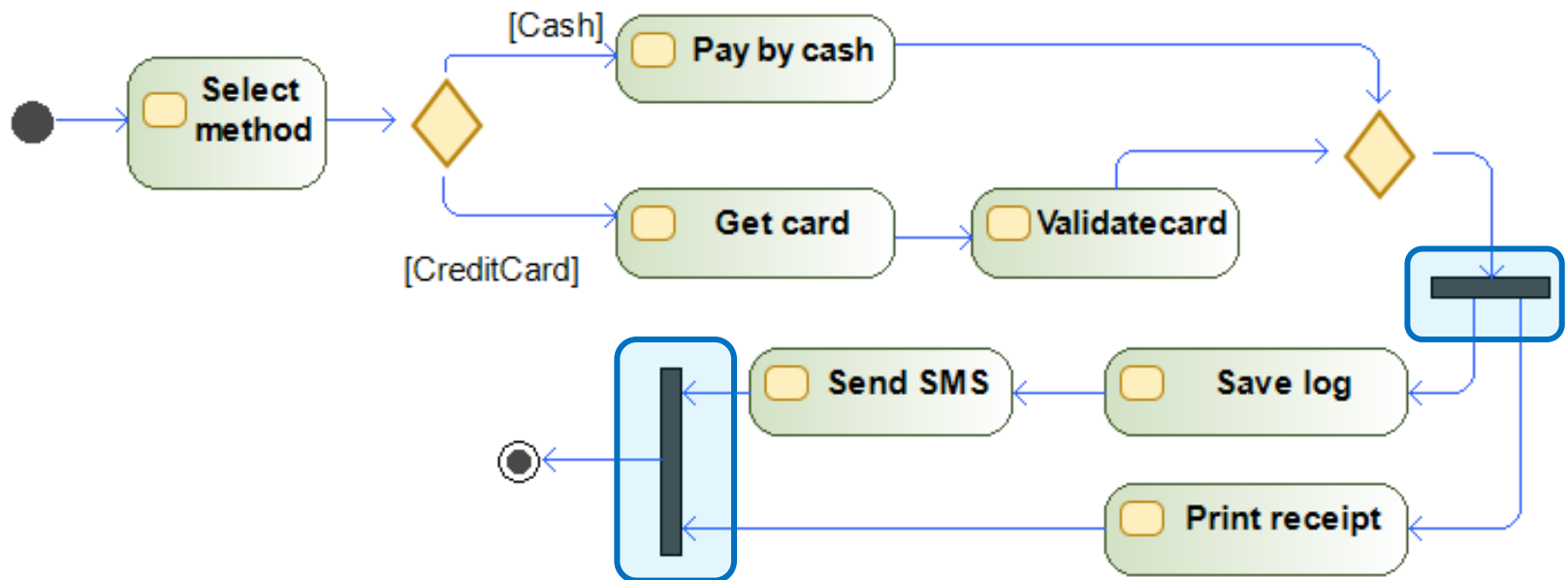
- Összefog több ágot, minden beérkező ágra lefut újra

- > Kártyás fizetésnél ellenőrizzük is le a kártyát!



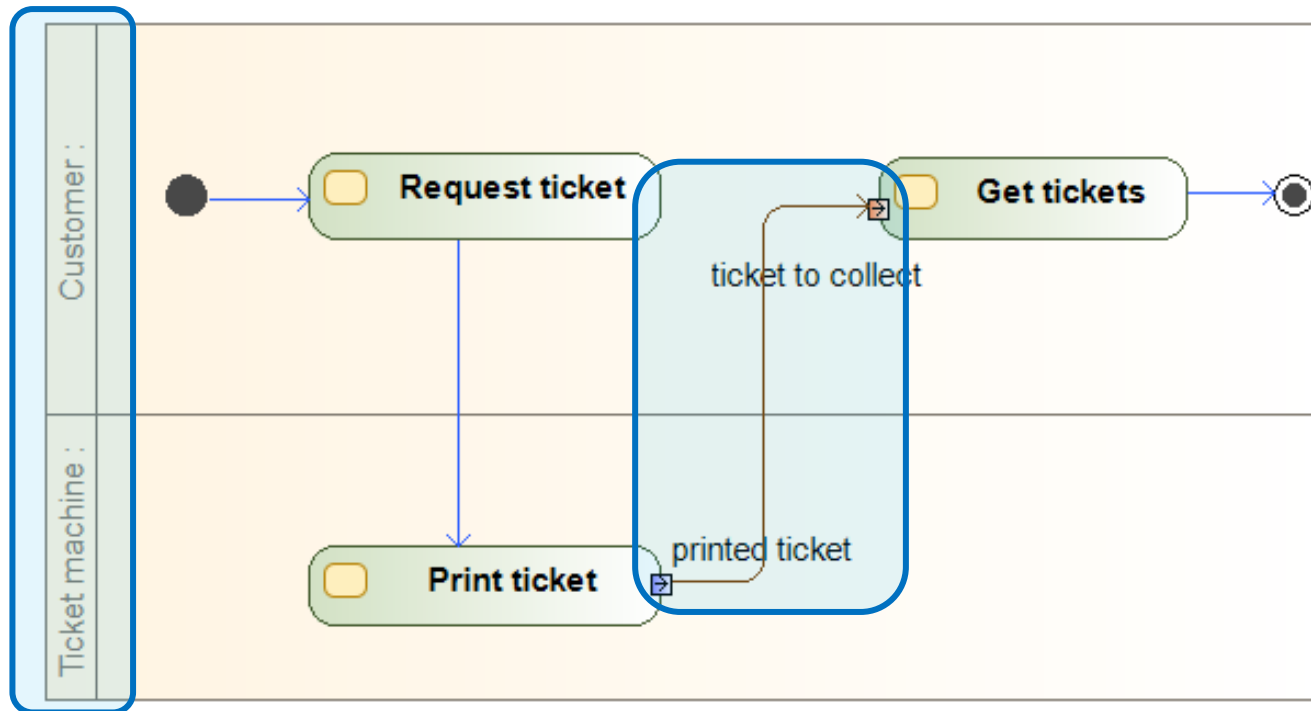
Fizetés

- A fizetés jóváhagyása után
 - > Párhuzamosan:
 - Jegyezzük be a fizetést, küldjünk sms-t!
 - Nyomtassunk ki bizonylatot!
 - > Elágazás (fork), ill. Egyesítés (join)
 - Párhuzamos futás, szinkronizáció



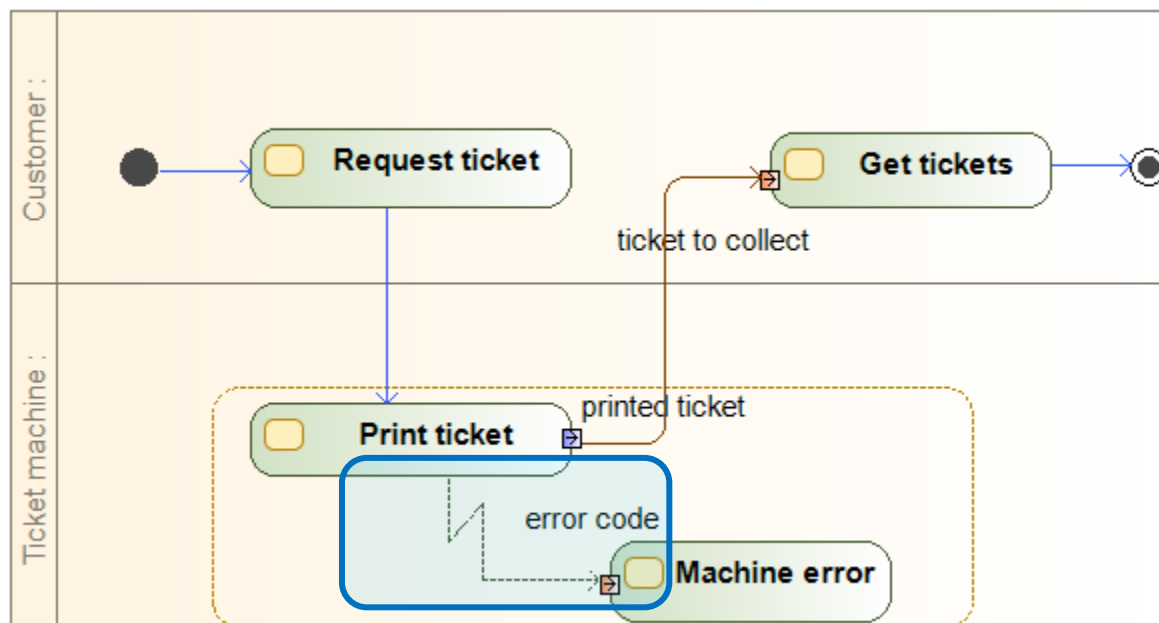
Jegy átvétele

- A jegyet az automata nyomtatja
 - > Két résztvevő rendszer (szerepkör) van a folyamatban!
 - Partíció (partition)
 - > A jegy átadása adat átadás, nem vezérlés!
 - Objektum él (object flow)
 - Objektum csomópont (object node)
 - Tüske (pin) – az objektum csomópont átadásának jelölésére, egy fajta objektum



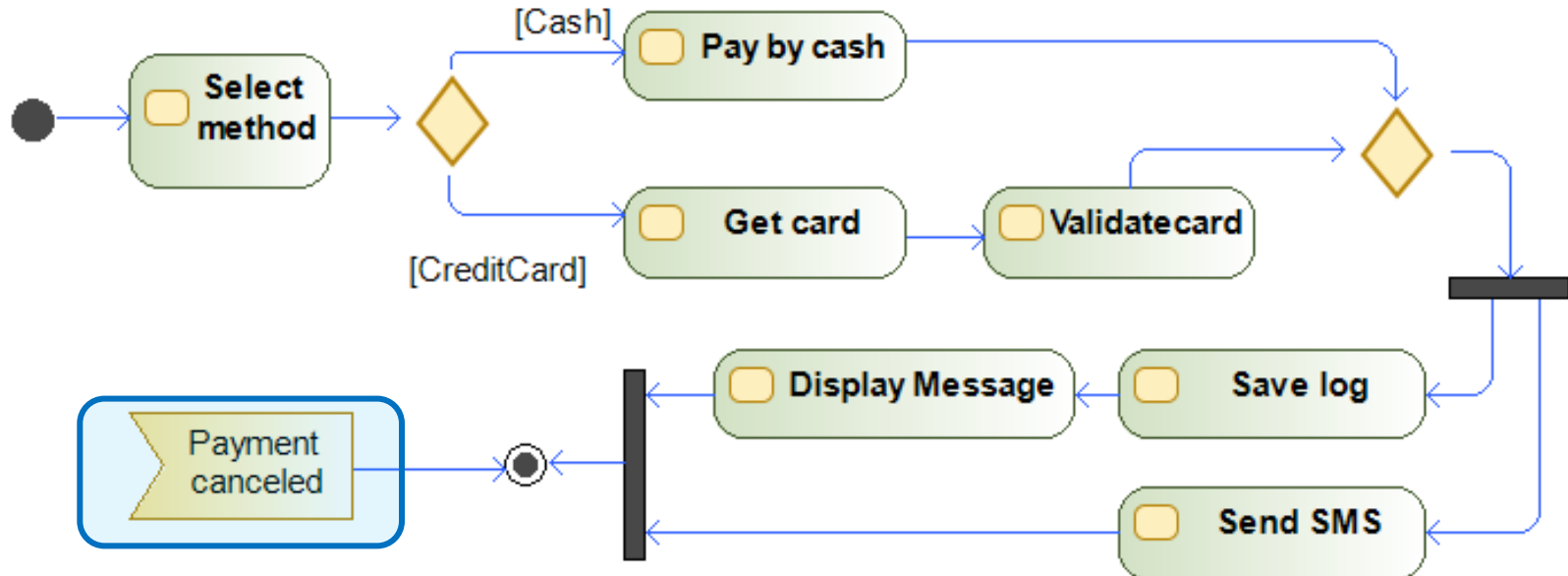
Hibakezelés

- Ha a nyomtatás során hiba adódik, kezelni kell
 - > Kivételkezelés, adatátadással
 - > **Kivétel (exception)**
 - Adott régióban történő váratlan megszakításhoz
 - (Modelio nem támogatja a „villám” jelölést) villám = aktivitásdiagram!



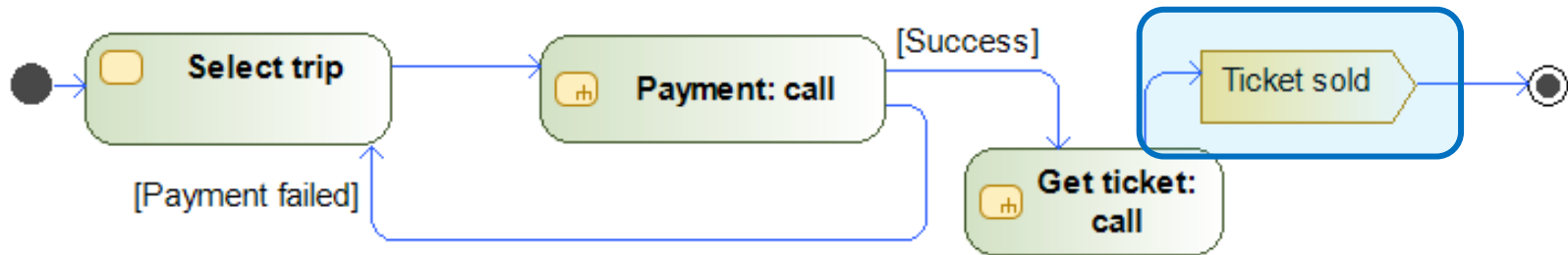
Események

- A fizetés legyen bármikor megszakítható!
 - > Esemény alapú vezérlés: akkor folytatódik a vezérlés, ha az esemény bekövetkezett
 - > Az esemény bárhol kiváltható (mi most nem modellezzük)
 - > Esemény fogadása akció (accept event action)



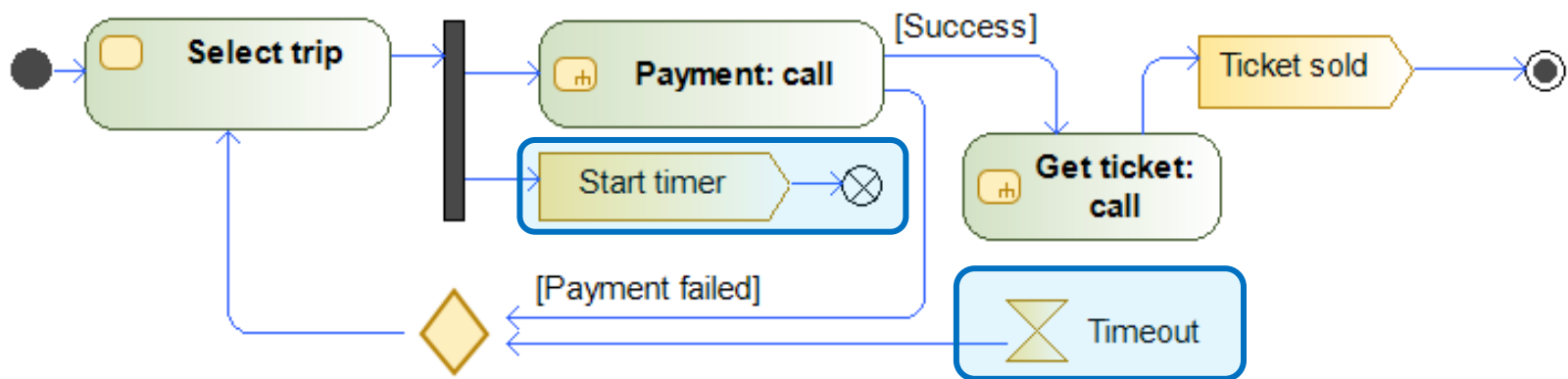
Események

- A sikeres tranzakciót eseményen keresztül publikáljuk!
> Esemény küldése akció (send signal action)



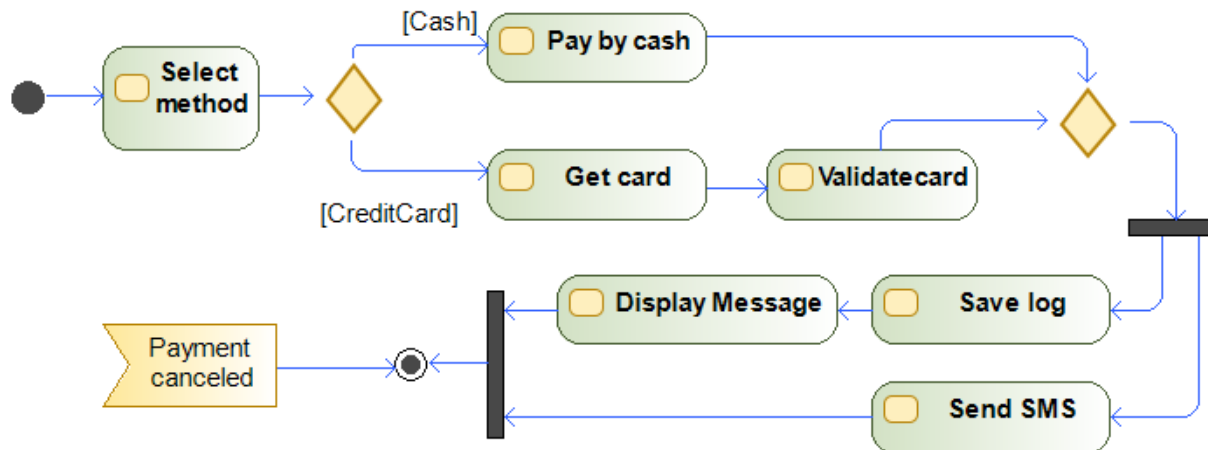
Események

- A tranzakciónak legyen időkerete! Ha az idő lejár, automatikusan váljon sikertelenné a tranzakció!
 - > Időesemény fogadása akció (accept time event action)
- Indítsuk el az időzítőt az elején!
 - > Az időzítő elindítása után az a folyamat ág véget ér, de a teljes folyamat nem!
 - > Folyam vége csomópont (flow final node)



Összefoglalás

- Aktivitás
- Kezdő
- Végcsomópont
 - > Folyam vége
 - > Végő csomópont
- Vezérlési (folyam) él
 - > Őrfeltétel
- Hívás aktivitás
- Döntés – összevonás
- Elágazás – egyesítés
- Partíció
- Objektum, objektum él
- Esemény – fogadás és küldés
- Időesemény



Állapotgép

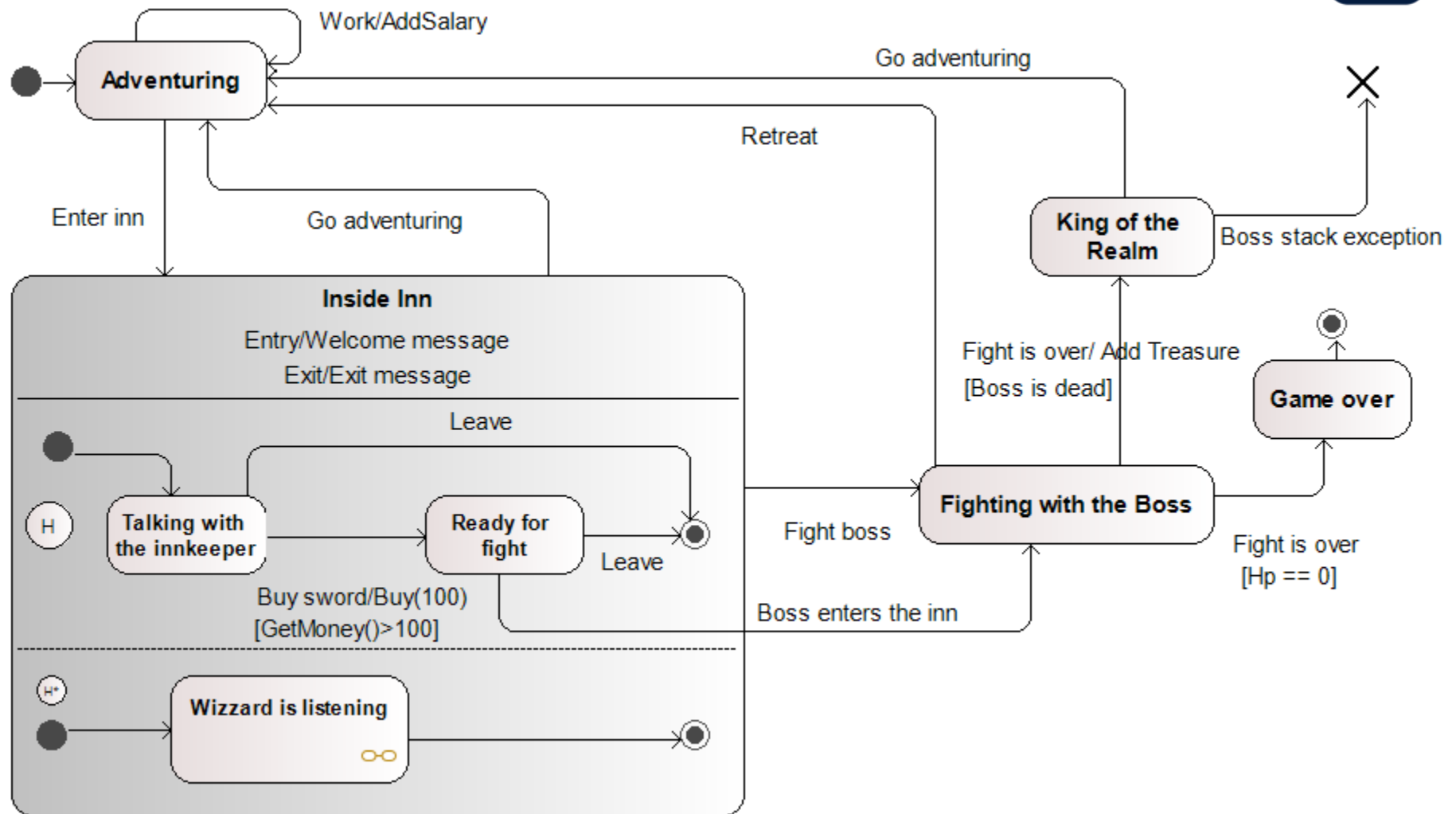
Állapotgépek

- Állapotgép (Statemachine, statechart diagram)
 - > Objektum-orientált, eseményvezérelt “véges automata”
 - > Fókusz:
 - A rendszer, vagy egy komponens állapotai
 - Állapotátmentek és feltételek
 - > Két fajta
 - Viselkedést leíró (Behavioral state machines)
 - Protokoll leíró (Protocol state machines)

Aktivitásdiagram: Folyamatot / munkafolyamatot modellez. (rendszer lépéseit írja le: megrendel -> feldolgoz -> kiszállít)

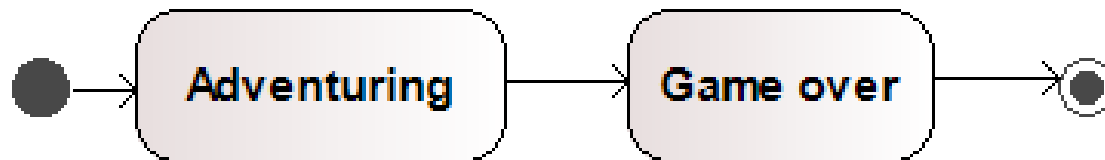
Állapotgép: Egy objektum életciklusát modellezi. (azt mutatja meg milyen állapotban van egy objektum: leadott -> feldolgozás alatt -> szállítás alatt)

Állapotgép: Kalandjáték



A kezdet és a vég

- Modellezzük egy kalandjátékot!
 - > A játék elkezdődik, kalandozunk, majd véget ér
 - > Kezdő állapot (initial state)
 - > Állapot (state)
 - > Átmenet (transition)
 - Ha több lehetséges átmenet van, nem-determinisztikusan választunk egyet
 - > Végő állapot (final state)



Főellenség

- Legyen egy főellenség, akit le kell győzni!



> Trigger (trigger)

- Az állapotátmenet oka, leggyakrabban egy esemény
- Kódban általában az objektumon kívülről **hívott** metódus

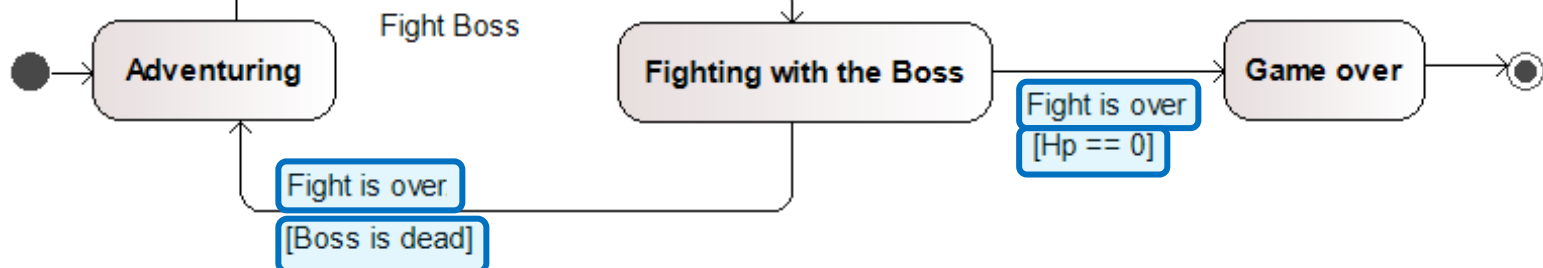
```
def fight_is_over_event():  
    if boss.hp == 0:  
        Adventuring  
    elif player.hp == 0:  
        Game over
```



- Ne legyen automatikus a győzelem/vesztés!

> Őrfeltétel (guard condition)

- Lehetséges átmenetnél a feltételnek is teljesülnie kell
- Triggerek finomhangolása



Jutalom

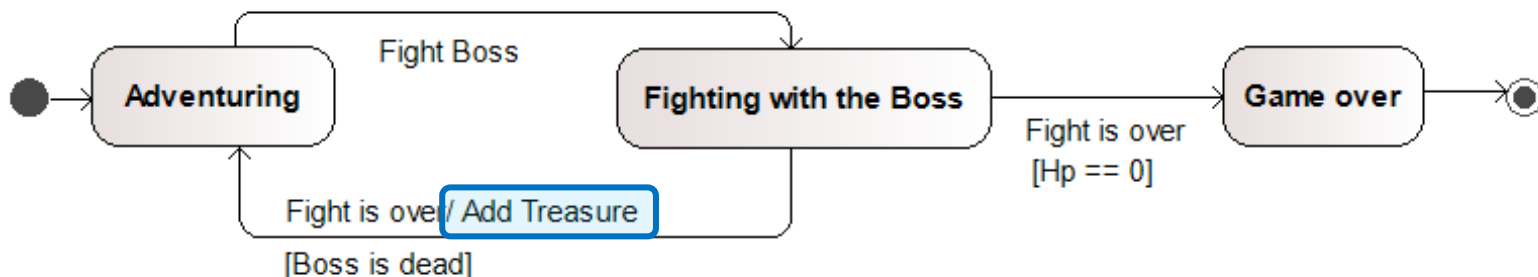
- Ha győztünk, kapjunk jutalmat!
 - > Akciók (action, hivatalosan: behavior expression)
 - Az átmenet során végrehajtandó akció
 - Kódban általában belső függvényhívás



A végrehajtás sorrendje:

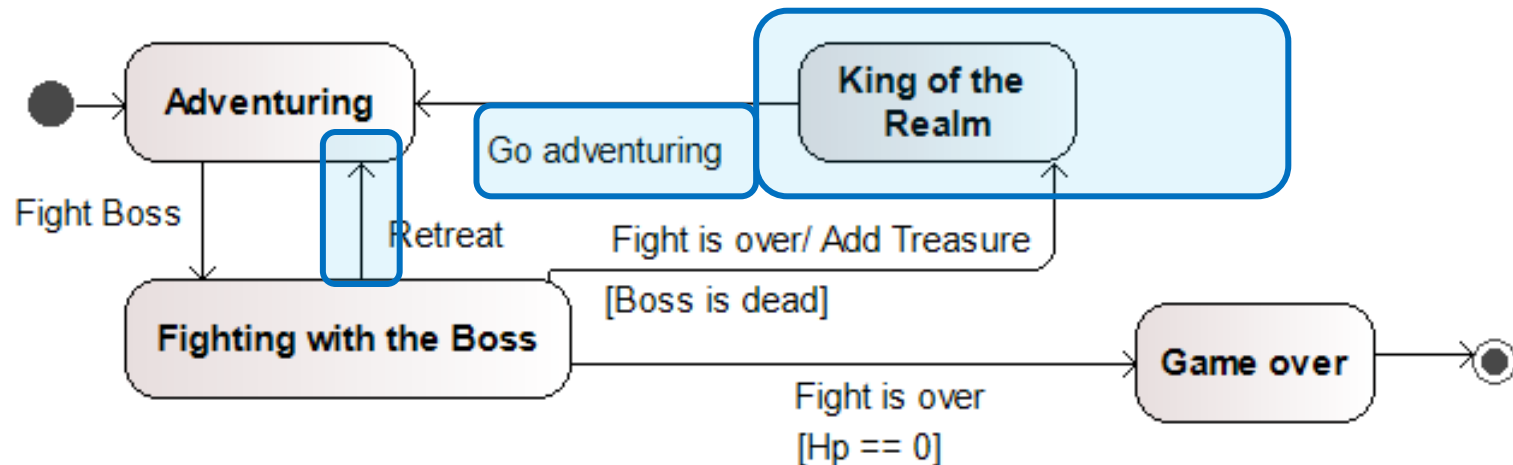
1. Trigger megérkezik (Fight is over)
2. Guard kiértékelése ([Boss is dead] → igaz/hamis?)
3. Ha a guard igaz → akció lefut (Add Treasure)
4. Állapot vált: Fighting with the Boss → Adventuring

```
def fight_is_over_event():  
    if boss.hp == 0:  
        Add Treasure  
        Adventuring  
    elif player.hp == 0:  
        Game over
```



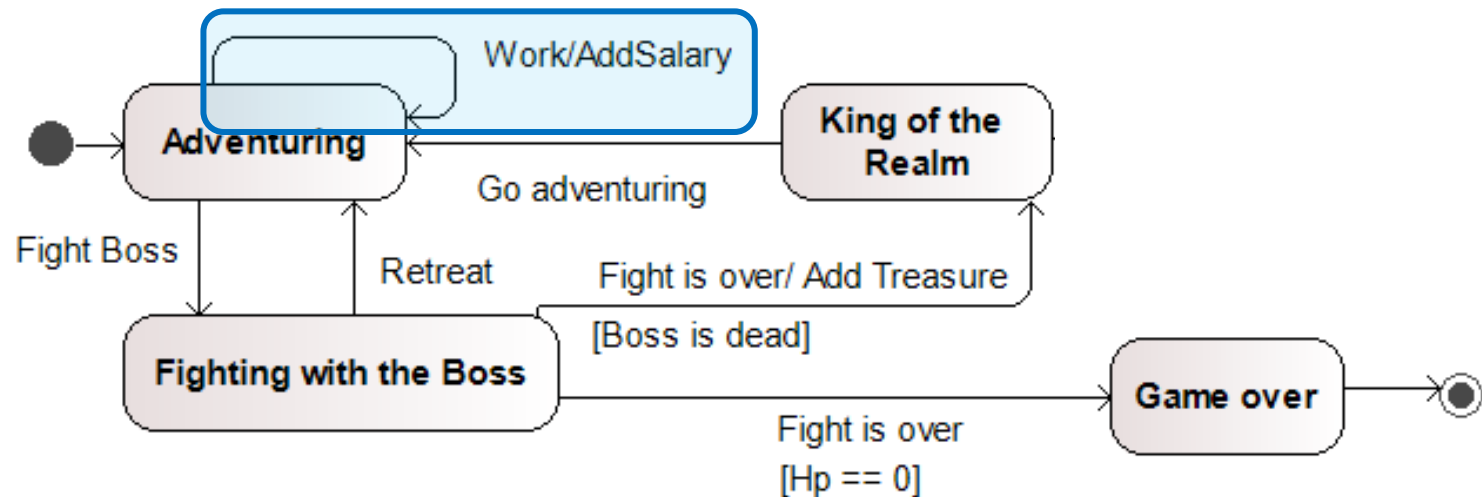
Visszavonulás és királyság

- Lehesse visszavonulni a csatából!
- Ha győztünk, kapjuk meg a királyságot!
 - > Utána még lehesse kalandozni!



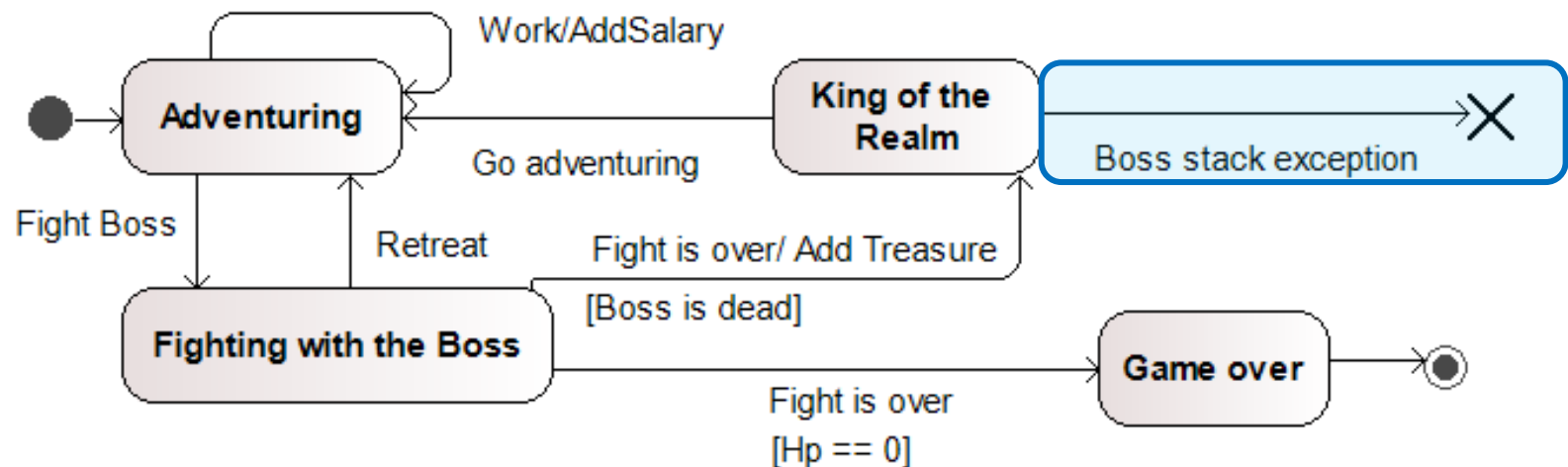
Munka, munka, munka

- Lehesse dolgozni (kalandozóként)!
 - > A munkáért kapjunk pénzt!



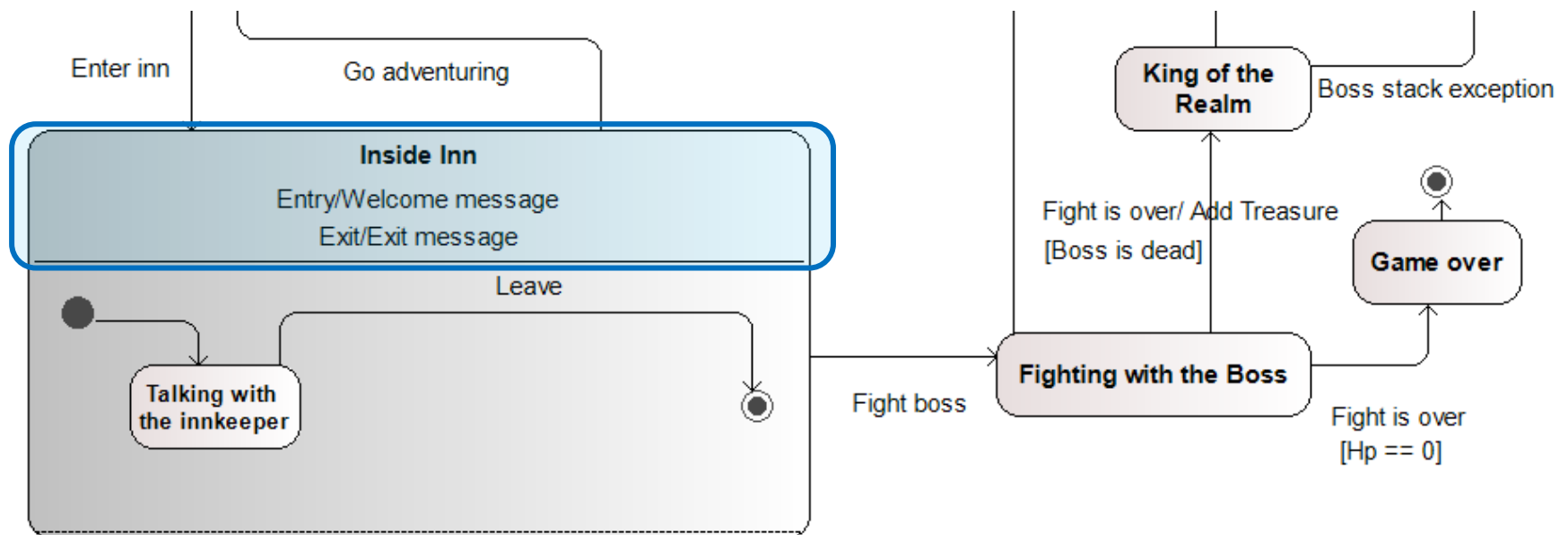
Hibakezelés

- Ha nincs több főellenség, kapjunk hibát!
 - > Terminálási állapot (terminate pseudo state)
 - > Kilépési akciók, állapotváltások nélkül véget ér a végrehajtás



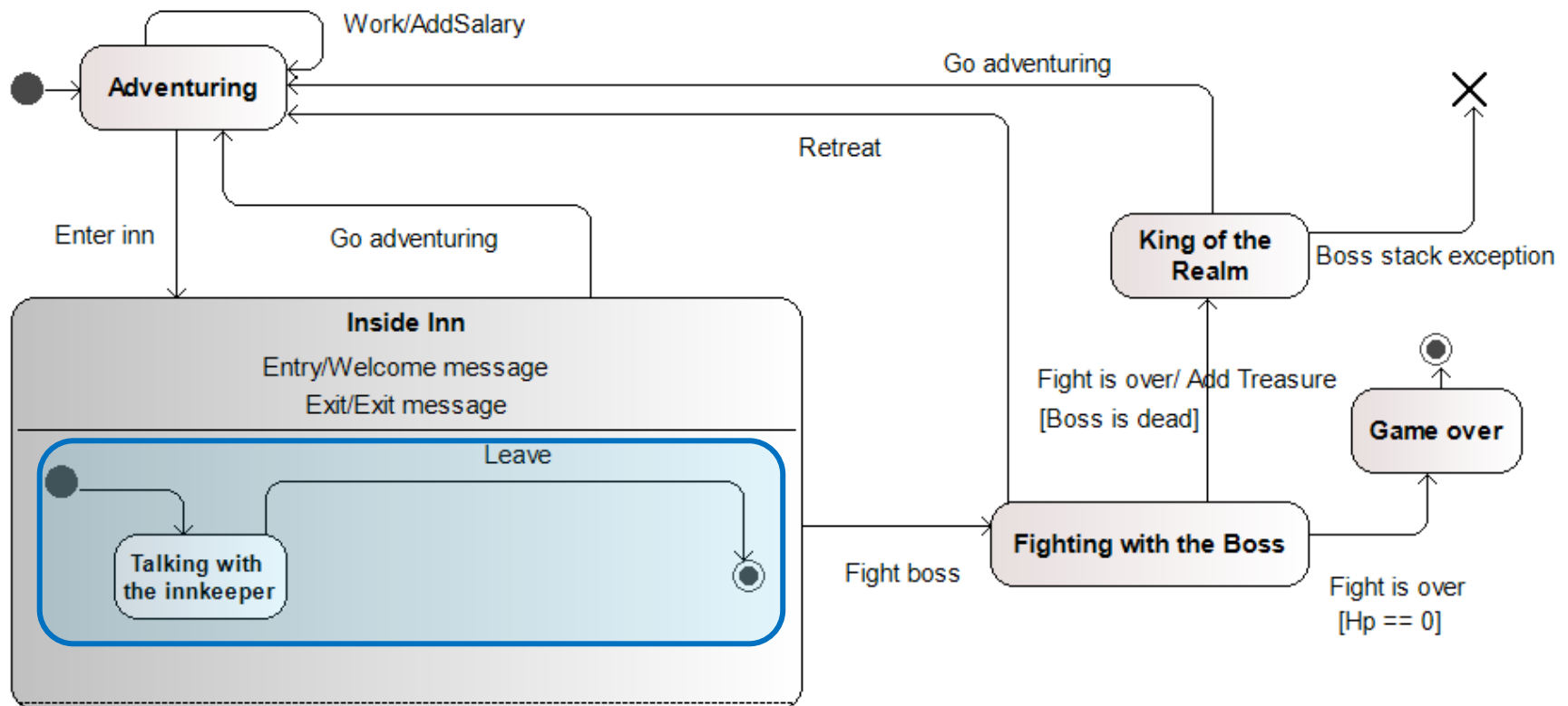
Fogadó

- Minden kaland egy fogadóban kezdődik...
 - > **Összetett állapot (composite state)**
 - > Kapjunk üdvözlést az elején, elköszönést a végén!
 - **Belépési akció (entry action)**: lefut, ha külső állapotból belépünk
 - **Kilépési akció (exit action)**: lefut, ha külső állapotba kilépünk
 - **Cselekvési akció (do action)**: folyamatosan fut a belépéstől a kilépésig (a példában ez nem szerepel)



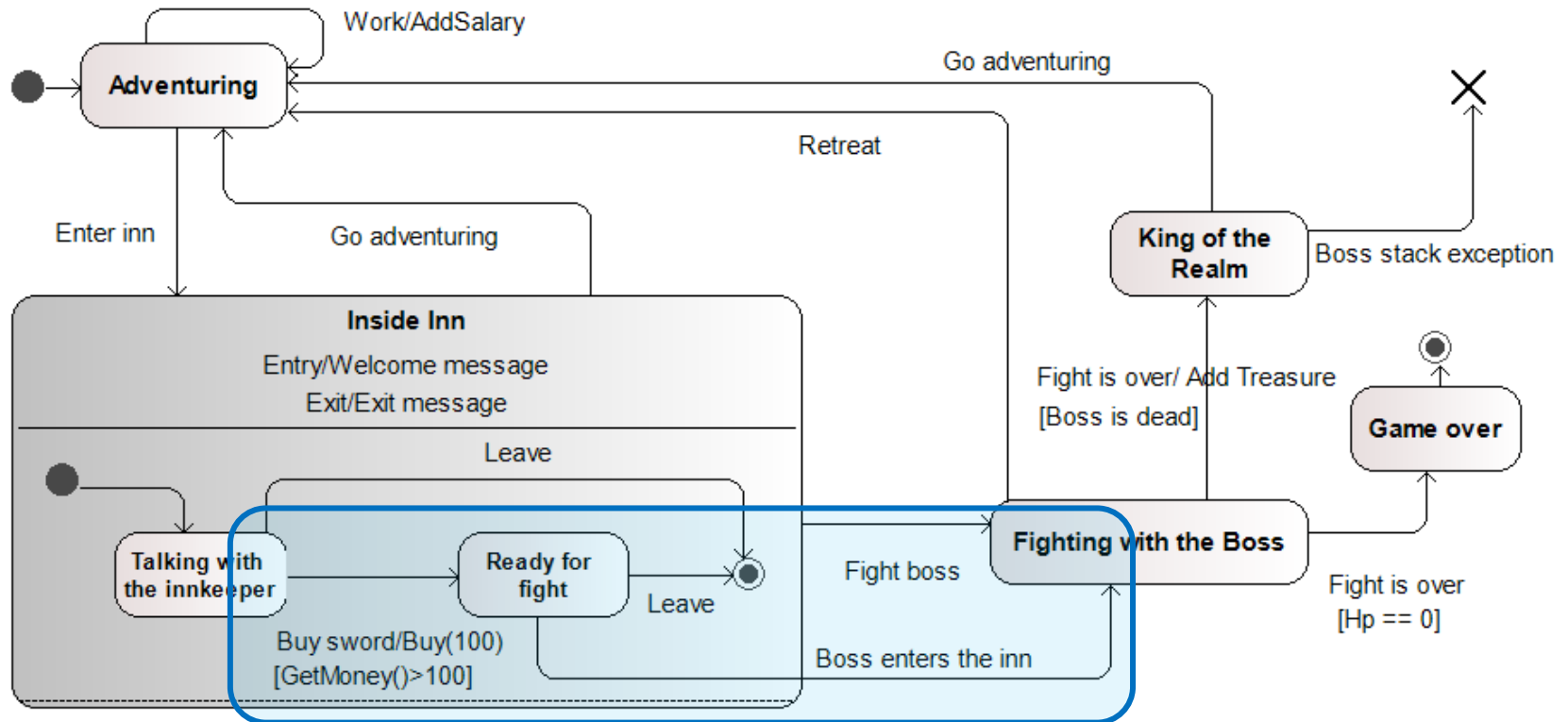
Fogadó (folytatás)

- Lehesse beszélgetni a fogadóssal!
 - > Belső állapotok/hierarchikus állapotgép



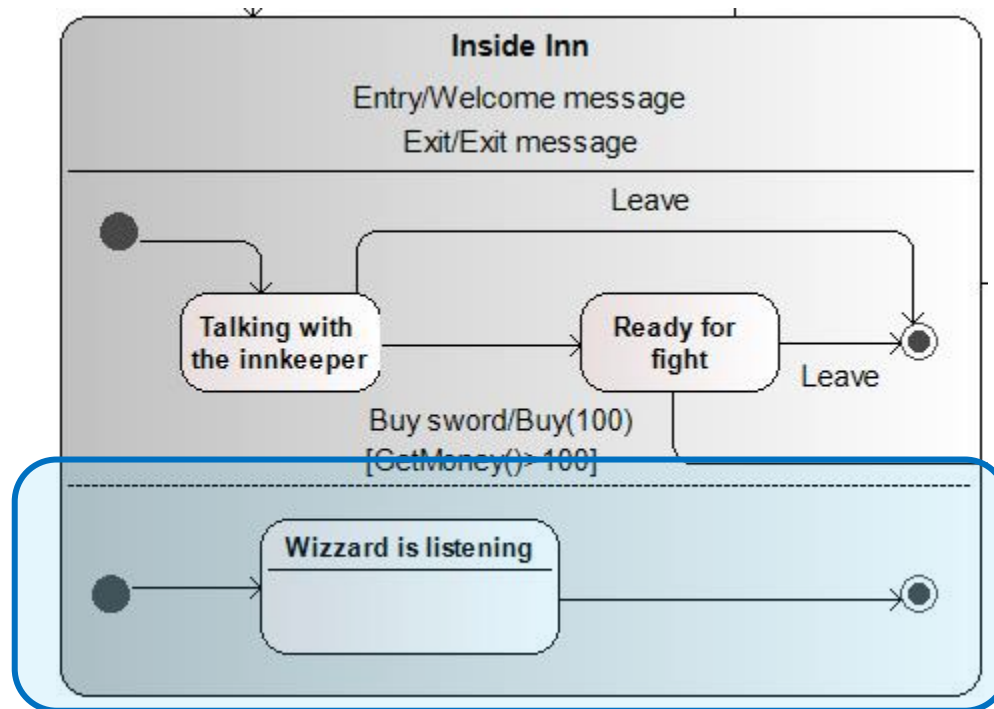
Kard

- Lehesse venni kardot a fogadóstól!
 - > A kard kerüljön pénzbe!
 - > Ha megvettük, készen vagyunk a csatára
 - > Ha a főellenség ilyenkor betoppan, azonnal harcolni kell
 - Közvetlen kilépés



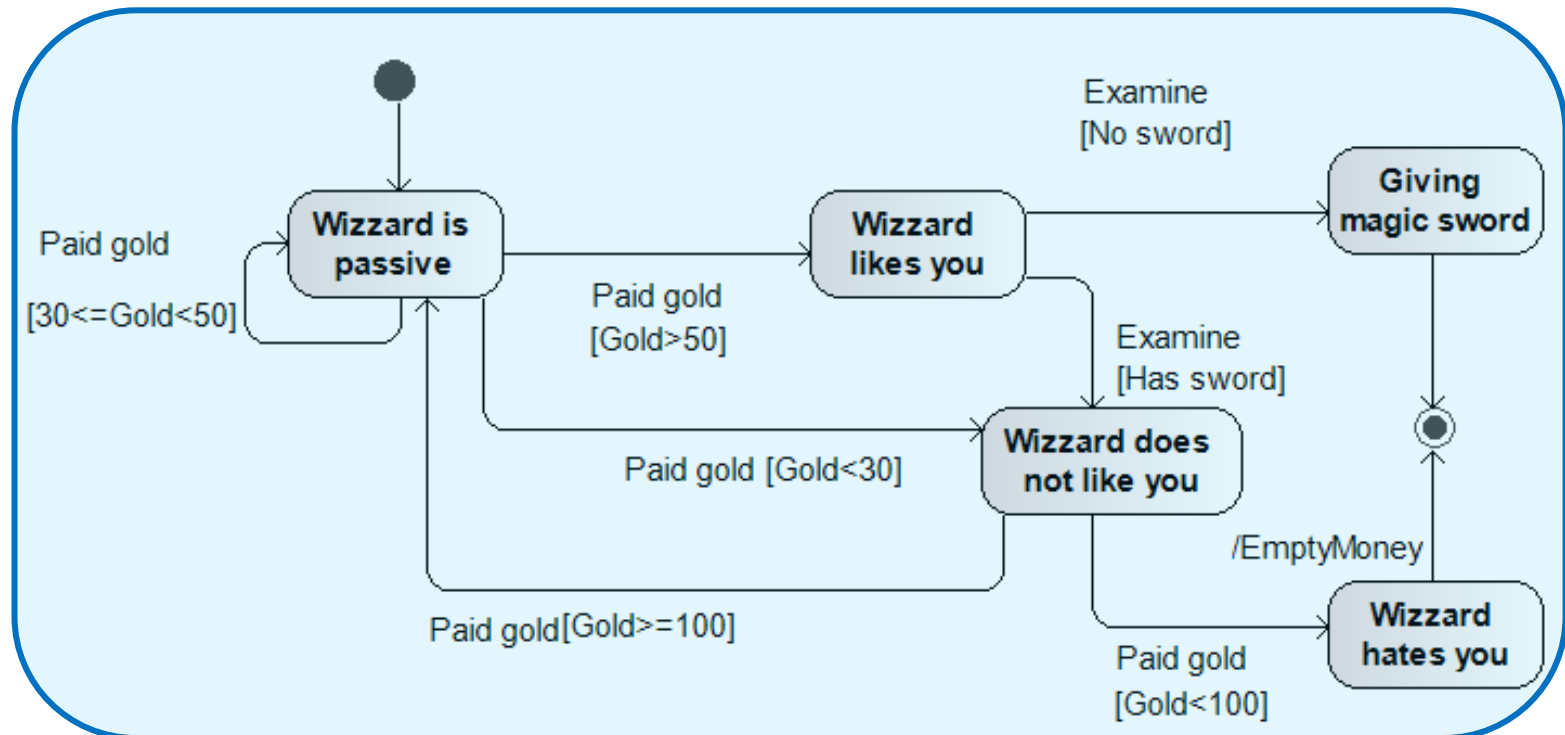
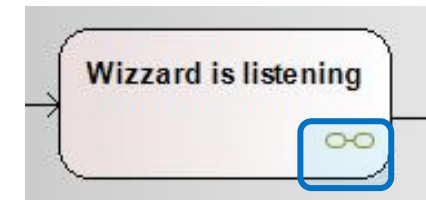
Varázsló

- Minden kalandban van egy varázsló...
 - > Az állapotváltás a fogadós szállal *párhuzamosan* zajlik
 - > Régió (region)
 - Összetett állapot ortogonális szétbontása



Varázsló

- Lehessen lefizetni a varázslót... de csak okosan!
 - > Állapoton belüli állapotgép
 - > **Alállapotgép állapot (submachine state)**
 - > Hivatkozás a másik összetett állapotra



Emlékezet

H^* -> Azaz ha kiléptél az **Inside Inn**-ből és visszatérsz, nem az **alapállapottól** indulsz újra, hanem onnan, **ahol abbahagytad**.

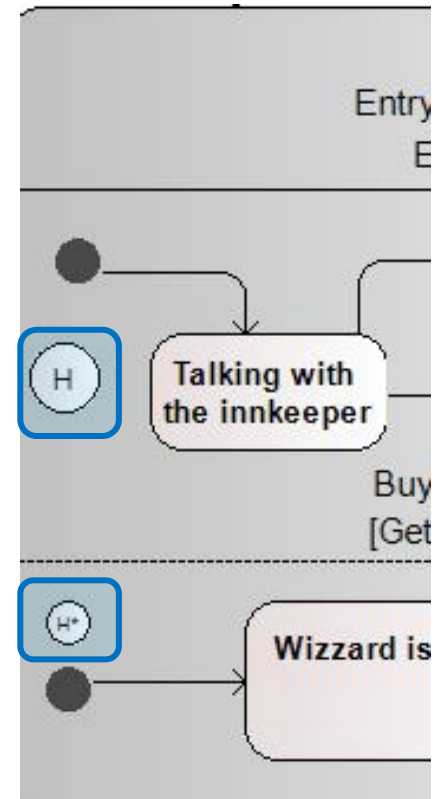
- Emlékezzon a rendszer, hol tartottunk a fogadóban, ha kimentünk és visszamentünk!

> Sekély történet (shallow history) (H)

- Visszaállítja a közvetlen tartalmazó állapotgép/régió állapotát visszatéréskor
- Csak egy hierarchia szinten keresztül működik

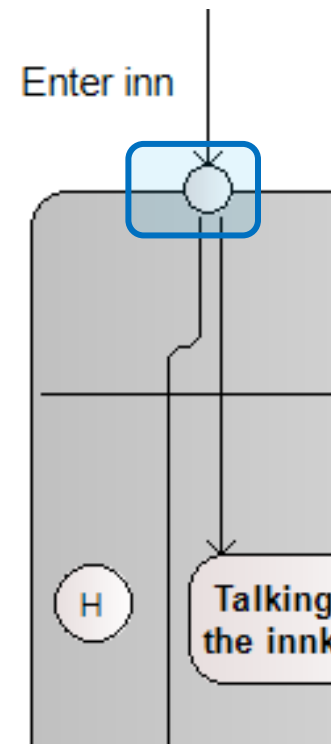
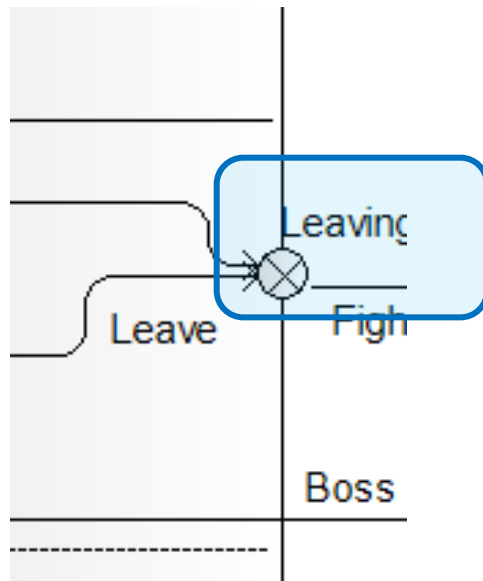
> Mély történet (deep history) (H^*)

- Visszaállítja a tartalmazó állapotgép/régió állapotát visszatéréskor
- Bármilyen mély hierarchiában működik



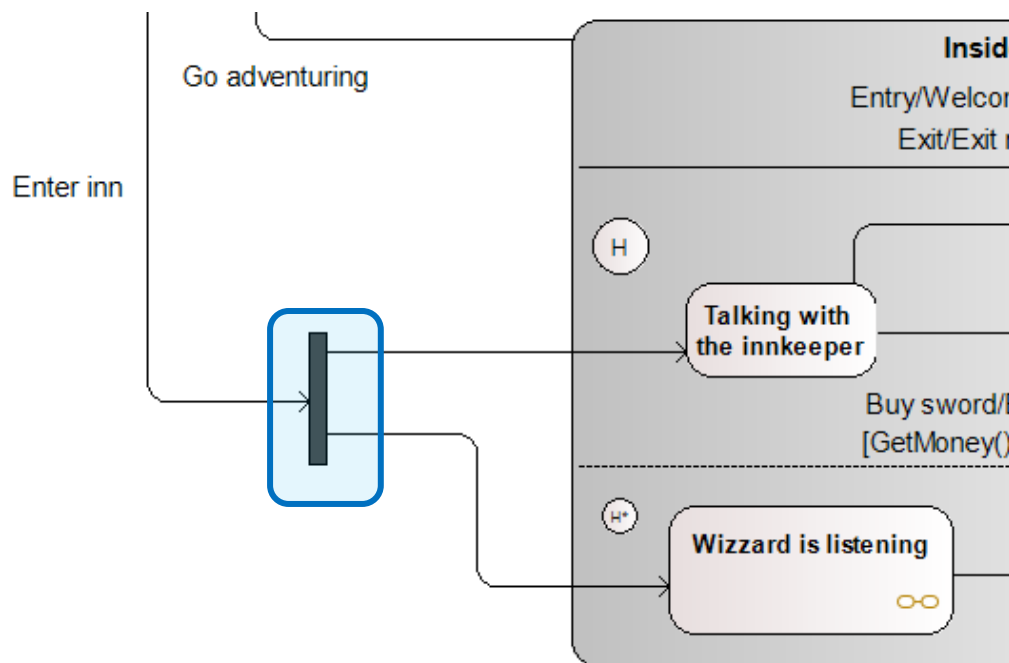
Belépési és kilépési pont

- Megadható külön nevesített be-, és kilépési pont
 - > Belépési/kilépési pont (entry/exit point)
 - Leaving / Enter inn
 - > Segíthet, ha különböző forgató- könyvekben különböző lépések kellenek
 - > Elrejtja a belső szerkezetet



Elágazás és egyesítés

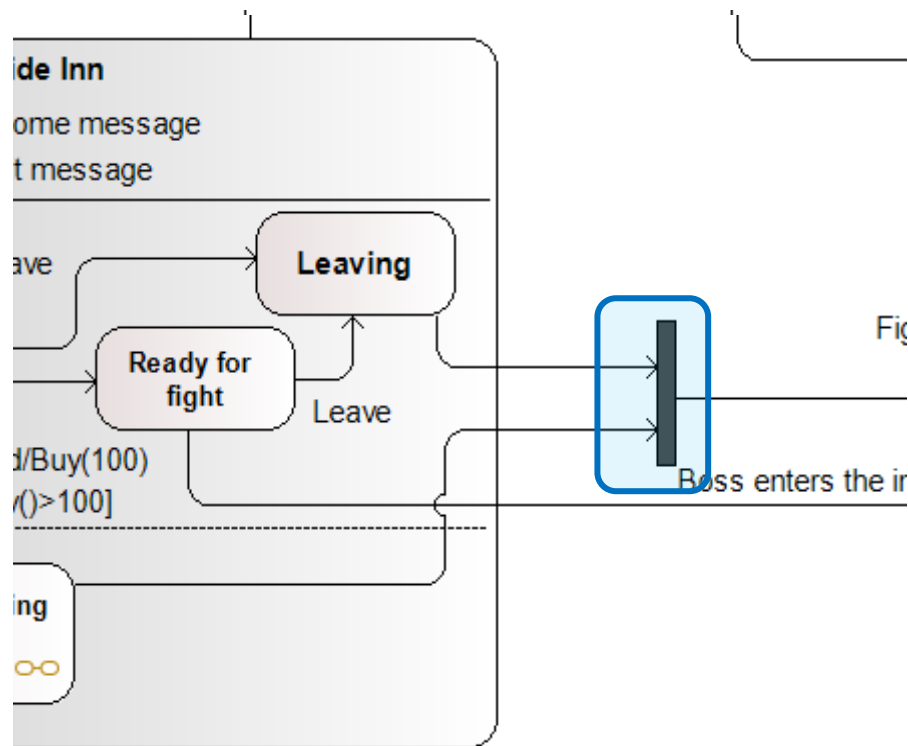
- Elágazó útvonalak
 - > Elágazás (fork)
 - > Egyszerre minden állapot aktiválódik
 - > Kimenő élek kötelezően esemény (trigger) és őrfeltétel nélkül!



Elágazás és egyesítés

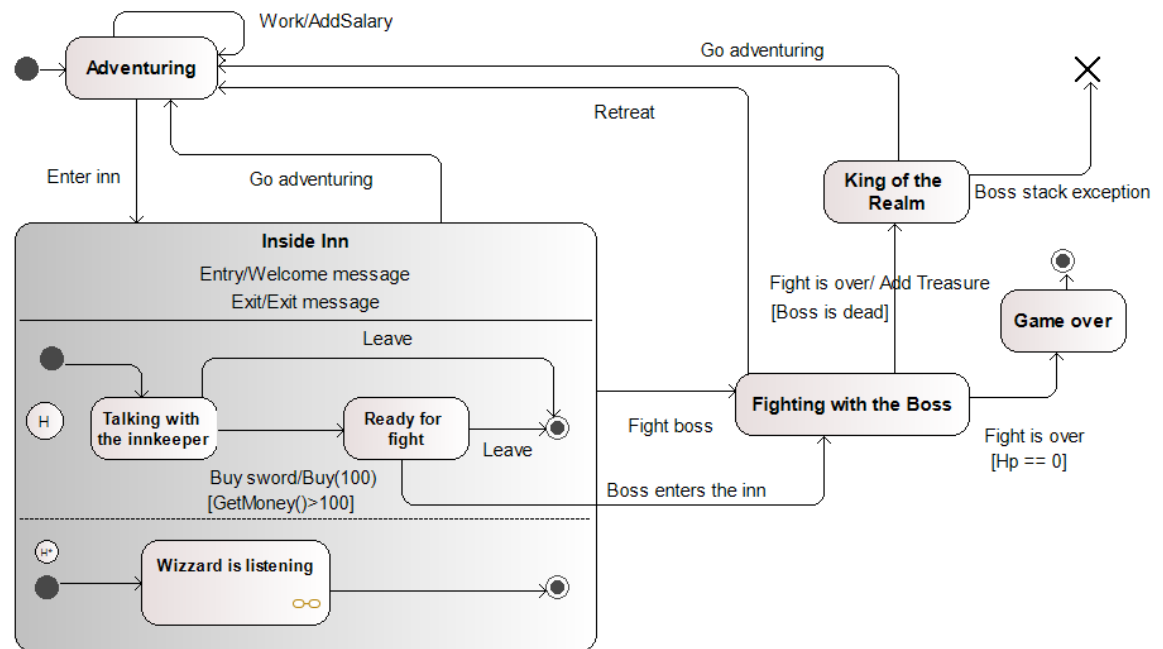
- Összefutó útvonalak

- > **Egyesítés (join)**
- > Megvárja a részállapotokat és egységesíti őket
- > Bemenő élek kötelezően esemény (trigger) és őrfeltétel nélkül!



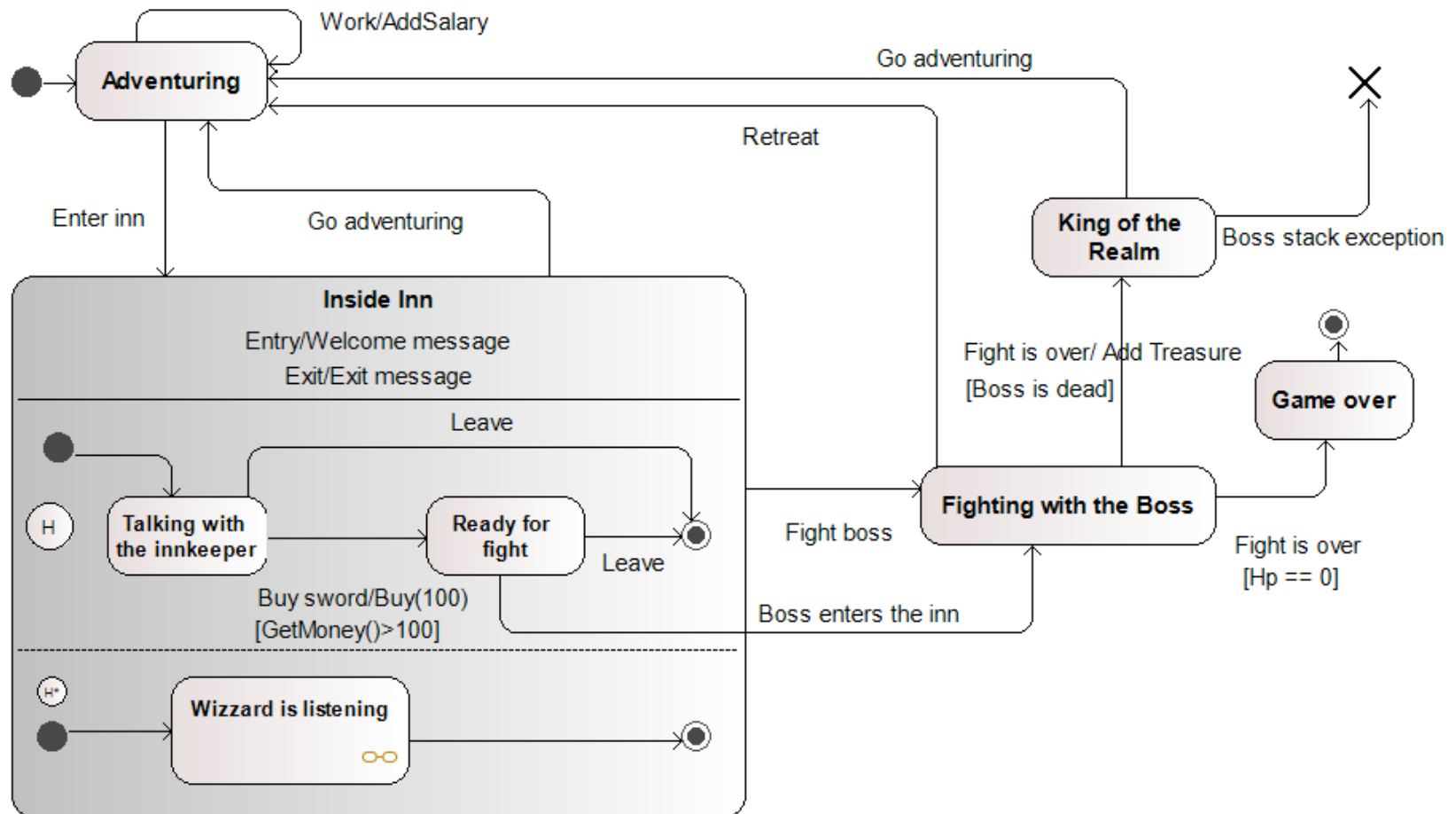
Állapotgép

- Állapotok
 - > Történet elem
 - > Explicit be-, és kilépési pont
 - > Elágazás, egyesítés
- Kezdő és végállapot
- Terminálás
- Átmenetek
 - > Esemény
 - > Örfeltétel
 - > Akció
- Összetett állapotok
 - > Régiók



Osztálydiagram

- Rajzoljuk meg az állapotgéphez tartozó osztálydiagrammot!



Szekvencia-aktivitás-állapot?

- **Szekvenciadiagram**

- > Szekvenciális interakció objektumok közt
- > Metódushívások, üzenetek tényleges időbelisége

- **Aktivitásdiagram**

- > Vezérlési folyam: aktivitások/tevékenységek
- > Lehet szekvenciális, szétágazó, vagy párhuzamos
- > A vezérlés (flow) fontosabb az őrfeltételeknél

- **Állapotgép**

- > Állapotokat és állapotátmeneteket ír le
- > Események vezérlik az állapotátmenetet

Köszönöm a figyelmet!

Levélcsomag átvétele (+ pontért)

- MPL csomagponton szeretnénk a csomag átvételét modellezni
- Belépünk a Postára és odamegyünk a sorszám kiadó automatához
- Ha nem működik, akkor hazamegyünk, a folyamat véget ér
- Ha működik, akkor kérünk egy sorszámot
- Beállítjuk az óránkon, hogy fél óra múlva csipogjon. Ha csipog, akkor hazamegyünk, bárhol járunk is a folyamatban
- Leülünk, majd alszunk egy kicsit
- A rendszer eseménnyel jelzi, ha mi következünk ekkor odaadjuk a személyinket az ügyintézőnek, aki ezt ellenőrzi, majd elmegy a raktárba a csomagért és kihozza azt.
- Párhuzamosan, egy időben átvesszük a csomagot, valamint megköszönjük azt
- Hazamegyünk, a folyamat véget ér

Szoftvertchnológia és -technikák

6. Előadás – Benedek Zoltán
Tervezési minták 1



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták 1

- > Definíció
- > Áttekintés
- > Szervezésük
- > Miben segítenek a tervezési minták
- > Template Method
- > Strategy

Definíció

- Programtervezési minta v. tervezési minta v. design pattern
- Tervezési minta leír egy gyakran előforduló programtervezési problémát, annak környezetét és a megoldás magját, amit alkalmazva számos gyakorlati eset hatékonyan megoldható.

Irodalom

- Az interneten bármely minta nevére rákeresve számos leírást találunk, a legtöbb minta a wiki-n is megtalálható
 - > (de kezeljük némi fenntartással, ezek a wiki leírások lehetnek hiányosak, vagy nem feltétlen a leggyakorlatiasabb megközelítést mutatják be)
 - [1] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: DESIGN PATTERNS, Elements of Reusable Object-oriented Software
 - > A nagy klasszikus, de ma már kicsit „régies”, ebben C++ és más, ma már ritkábban használt nyelven vannak a példák
 - https://sourcemaking.com/design_patterns
-
- Megjegyzés: előadáson és gyakorlatokon C# példák lesznek

Példakódok

- A legtöbb mintához tartozik példakód, kódjegyzésekkel ellátva
- GitHub-on érhető el
 - > A kód webböngészőben is nézhető/böngészhető a kód <https://github.com/bzolka/AUT-SZTT>
 - Ez alatt navigáljunk le a Előadás/Demo/DesPattCode mappába, az egyes minták demo kódja általában a mintának megfelelő nevű almappában található
 - > De célszerűbb az egészet a <https://github.com/bzolka/AUT-SZTT> alatt „Clone and download” gomb segítségével letölteni (vagy a “git clone https://github.com/bzolka/AUT-SZTT” paranccsal parancssorból klónozni), így Visual Studio-ban a solution megnyitható.
 - > A legtöbb mintához „futtatható” kód is tartozik: a mintához tartozó mappában levő Program.cs fájlban a Main2 függvényt előbb nevezzük át Main-re
 - > Ha más mintához tartozó kódot szeretnénk „futtatni”, nevezzük vissza a Main-t Main2-re, mert a VS projektünkben csak egy Main nevű statikus függvény lehet: ha több is van, fordítási hibát kapunk.

Bevezető példa

Példa

- Egy alkalmazást fejlesztünk, melyben adatokat kell tömöríteni
 - > Több tömörítési algoritmust szeretnénk támogatni: zip, rar, 7zip
 - > Futás közben is lecserélhető legyen az algoritmus
 - > Könnyen bővíthető legyen új algoritmusokkal

Példa – kiindulás

Csak zip algoritmust támogat (nézzük VS alatt)

```
class DataProcessor
{
    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        byte[] inputData;
        while ( (inputData = readData()) != null) {
            byte[] compressedData = zipData(inputData);
            processCompressedData(compressedData);
        }
    }

    byte[] readData() { /* adat olvasása hálózatról */ }

    byte[] zipData(byte[] data) { /* adattömörítés zip algoritmussal*/ }

    void processCompressedData(byte[] data) { /* tömörített adat feld.*/*; }
}
```

Példa –előző továbbfejlesztve

Több algoritmust támogat – 1. rész

```
class DataProcessor2
{
    // ...

    CompressionAlg compressionAlg;

    public DataProcessor(CcompressionAlg compressionAlg)
    {
        this.compressionAlg = compressionAlg;
    }

    public void Run() // változatlan
    {
        byte[] inputData;
        while ((inputData = readData()) != null)
        {
            byte[] compressedData = compressData(inputData);
            processCompressedData(compressedData);
        }
    }

    byte[] readData() { /* adat olvasása hálózatról */ }
    void processCompressedData(byte[] data) { /* töm. adat. feld. */; }
```

Példa –továbbfejlesztve

Több algoritmust támogat – 2. rész

```
byte[] compressData(byte[] data)
{
    switch (compressionAlg)
    {
        case CompressionAlg.Zip:
            return compressUsingZip(data);
        case CompressionAlg.Zip7:
            return compressUsingZip7(data);
        case CompressionAlg.Rar:
            return compressUsingRar(data);
    }
}

byte[] compressUsingZip(byte[] data) { /* adattömörítés zip alg. */ }
byte[] compressUsingZip7(byte[] data) { /* adattömörítés 7zip alg.*/ }
byte[] compressUsingRar(byte[] data) { /* adattömörítés rar alg. */ }

}
```

Példa –továbbfejlesztve

- Megoldás elve
 - > Konstruktorban átadjuk paraméterként (enum), milyen tömörítő algoritmust szeretnénk, ezt eltároljuk egy tagváltozóban (compressionAlg)
 - > Ezen tagváltozó alapján választjuk ki később a tömörítő algoritmust
- Problémák
 - > A megoldás nem bővíthető könnyen új algoritmussal
 - A DataProcessor2 osztályba be van építve, milyen algoritmusokat ismer.
 - Ha újat szeretnénk bevezetni, a DataProcessor2 kódját módosítani kell. Ezt követően pedig újra kell tesztelni az osztályt.
 - Olyan bővíthetőségi megoldást keresünk, mely NEM igényli a DataProcessor2 módosítását új algoritmus bevezetésekor.
 - A megoldás majd a **STRATEGY** tervezési minta alkalmazása lesz, rövidesen visszatérünk rá
- Sok tervezési minta létezik, pl. Strategy, Singleton, Composite, Observer, stb.
 - > Mind más programtervezési problémára nyújt megoldást

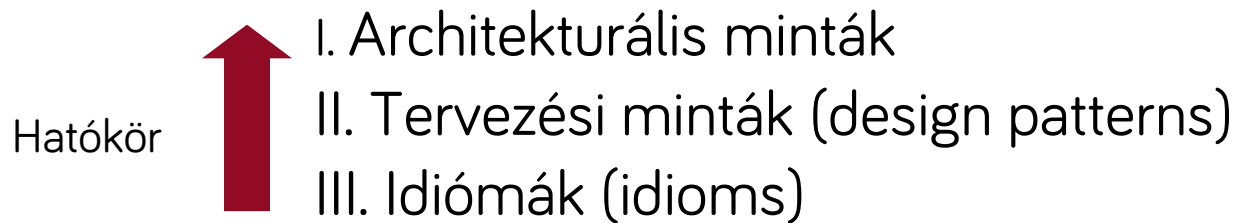
Tervezési minták jellemzői

Tervezési minták leírása

- Minden tervezési minta leírásához négy alapelem tartozik
 - > Pattern név (pattern name)
 - A minta neve, hatékonyan azonosítja a mintát.
 - **Fontos a kommunikáció miatt:** a fejlesztőknek elég a minta nevére hivatkozniuk, ebből már értik is, hogy milyen objektumok milyen szerepkörben szerepelnek az adott tervben/kódban (persze csak ha ismerik az adott mintát)
 - > Probléma (problem)
 - A **probléma** és a **környezet (context)** bemutatása, gyakran konkrét példán keresztül
 - > Megoldás (solution)
 - Leírja a megoldásban szereplő elemeket, kapcsolatukat, az egyes elemek felelősségét és együttműködését. Nem konkrét eset megoldása, inkább a megoldás absztrakt leírása.
 - > Következmények (consequences)
 - A minta alkalmazásának következményeit írja le. Fontos tapasztalatokat tartalmaz, és segít a minta kiválasztásban.

Minták csoportosítása hatókör szerint

- A szoftverrendszer milyen szintjén használhatók (architektúra → alrendszerek → ... → kódolás egy adott programnyelven)
- Három csoport



Minták csoportosítása hatókör szerint

- **Architektúra szint**

- > Alapvetően meghatározza az adott alkalmazás/alrendszer felépítését
- > Egy későbbi előadáson megnézünk párat, pl. MVVM, Document-View, Layers

- **Tervezési minta szint**

- > Egy alkalmazásban/alrendszerben több, sok is használható
- > Függetlenek a programozási nyelvektől pl: Singleton / Strategy

- **Idióma**

- > Egész alacsony szintűek, sokszor csak adott programozási nyelv kontextusában használhatók pl: Python - with open ("file.txt" as f

- Mostantól a középső, „**Tervezési minta**” szintre besorolható tervezési mintákról lesz szó

- > Ezt nem kell túl szigorúan venni, több design pattern jól alkalmazható architektúra vagy idióma szinten is

„Klasszikus”/GoF tervezési minták

- Alapirodalom
 - > [Erich Gamma](#), Richard Helm, Ralph Johnson, John Vlissides: DESIGN PATTERNS, Elements of Reusable Object-oriented Software
 - > Először nevesítettek és katalogizáltak tervezési mintákat
 - > Négyen írták: köznyelvben „[Gang of Four](#)” (röviden GoF), vagyis a „négyek bandája”ként szokás rájuk hivatkozni.
 - Nagyon elterjedt ez a hivatkozás, illik tudni!

GoF tervezési minták kategóriák

A kiemelteket nézzük majd (ezeket kell tudni)

Létrehozási (creational)	Strukturális (structural)	Viselkedési (behavioral)
Factory Method	Adapter	Template Method
Abstract Factory	Composite	Strategy
Singleton	Façade	Observer
Builder	Proxy	Command
Prototype	Bridge	Memento
	Decorator	Interpreter
		Chain of Responsibility
		Iterator
		Mediator
		Flyweight
		State
		Visitor

Hogyan segítenek a tervezési minták a probléma megoldásában

- A **tervezési** minták a fejlesztés **tervezés** fázisában segítenek (az analízis minták az analízis fázisában)
 - > Tervezés: amikor kódra „képezzük le” az analízis során kidolgozott fogalmi modellt
- Az alábbiak elérése nehéz, ezekben a magasszintű célokban segítenek a tervezési minták:
 - > I. Könnyen változtatható, kiterjeszthető kódot készíteni (design for change)
 - > II. Újrafelhasználható kódot készíteni (design for reuse)
 - > III. Könnyen unit (egység) tesztelhető kódot készíteni

Részletesebben



I. Változtathatóság, bővíthetőség

- Nem adódik magától, szem előtt kell tartani a tervezéskor. A tervezés egyik célja!
- Miért?
 - > Rendszert tervezni nehéz
 - > Sokszor nem is tudjuk az elején pontosan definiálni a feladatot, a megrendelő sem lát nagyon előre, nem is lehet elvárni tőle
 - Az utóbbi években előtérbe kerülő **agilis szemléletmód** esetén ez hatványozottan igaz
 - A szoftver termékek az első kiadás után általában hosszú éveken át, számos iterációban fejlődnek tovább
 - > A szoftverfejlesztésben csak egy dolog (biztos) és változatlan: maga a változás
 - Változnak a felhasználói követelmények
 - Változnak a technológiák

I. Változtathatóság, bővíthetőség

- Tervezési alapelv: tervezzünk úgy, hogy a rendszer nem végleges (design for change)!

Válasszuk külön az alkalmazás azon részeit, melyek változatlanok és melyek változnak (arra számítunk, hogy változni/bővülni fognak). Zárjuk egységbe ezen változó kódrészeket, hogy később úgy tudjuk bővíteni/változtatni az alkalmazást, hogy a nem változó funkciók/részek kódjához ne kelljen hozzányúlni.

- > Nem mindent tervezünk bővíhetőre, mert költséges!
 - Csak ahol számítunk arra, hogy bővíteni/módosítani kell majd

I. Változtathatóság, bővíthetőség

- Negatív példa: készítünk egy alkalmazást, melyben minden mindennel összefügg, az SRP elvekre sem ügyelünk, valamint hiányzik a kritikus pontokban a módosításra/bővíthetőségre való felkészítés
 - > A kollégák nem fognak szeretni minket, ha a kódunkhoz kell nyúlniuk
 - > Nekünk sem lesz kedvünk a kódhoz többet hozzányúlni
 - > Egy bug javításával/pici módosítással
 - Sok kódot kell átnézni, újra tesztelni
 - Több új bugot viszünk be a rendszerbe
- Pozitív példa
 - > A meglévő kód bővítése, módosítása is pont annyira élvezhető feladat, mintha teljesen új kódot kellene írni

II. Újrafelhasználhatóság (reuse)

- Nem adódik magától, szem előtt kell tartani a tervezéskor. A szoftvertervezés egyik célja!
- A költségcsökkentés leghatékonyabb módja a cégek számára
- Akár egy projekt különböző moduljaiban, akár projektek között is!
- Ha az alkalmazásban minden kódrészlet mindennel összefügg, akkor nem tudunk belőle részeket újra felhasználni, mert minden rész újrafelhasználásához szükség van az összes függőségre
 - > Lazán csatolt „részekre” van szükségünk
- Hosszú távon kifizetődő lehet: saját keretrendszer kifejlesztése

II. Újrafelhasználhatóság (reuse) *

- **Kitérő: Keretrendszer (framework) definíció***

- > Minden keretrendszer adott probléma területet céloz meg (pl. ablakozós rendszerek, dokumentumkezelő rendszerek, matematikai rendszerek, grafikus rendszerek, CAD rendszerek, játékok, stb.)
- > Meghatározza az alkalmazás architektúráját, ehhez alapvető építőköveket (pl. osztályok) biztosít.
- > OO környezetben: előre definiált, együttműködő osztályok vannak, pontosan meghatározott felelősséggel. Az alkalmazás fejlesztésekor gyakran:
 - le kell belőlük származtatni,
 - Virtuális/absztrakt függvényeket kell felüldefiniálni
- > **Cél: Adott szakterületen egy alkalmazás fejlesztésekor a lehető legkevesebbet kelljen kódolni (mert a keretrendszer már szolgáltatásként nyújtja)**
- > Konzisztens struktúrája lesz az alkalmazásainknak
- > Magas szinten már “tervezni sem kell”, hiszen az architektúra meghatározott - terv újrafelhasználás (design reuse)

pl:

Java - Spring framework

JavaScript - React (UI)

III. Unit (egység) tesztelhetőség

- Egy hosszú életű projekt esetén fontos, hogy a kód minél nagyobb részéhez automata módon futtatható **unit (egység) tesztek** készítsünk (legalábbis a logikai részekhez, felhasználói felületekhez kevésbé)
- **Unit tesztek esetén kóddal tesztelünk kódot**
- Egy unit teszt az osztályt önmagában, a függőségei (más logikai osztályok, melyeket használ) nélkül tesztel
 - > A unit tesztek futtatásához az osztály függőségeit le szeretnénk cserélni olyan implementációkra, melyek a tesztet szolgálják
 - > Ha egy osztályba be vannak építve a függőségei, akkor ez nem tehető meg, az osztály gyakorlatilag nem lesz unit tesztelhető
 - > A megoldás: az osztály a függőségeit lazán csatoltan kell kezelje (pl. interfészekon keresztül)
- Bizonyos tervezési minták abban is segítenek, hogy a kódunk könnyen unit tesztelhető legyen

System of Patterns *

- Mi a system of patterns?
 - > A rendszer tervezésekor több mintát is felhasználunk
- Általában egy (illetve alrendszerenként egy) architektúra szintűt
 - > Ez megadja az alapstruktúrát
- Az egyes részek megtervezésekor (detailed design) több tervezési mintát is felhasználunk, akár egymással kombinálva
- A kombinált megoldásba minden minta beleviszi a maga pozitív tulajdonságait, vagyis egy megoldást egy részproblémára

Néhány fontosabb, gyakrabban használt tervezési minta

- Létrehozási
 - > Factory method
 - > Abstract factory
 - > Singleton
 - > Dependency Injection*
- Strukturális és viselkedési
 - > Template method
 - > Strategy
 - > Observer
 - > Command
 - > Command Processor
 - > Memento
 - > Adapter
 - > Facade
 - > Proxy
 - > Composite

Mit kell tudni?

A minta neve alapján vagy példán, vagy általánosságában ismertetni a mintát (milyen környezetben, milyen problémát, hogyan old meg) . A bemutatáshoz a legtöbb mintához osztály, illetve néhány esetben szekvencia diagramot is kell rajzolni).

+

Szöveges feladatléírás alapján rá kell jönni, milyen mintákat célszerű bevetni, és ezeket alkalmazni is kell tudni.

Bővíthetőséghez, kiterjeszthetőséghez kapcsolódó alap tervezési minták

Template method (sablon metódus) minta

Strategy (stratégia) minta

Egyéb

Célok, elvek

- Azon osztályoknál, melyeknél fontos a bővíthetőség, építsük be ennek lehetőségét
 - > Absztrakt/virtuális függvények hívásával -> Template Method minta
 - > Delegáljuk más osztályoknak -> Strategy minta
- A cél, hogy ha bővíteni kell a funkcionalitást, az osztály kódját NE KELLJEN MÓDOSÍTANI

Template method (Sablonmetódus)

Korábbi példa némi kiegészítéssel

- Egy alkalmazást fejlesztünk, melyben adatokat kell tömöríteni
 - > Több tömörítési algoritmust szeretnénk támogatni: zip, rar, 7zip
 - > Könnyen legyen bővíthető új tömörítési algoritmusokkal
 - > Futás közben is legyen lecserélhető az algoritmus
 - > Legyen megszakítható (cancel) a művelet, de ne legyen beégetve a megszakítás módja, pl.:
 - Billentyű lenyomás
 - Gomb kattintás

Kiindulási példa áttekintése

- Lásd: DesPattCode\
StrategyAndTemplateMethod_Common\
DataProcessor_Initial mappa DataProcessor osztály
feletti megjegyzések !

https://github.com/bzolka/AUT-SZTT/blob/master/EI%C5%91ad%C3%A1s/Demo/DesPattCode/DesPattCode/StrategyAndTemplateMethod_Common/DataProcessor_Initial/DataProcessor.cs

DataProcessor

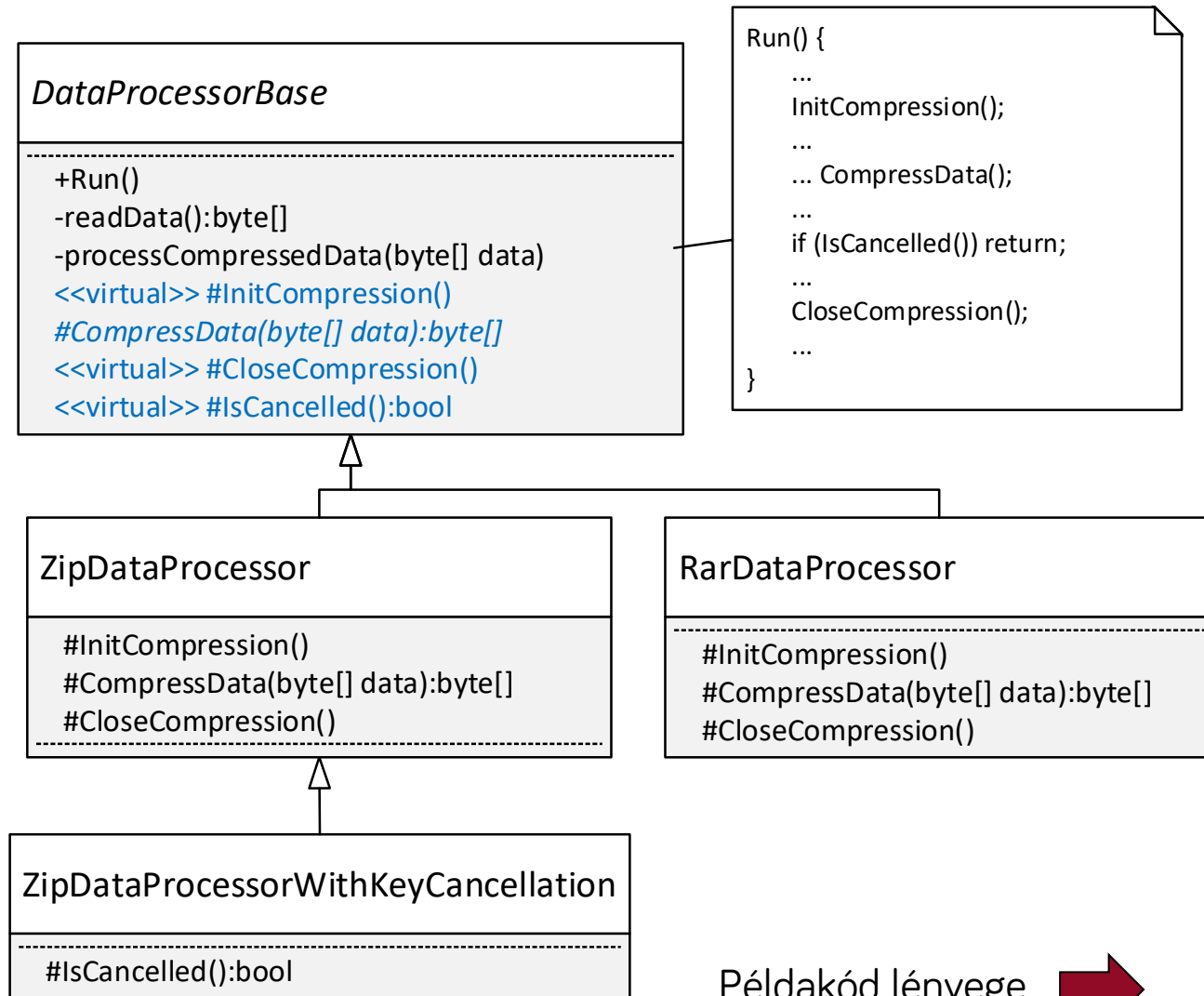
```
+Run()  
-readData():byte[]  
-processCompressedData(byte[] data)  
-initZip()  
-compressData(byte[] data):byte[]  
-closeZip  
-isCancelled
```

Template method minta alkalmazása

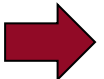
- A változatlan kódrészeket tegyük egy őosztályba
- A változó/kiterjeszthető részeket nem drótozzuk az ős műveleteibe, hanem virtuális és/vagy absztrakt függvényekre bízunk
- A leszármazott osztályban ezen függvények felülírásával adjuk meg a specifikus viselkedést
 - > A példában:
 - Tömörítés módja
 - Cancel módja

Template Method példa áttekintése

- Lásd forráskód:
DesPattCode\
TemplateMethod
mappában fájlok
- **DataProcessorBase**
ősosztály a lényeg, a
különböző
implementációkra
közös kódot
tartalmazza. Az
implementációfüggő
részeket
virtuális/absztrakt
függvényekre bízva
(InitCompression,
CompressData,
CloseCompression,
IsCancelled)



Példakód lényege



Template Method példa

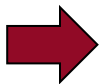
- **DataProcessorBase** őssosztály **Run** művelete a lényeg, a különböző implementációkra közös kódot tartalmazza. Az implementációfüggő részeket virtuális/absztrakt függvényekre bízza (**InitCompression**, **CompressData**, **CloseCompression**, **IsCancelled**)

```
...  
public void Run()  
{  
    byte[] inputData;  
    InitCompression(); // Tömörítés inicializás, impl. függő  
    try  
    {  
        while ((inputData = readData()) != null)  
        {  
            // Tömörítés, impl. függő  
            byte[] compressedData = CompressData(inputData);  
            processCompressedData(compressedData);  
            if (IsCancelled()) // Cancel vizsgálat, impl. függő  
                return;  
        }  
    }  
    finally  
    {  
        CloseCompression(); // Tömörítés lezárás, impl. függő  
    }  
}  
...
```

Template Method célja

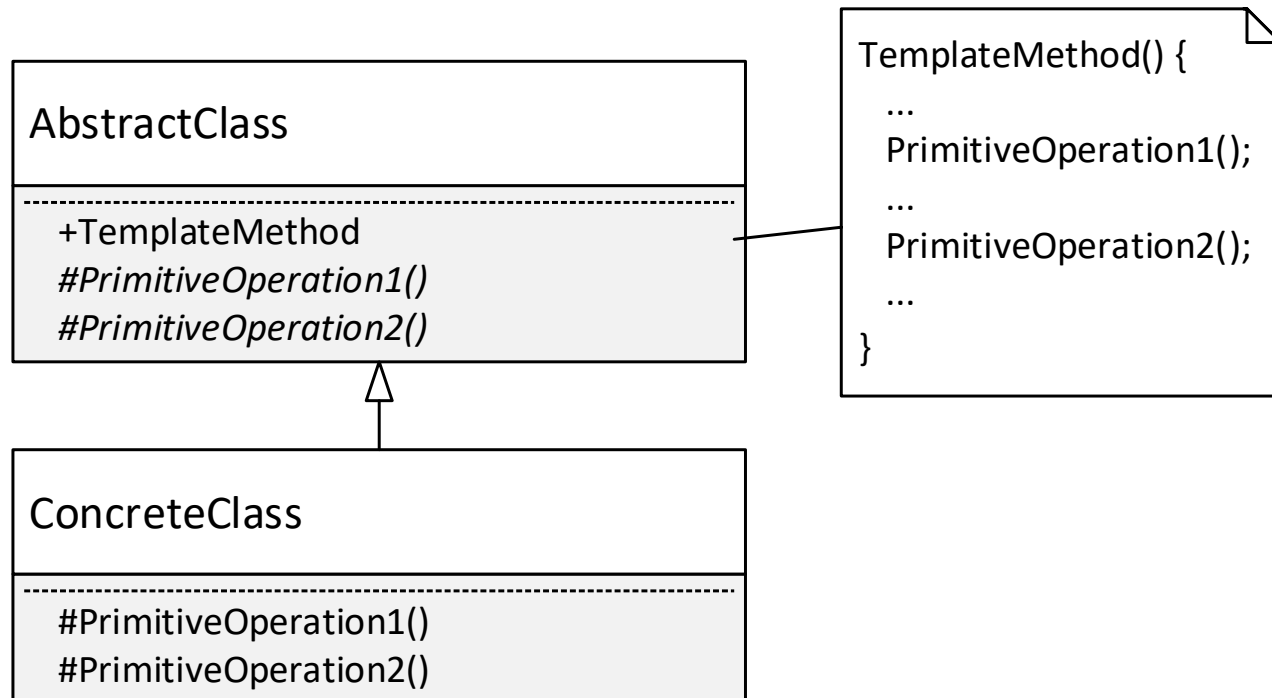
- Egy műveleten belül algoritmus vázat definiál, és az algoritmus bizonyos lépéseinek implementálását a leszármazott osztályra bízta.
 - > A példánkban az algoritmus váz a Run függvény volt, a leszármazottra bízott lépések az InitCompression, CompressData, CloseCompression, IsCancelled

Általánosítsuk



Template Method általánosan

- Amit eddig néztünk (DataProcessor), az csak egy **példa** volt a Template Method mintára. **Maga a minta teljesen általános:**



Template Method következmények

- Következmények

- > Lehetővé teszi, hogy az algoritmus/folyamat invariáns részeit egy helyen definiáljuk és a változó részeket a leszármazott osztályban adjuk meg.
- > Így megoldható a kódduplikálás elkerülése (DRY elv!): a hierarchiában a közös kódrészeket a szülő osztályban egy helyen adjuk meg (template method-ban), mely a különböző viselkedést megvalósító egyéb műveleteket hívja meg. Ezeket a “különböző viselkedést megvalósító egyéb műveleteket” a leszármazott osztályban felül kell/lehet definiálni.
- > Lehetővé teszi kiterjesztési pontok definiálását a kódban (ún. hook függvényeknek is szokás nevezni)

- Megjegyzés

- > Keretrendszer esetében gyakori

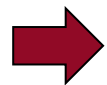
Kiértékelés

- Pozitívum
 - > Kódduplikáció elkerülése (lásd előző dia)
 - > **Új viselkedés könnyen bevezethető**, nem kell a meglévő kódot (lényegi helyen) változtatni
 - A példában DataProcessorBase kódjához nem kell hozzányúlni, csak le kell belőle származtatni egy új osztályt
- Negatívum
 - > Futás közben nem tudjuk egyszerűen cserélni viselkedést, rugalmatlan
 - A származtatási hierarchia fordításkor eldől
 - > Sok keresztkombinációra lehet szükség, lásd következő diák

Ha a viselkedésnek több aspektusa/dimenziója van

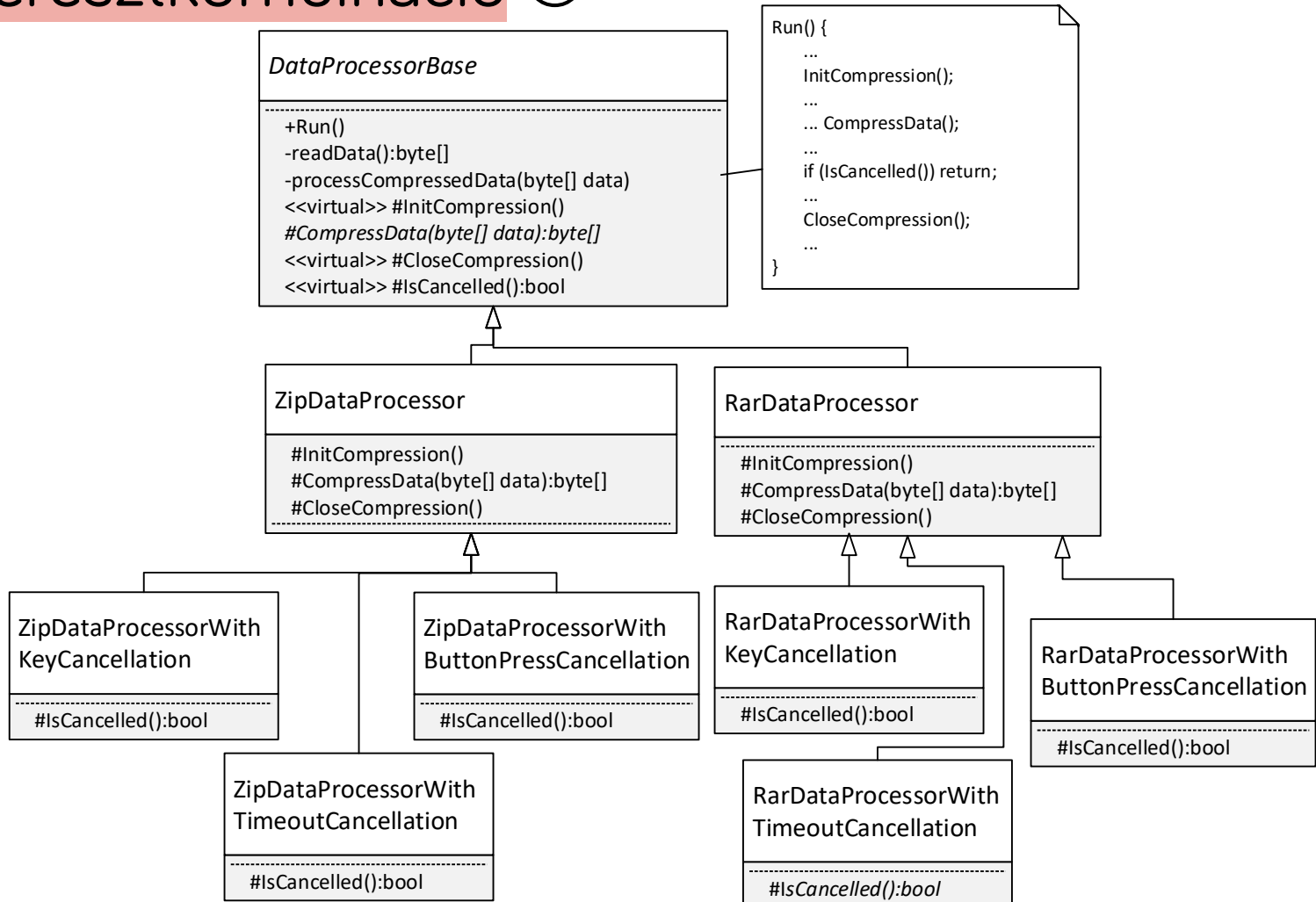
- DataProcessor példa aspektusok (kettő van)
 - > Tömörítés mód: Zip, Rar, 7Zip
 - > Cancel mód: billentyű lenyomás, timeout, gombkattintás (pl. UWP Cancel gomb)
- Minden használt kombinációnak egy külön leszármazott osztály kell
 - > Áttekinthetetlen és karbantarthatatlan hierarchiát eredményez, ha sok a kombináció

Példa



DataProcessor példa

Sok keresztkombináció ☹️



Sok keresztkombináció

A megoldást a Strategy minta alkalmazása jelenti
Rövidesen ...

Előfordulása

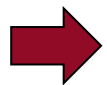
- Egyszerűbb esetekben gyakran használjuk
- Keretrendszeres esetén gyakori, pl. le kell származtatni a keretrendszer beépített Window, Page, Application, Control, stb. osztályából és virtuális függvényeket lehet/kell felüldefiniálni

Strategy (Stratégia)

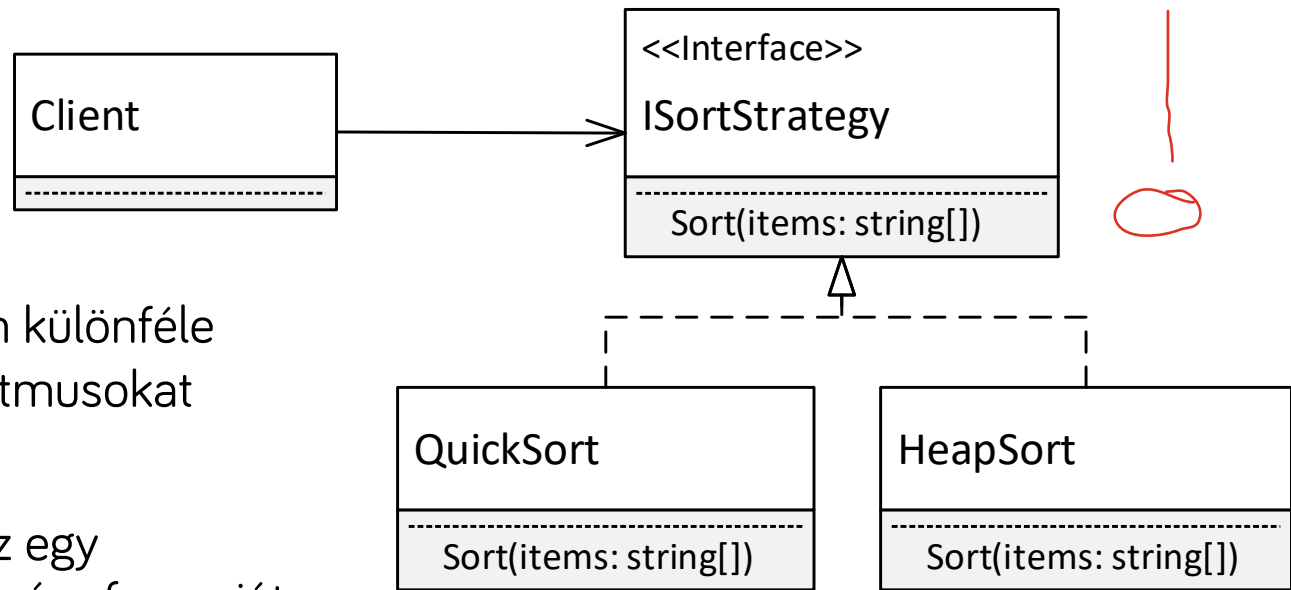
Strategy minta célja

- Célja az algoritmusok/viselkedések egy csoportján belül az egyes algoritmusok/viselkedések egységbe zárása és egymással kicserélhetővé tétele.
 - > A kliens szemszögéből az általa használt algoritmusok/viselkedések szabadon kicserélhetők.

Ez így nehezen érthető, nézzünk egy egyszerű példát



Strategy egyszerű példa - sorrendezés



- A kliens objektum különféle sorrendező algoritmusokat használhat
- A kliens tartalmaz egy **ISortStrategy** típusú referenciát egy konkrét (QuickSort v. HeapSort) objektumra.
- A referencia által hivatkozott implementációs objektumot könnyű kicserélni.

Példakód

- Kód, lásd „Strategy – Sort” mappa

```
// Strategy interfész
interface ISortStrategy
{
    void Sort(string[] items);
}

// QuickSort strategy implementáció
class QuickSort : ISortStrategy
{
    public void Sort(string[] items)
    {
        Console.WriteLine("Sorting items using quick sort algoritm ...");
    }
}

// HeapSort strategy implementáció
class HeapSort : ISortStrategy
{
    public void Sort(string[] items)
    {
        Console.WriteLine("Sorting items using heap sort algoritm ...");
    }
}
```

Példa folytatása

```
// Kliens, a Sort metódusa az aktuálisan beállított stratégiával rendez
class Client
{
    ISortStrategy sort; // Hivatkozás az aktuálisan beállított stratégia implementációra

    // Lehetőséget biztosít a stratégia beállítására
    public void SetSortStrategy(ISortStrategy sort)
    {
        this.sort = sort;
    }

    public void Sort(string[] items)
    {
        sort.Sort(items); // Sorrendezés az aktuális stratégiával
    }
}

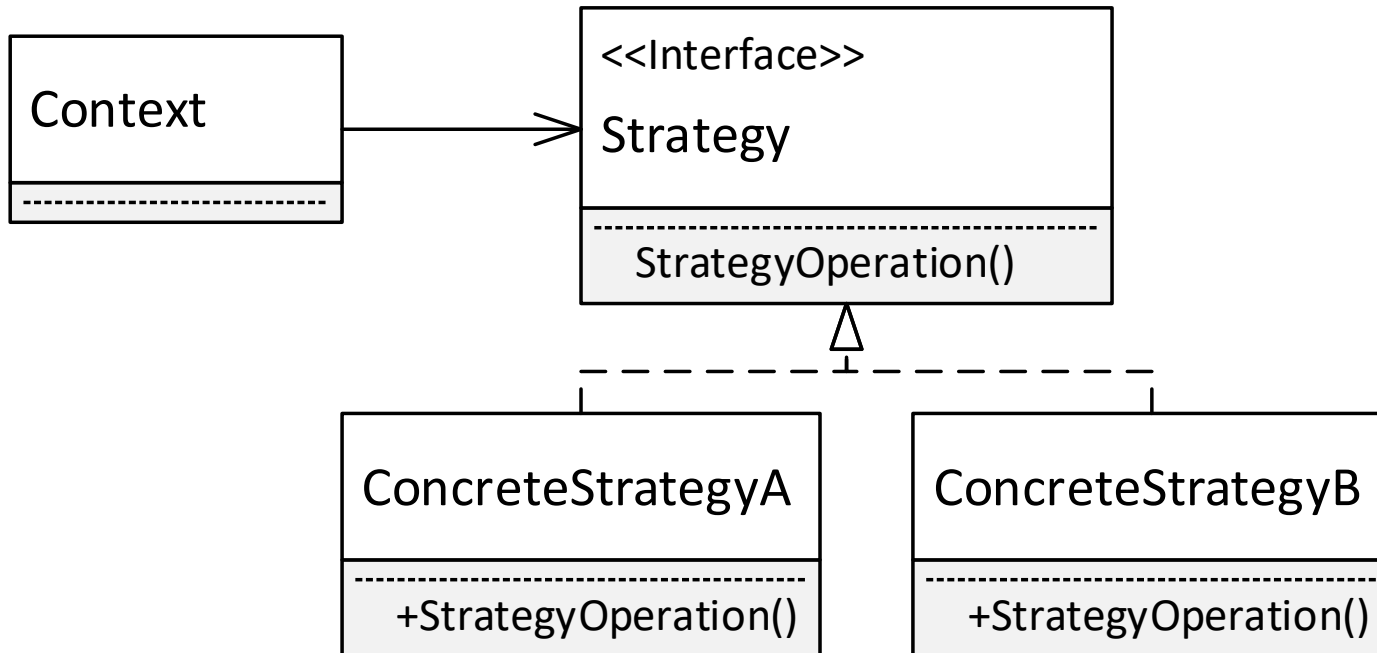
class Program
{
    static void Main(string[] args)
    {
        var items = new string[] { "körte", "alma", "szilva" };

        Client client = new Client();

        // Kezdetben használjuk a quick sort algoritmust
        client.SetSortStrategy(new QuickSort());
        client.Sort(items);

        // Mostantól a heap sort algoritmust használjuk
        client.SetSortStrategy(new HeapSort());
        client.Sort(items);
    }
}
```

Strategy - általános osztálydiagram



- **Strategy** interfész név mellett szokás a **Behavior** illetve **Policy** nevek használata is.
- Lehet több összetartozó művelet is (az ábrán csak egy szerepel)

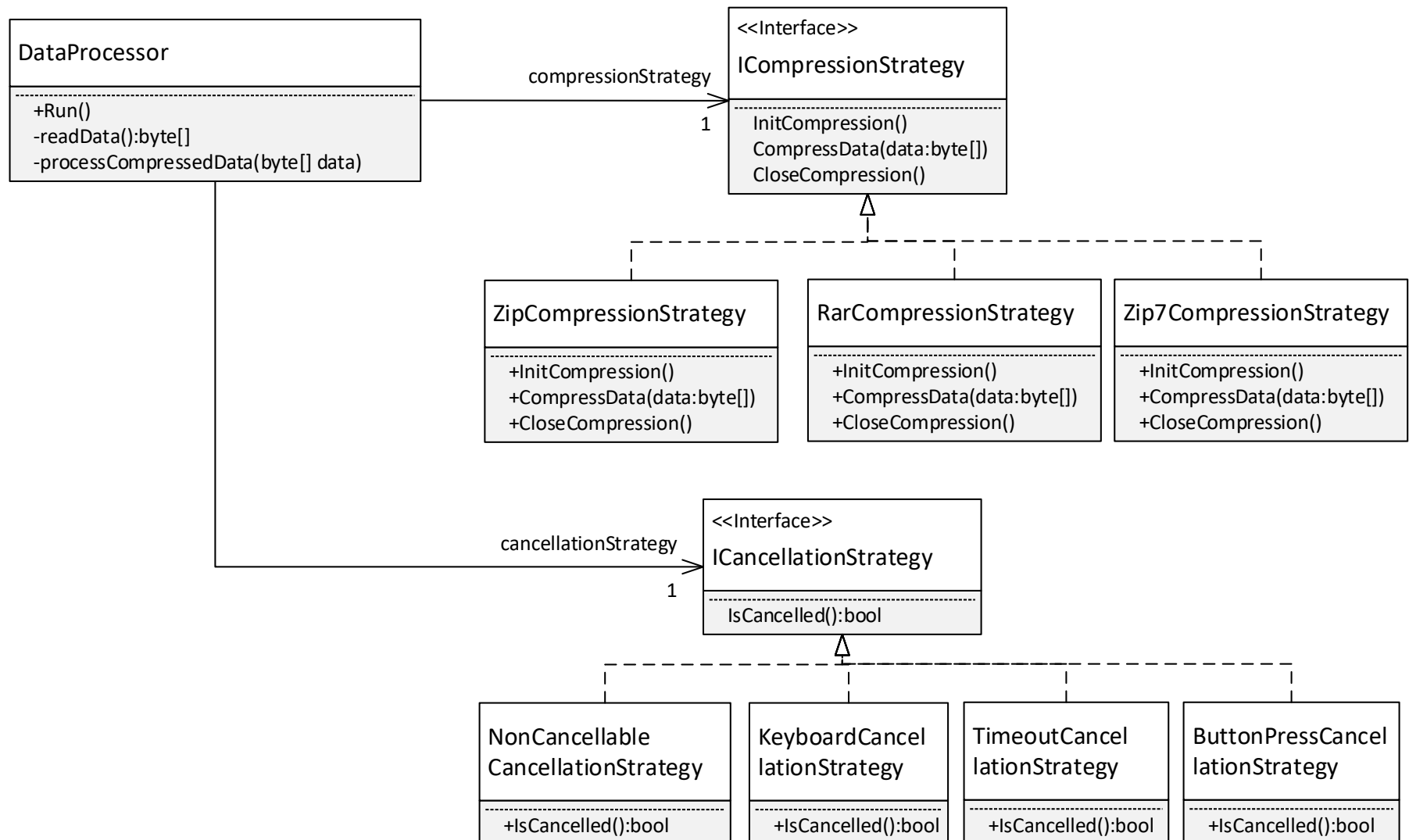
Strategy, DataProcessor példa

- Az osztály viselkedésének minden olyan aspektusára/dimenziójára, melyet lecserélhetővé/bővíthetővé szeretnénk tenni, külön strategy hierarchiát vezetünk be
 - > **Tömörítés mód variációk:** Zip, Rar, 7Zip
 - Interfész: ICompressionStrategy
 - Implementációk: ZipCompressionStrategy, RarCompressionStrategy, Zip7CompressionStrategy
 - > **Cancel mód variációk:** nem megszakítható, billentyű lenyomás, timeout, gombkattintás (pl. UWP Cancel gomb)
 - Interfész: ICancellationStrategy
 - Implementációk: NonCancellableCancellationStrategy, KeyboardCancellationStrategy, TimeoutCancellationStrategy, ButtonPressCancellationStrategy
- Az osztály külön (interfész típusú) hivatkozást tartalmaz minden viselkedés aspektusra/dimenzióra
 - > ICompressionStrategy tag
 - > ICancellationStrategy tag

Lássuk a gyakorlatban



DataProcessor osztálydiagram



DataProcessor példa, kód

- Kód, lásd DesPattCode\Strategy mappa
 - > Nincs minden osztály megvalósítva
 - > A DataProcessor konstruktorban kapja meg a függőségeit, egy ICompressionStrategy és ICancellationStrategy objektumot
 - Ha menet közben is meg akarjuk változtatni valamelyiket, egy megfelelő Setter függvényt vezessünk be (lásd korábbi sorrendező példa)

DataProcessor példa, kód

```
class DataProcessor
{
    ICompressionStrategy compressionStrategy; // Aktuális cancel stratégia/viselkedés
    ICancellationStrategy cancellationStrategy; // Aktuális tömörítési stratégia/viselkedés

    public DataProcessor(ICCompressionStrategy compressionStrategy,
        ICancellationStrategy cancellationStrategy)
    {
        ...
        // Eltároljuk a paraméterként kapott stratégiákat
        this.compressionStrategy = compressionStrategy;
        this.cancellationStrategy = cancellationStrategy;
    }

    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        byte[] inputData;
        compressionStrategy.InitCompression(); // Tömörítés inicializás, impl. függő
        try
        {
            while ((inputData = readData()) != null)
            {
                // Tömörítés, impl. függő
                byte[] compressedData = compressionStrategy.CompressData(inputData);
                processCompressedData(compressedData);
                if (cancellationStrategy.IsCancelled()) // Cancel vizsgálat, impl. függő
                    return;
            }
        }
        finally
        {
            compressionStrategy.CloseCompression(); // Tömörítés lezárás, impl. függő
        }
    }
}
```


DataProcessor példa, kód

- A stratégiák bármilyen kombinációban egyszerűen használhatók!

```
static void Main(string[] args)
{
    // A stratégiák bármilyen kombinációban egyszerűen használhatók. Nézzünk párat!
    // Jelen esetben a Zip algoritmust választjuk, és billentyű alapú Cancel lehetőséget
    var processor = new DataProcessor(
        new ZipCompressionStrategy(),
        new KeyboardCancellationStrategy(ConsoleKey.X)
    );
    processor.Run();

    // Második esetben a Rar algoritmust választjuk, és nem akarunk Cancel lehetőséget
    // biztosítani
    var processor2 = new DataProcessor(
        new RarCompressionStrategy(),
        new NonCancellableCancellationStrategy()
    );
    processor2.Run();
}
```

Kiértékelés - pozitívumok

- Bővíthetőség

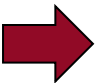
- > Új viselkedés könnyen bevezethető, nem kell a meglévő kódot (lényegi helyen) változtatni

- A példában a meglévő DataProcessor kódjához nem kell hozzányúlni, csak új ICompressionStrategy vagy ICancellationStrategy implementációt kell bevezetni

- > Több aspektus/dimenzió esetén nincs kombinatorikus robbanás az osztályhierarchiában

- Unit tesztelhetőséget segíti

Példa



Kiértékelés – unit tesztelhetőség

- Unit tesztek során az osztályt önmagában, a függőségei nélkül teszteljük (ezért is hívják UNIT tesztnek).
- Példa: a DataProcessor tesztelése során nem kívánjuk tesztelni azt, hogy az adatok tömörítése, vagy a cancel során a billentyű lenyomás hogyan történik.
 - > Pl.
 - ha hiba van a zip algoritmusban, és az elesik, akkor az **ne akassza meg** a DataProcessor unit tesztjeit
 - A tömörítés lassú, **ne lassítsa** ez feleslegesen a DataProcessor unit tesztjeit
 - > A tömörítés és cancel unit tesztelését más, ezekre dedikált unit tesztek kell végezzék.
 - > Ha ezek implementációja be van építve a DataProcessor osztályba, akkor a DataProcessor nem unit tesztelhető

Kiértékelés – unit tesztelhetőség

- A Strategy minta segíti, hogy a DataProcessor unit tesztelhető legyen. Hogyan? →
- Amikor a DataProcessor osztályt unit teszteljük, olyan „dummy” ICompressionStrategy és ICancellationStrategy implementációkat adunk át, melyek nem csinálnak semmit, vagy magát a tesztet segítik
 - Pl. DummyCompressionStrategy, mely nem tömörít, csak visszaadja az eredeti adatot
 - Pl. NonCancellableCancellationStrategy, mely nem Cancel-el, illetve egy AlwaysCancelCancellationStrategy, mely azonnal automatikusan cancel-el (attól függően, hogy mit tesztelünk éppen).
 - Konkrét példa a félév végén tesztelésről szóló gyakorlaton (?)

DataProcessor példa, SRP elv

- A DataProcessor legutolsó megvalósítása valóban teljesíti az SRP elvet, az osztálynak valóban egyetlen felelőssége van?
- Nem, még mindig több felelőssége van!
 - > A Run-ban a folyamat vezérlése
 - > Adatbeolvasás logikája (readData művelet)
 - > Adatfeldolgozás logikája (processCompressedData művelet)
- Az adatbeolvasás és adatfeldolgozás logikáit is ki lehetne faktorálni az osztályból, a compression és a cancel mintájára interfészek mögé lehet tenni
 - > Ez segíteni a unit tesztelhetőséget, és a bővíthetőséget
 - > Gyakorlásképpen megcsinálható

Strategy

- **Program to an interface** (and not to an implementation) elv egyik megtestesülése

Kiterjeszthetőség összefoglaló

Áttekintés

- SOLID Open/Closed elv
- Lehetőségek
 - > Template method
 - Leszármazottra bízva a kiterjesztést
 - Egyszerűbb
 - Kevésbé rugalmas
 - A leszármaztatás miatt (a leszármazási hierarchia fordításkor dől el, futás közben nem változtatható)
 - Több aspektus: áttekinthetetlen, sok keresztkombináció
 - > Strategy
 - Más osztályokra bízva (delegál), ez a legrugalmasabb
 - > Van más lehetőség is → metódusreferencia, lambda használata

Egyéb kiterjeszthetőségi technikák

- Metódusreferenciák (delegate-ek), lambda kifejezések használata
 - > Egyre inkább terjed
 - > C#, C++, JavaScript nyelvek is támogatják
 - > Java-ban „csak” lambda kifejezések
- Mostantól C#-ot nézünk
 - > Azt a kódot, amely a kiterjeszthetőséget szolgálja, egy függvény/delegate vagy lambda formájában adjuk át
 - > A részletek "Eseményvezérelt és vizuális programozás,, tárgyból

Metódusreferencia, lambda használata

- Lásd DesPattCode\Delegate példa

```
class DataProcessor
{
    // ...

    Func<bool> onCancelled; // Metódusreferencia, konstruktorban kapja meg, Run-ban hívjuk

    public DataProcessor(Func<bool> onCancelled)
    {
        this.onCancelled = onCancelled;
    }

    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        ...
        if (onCancelled != null && onCancelled()) // Meghívjuk a kódot
            return;
        ...
    }
}

...
// Konstruktorban adjuk át a DataProcessor által hívandó kódot, pl. lambda formájában
var processor1 = new DataProcessor(() => { return false });
```

Kiterjeszthetőség, módosíthatóság

- Számos más minta is (gyakorlatilag valamennyi) a könnyű kiterjeszthetőséget, módosíthatóságot szolgálja, csak kevésbé direkt módon!

Szoftvertchnológia és -technikák

7. Előadás – Benedek Zoltán
Tervezési minták 2



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták 2

- > Observer
- > Létrehozási minták
 - Dependency Injection
 - Singleton
 - Abstract factory
 - (Factory method – nem tananyag)

Observer (Megfigyelő)

Observer célja

- Cél (két megközelítésben)
 - > Lehetővé teszi, hogy egy objektum (subject/alany) értesítést küldjön más objektumoknak (observer/megfigyelő) az állapotának változásairól. Mindezt anélkül, hogy az alany függene a megfigyelők konkrét típusától.
 - > Lehetővé teszi, hogy objektumok értesíteni egymást állapotuk megváltozásáról anélkül, hogy függőség lenne köztük a konkrét osztályaiktól (ez az absztraktabb definíció, mélyebb célokat fogalmaz meg).

Még nem értjük, lássunk egy példát



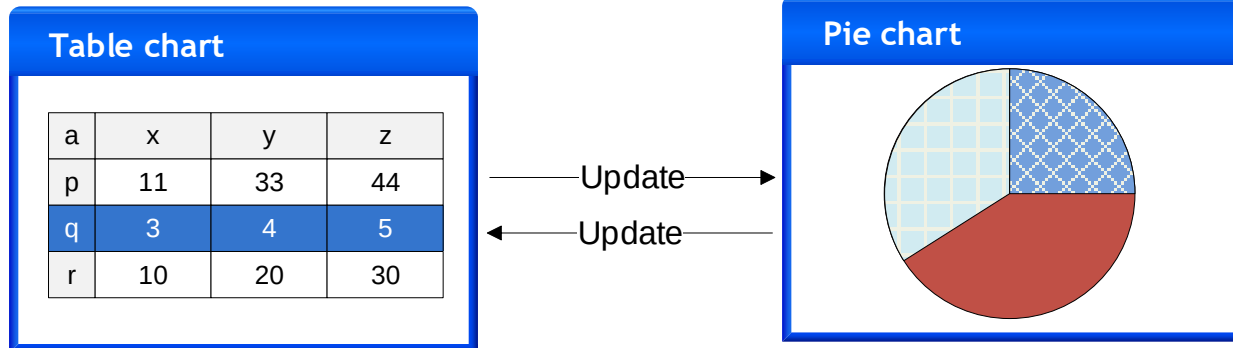
Példa

- Egy táblázatkezelő alkalmazásban támogatni kell az adatok különböző megjelenítését
 - > Első körben **TableView** és **PieChartView**
 - > T.f.h. a felhasználó mindkettőben tudja módosítani az adatokat
 - > Meg kell oldani, hogy a két nézet konzisztens nézetét mutassa az adatoknak. Bármelyikben módosul az adat, azt jeleznie kell a másiknak.
 - Megoldás első körben: mindkét osztály tartalmaz egy hivatkozást a másikra

Ábra és kód



Példa



```
class TableView
{
    // ...

    double[,] data;
    PieChartView pieChartView;

    public void Update(double[,] data) {
        /* this.data beállítása data param alapján */
    }
    public void SetCellData(int col, int row, double value)
    {
        this.data[col, row] = value;
        pieChartView.Update(data);
    }
    public void Draw() { /* ... */ }
}
```

A PieChartView kódja hasonló, csak a TableView-ra tartalmaz hivatkozást.

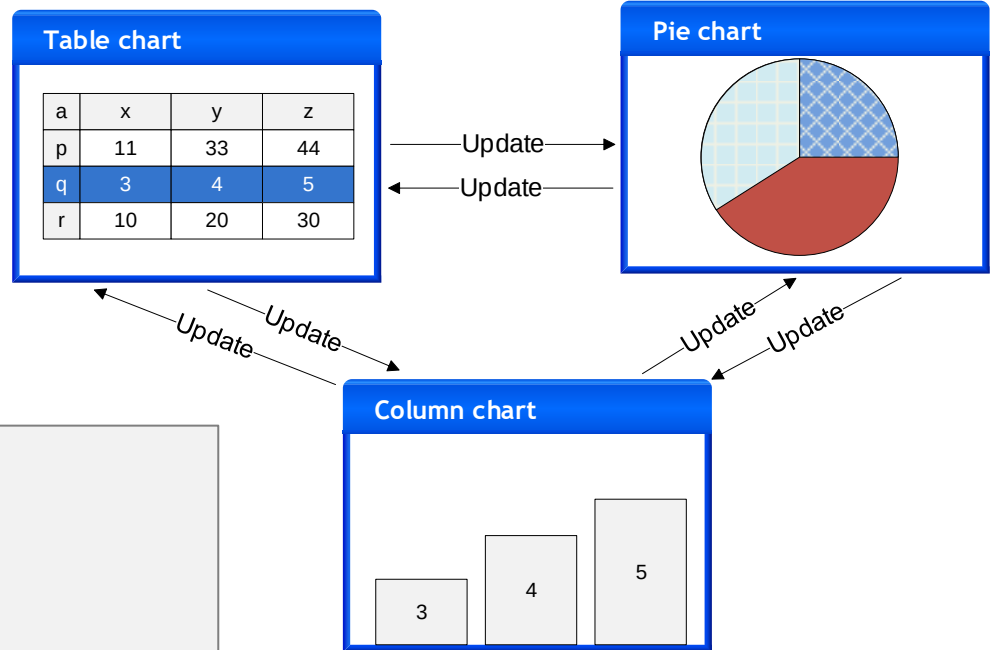
Példa

- T.f.h bővíteni kell a megoldásunkat
 - > Be kell vezetni az oszlopdiagramos megjelenítést (**ColumnChartView**)
 - > T.f.h. ebben is lehet módosítani az adatokat
 - > Konzisztens nézetek: bármelyik nézet módosul, frissíteni kell a másik kettőt is!
 - > Új nézetet szeretnék bevezetni, minden MEGLÉVŐ nézet osztályt módosítani kell ☹️☹️☹️
 - Fel kell vegyük a meglévő osztályokba a hivatkozást az új nézetre

Ábra és kód



Példa



```
class TableView
{
    // ...

    double[,] data;
    PieChartView pieChartView;
    ColumnChartView columnChartView;

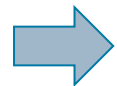
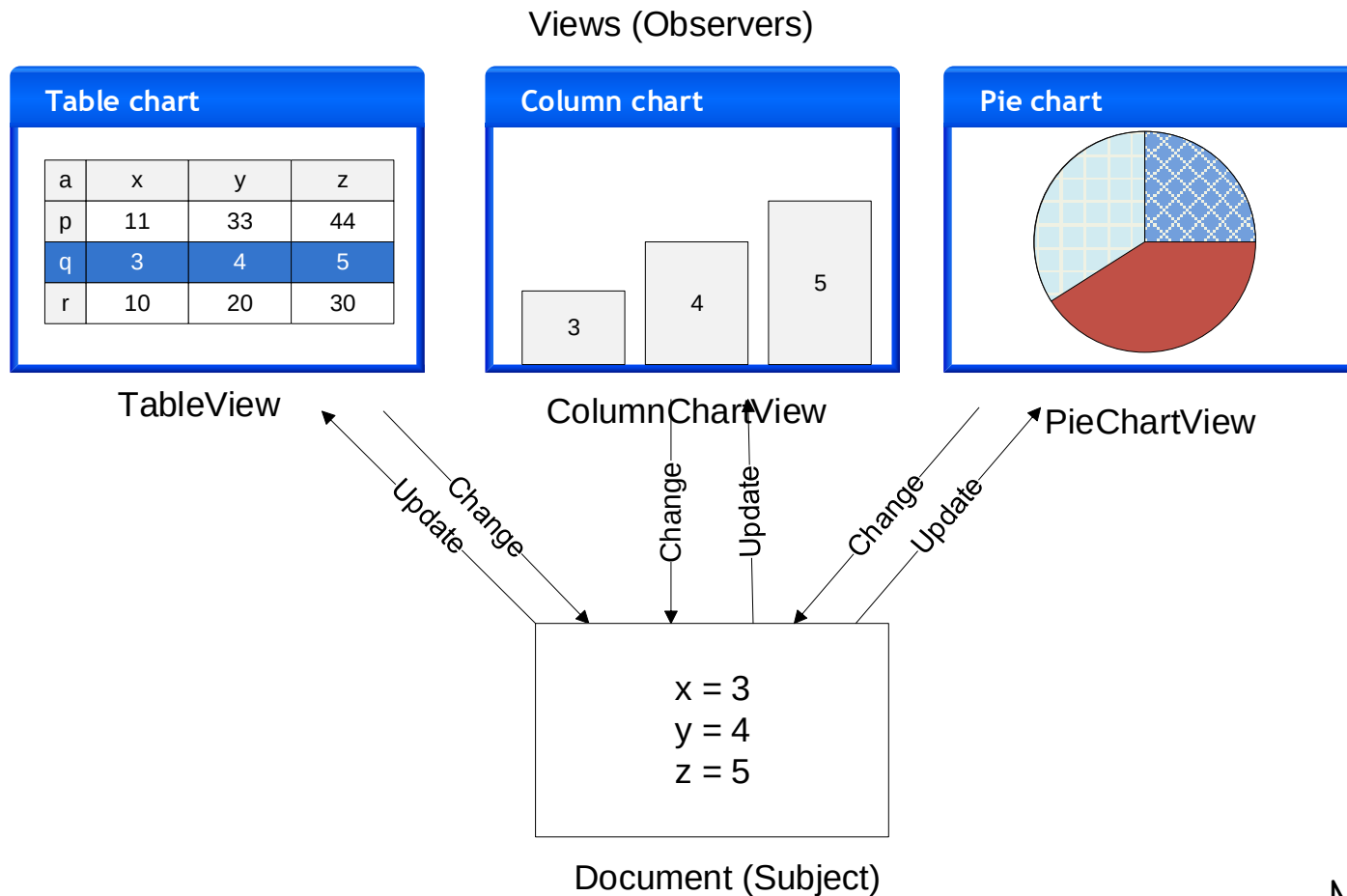
    public void Update(double[,] data) {
        /* this.data beállítása data param alapján */
    }
    public void SetCellData(int col, int row,
        double value)
    {
        this.data[col, row] = value;
        pieChartView.Update(data);
        columnChartView.Update(data);
    }
    public void Draw() { /* ... */ }
}
```

A PieChartView és a ColumnChartView kódja hasonló: a másik két nézetre tartalmaz hivatkozást

Példa kiértékelése

- A megoldásunk (mindenki-mindenkivel összekötve, közvetlen függvényhívással) eddig is nehezen volt bővíthető
 - > A további bővítés teljes rémálom
 - > Áttekinthetetlen + új nézet bevezetésekor mindig módosítani kell a meglévő nézeteket is
 - > Nehéz karbantartani, továbbfejleszteni, újra felhasználni részeket, mert túl szoros a csatolás az osztályok között (a nézetek függenek egymástól).
- Alkalmazzuk a példánkban az **Observer** mintát

Nézetek frissítése Observer mintával



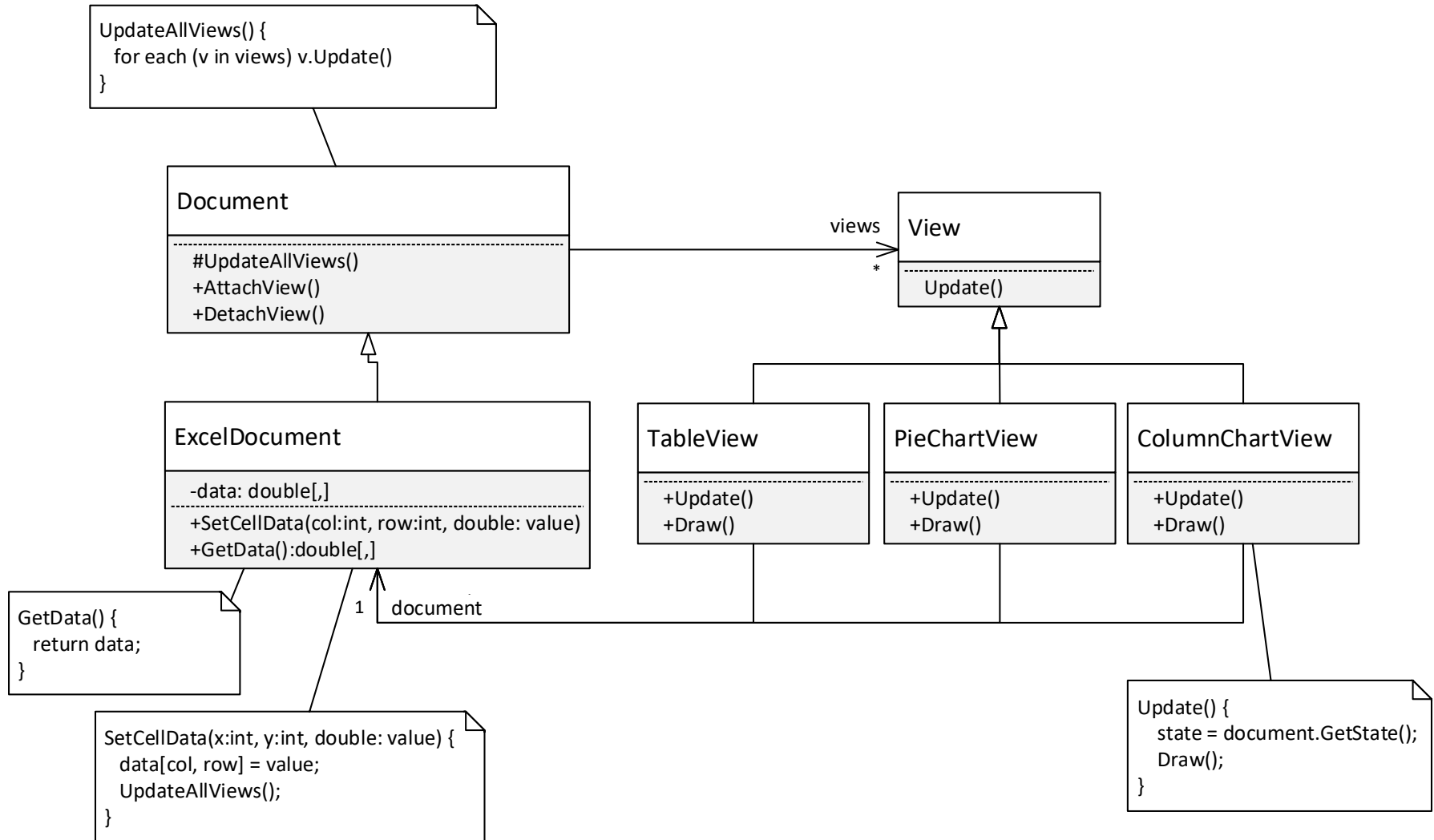
Magyarázat

Nézetek frissítése Observer mintával

- **Magyarázat, minden pont fontos!**

- > Az adatoknak egyetlen, közös „hiteles” forrása legyen: emeljük ki az adatokat és az azon értelmezett műveleteket egy osztályba, ez lesz a **dokumentum** (subject)
- > A dokumentumhoz különböző **view**-kat lehet beregisztrálni (observer)
- > Ha valamelyik view megváltoztatja a dokumentum adatait, a dokumentum értesíti az összes beregisztrált view-t a változásról
- > Az értesítés hatására a view-k lekérdezik a dokumentum állapotát és frissíti magát
- > A dokumentum csak egy közös View interfészen/ősoosztályon keresztül tárolja a beregisztrált view-kat (nem függ az egyes nézet típusoktól)

Példa, osztálydiagram



Osztálydiagram magyarázat

- **Document**

- > Az osztály objektumai reprezentálnak egy-egy dokumentumot, és a *views* nevű listájában tárolja a beregisztrált nézeteket.
- > Tartalmazza a dokumentumokra közös kódot, ha több különböző típusú dokumentum is van (a példánkban nincs)

- **ExcelDocument**

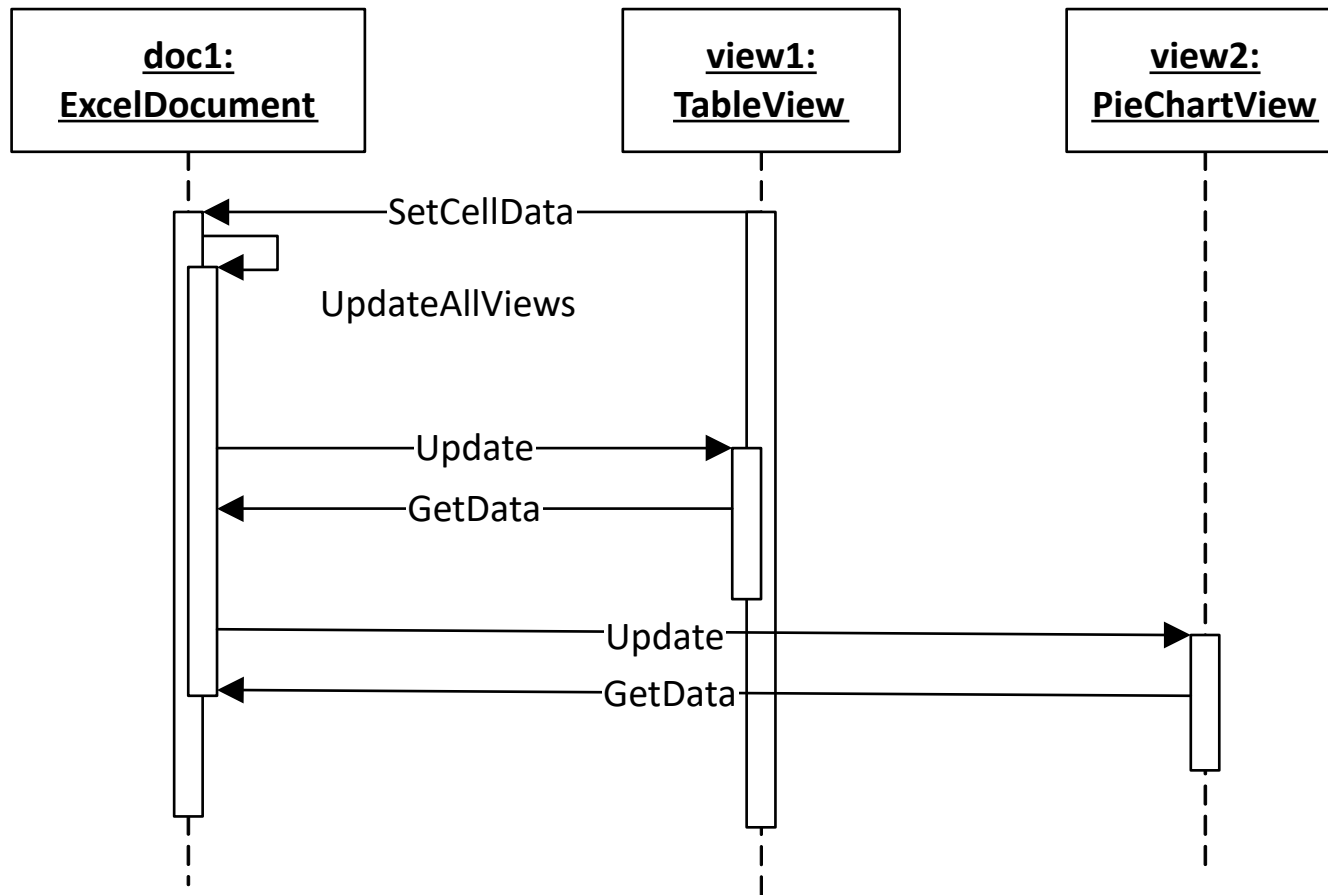
- > Egy példa *Document* leszármazottra.
- > Tárolja a *data*, stb. tagváltozójában a dokumentum adatait (pl. cella értékek).
- > Publikus lekérdező függvényeket biztosít a többi osztály számára az adatokhoz (elsősorban a nézet számára), pl. *GetData*.
- > Publikus adatmódosító műveleteket biztosít a többi osztály számára, pl. *SetData*. Ezek módosítják a tagváltozókat, majd az *UpdateAllViews* hívásával értesítik a többi nézetet a változásról.
- > Az *UpdateAllViews* frissíti a beregisztrált nézeteket (Update-et hív mindre).

Osztálydiagram magyarázat

- **View**
 - > Az egyes nézetek közös őse vagy interfésze, lehetővé teszi egységes kezelésüket.
- TableView, PieChartView, stb.
 - > A dokumentum egyes nézeteit reprezentálják.
 - > A *View*-ből származnak (vagy a *View* interfészt implementálják, ha a *View* interfész)
 - > Felüldefiniálják vagy implementálják a *View Update* műveletét. Az *Update* műveletben a nézet a dokumentum aktuális állapota alapján frissíti magát.
 - > Tartalmaznak egy referenciát a dokumentumra, amin keresztül a leszármazott nézetek elérhetik a dokumentumot, melynek adatait megjelenítik.
 - Alternatív megoldás lehet, ha a *View* közös őse tartalmazza ezt a referenciát a *Document* ősről
- Nézetben hivatkozás a *Document* (subject)-re, két lehetőség
 - > A példában a *View* implementációkba tettük, feltételezve, hogy a *View* egy interfész
 - > Ha a *View* osztály (nem interfész), akkor a *View* ősebe is tehető.
 - A gyakorlatban a *View*-kat a .Net, Java, stb. keretrendszer *Form/Window/Control*/egyéb osztályából származtatjuk

Példa szekvenciadiagram

- Pár diával korábban szövegesen már szerepeltek a lépések



Példa kód

- Kód: lásd *ObserverDocView* mappa
- Egy kicsit szofisztikáltabb változat:
 - > A nézet vonatkozásában van *IView* interfész, de egy *ViewBase* közös őosztály is: ez utóbbiba a nézetekre közös kód került.
- Az egyszerűség kedvéért egy ez konzol alkalmazás, a *ColumnChartView* és a *PieChartView* így természetesen nem rajzol

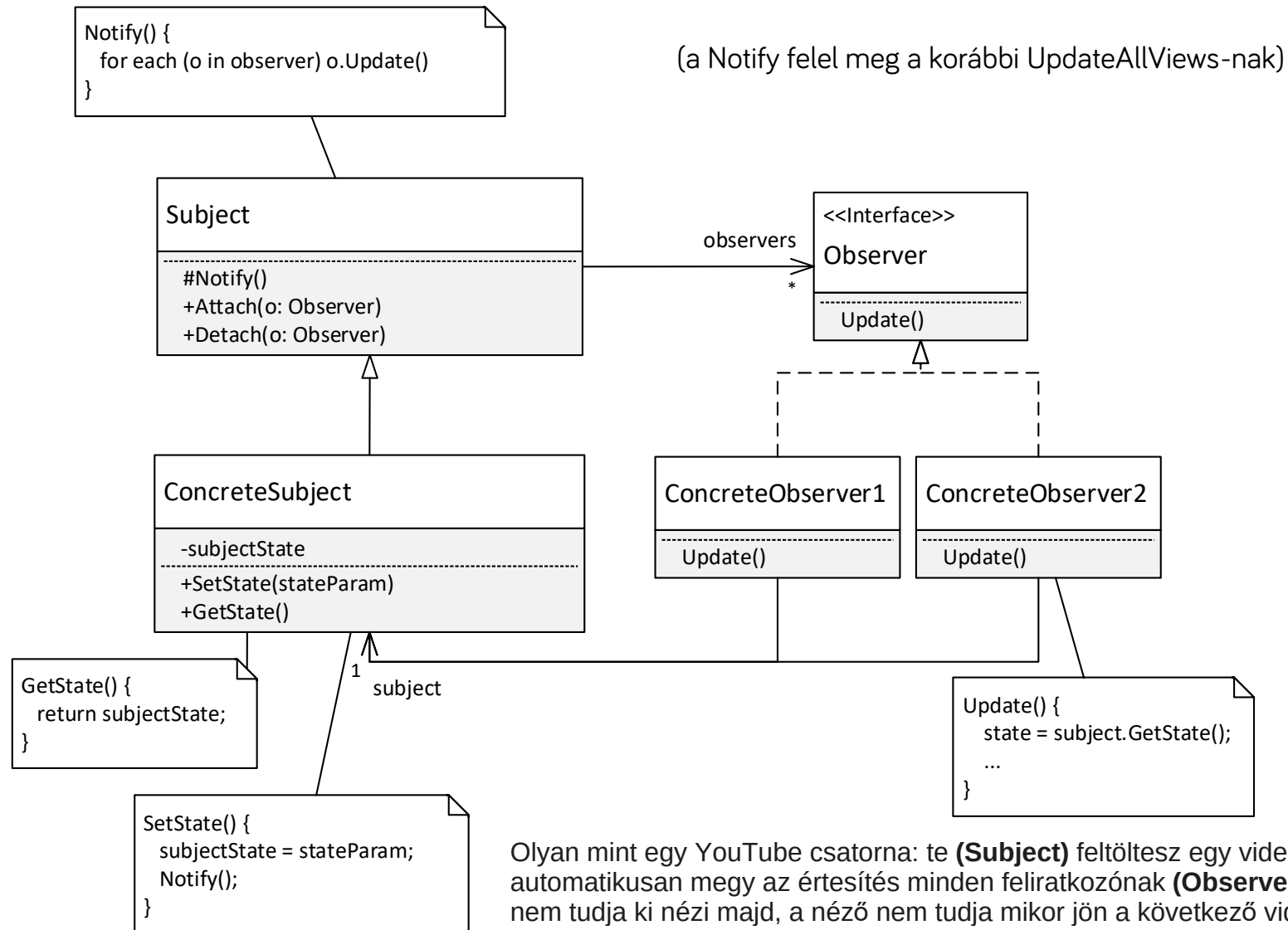
Példa összefoglaló

- Ez egy példa volt, az Observer minta alkalmazására egy speciális esetre, adatok megjelenítésére
 - > Ez maga a Document-View architektúra, még visszatérünk rá pár előadás múlva
- Observer általánosítva
 - > Lehetővé teszi, hogy egy objektum (subject/alany) értesítést küldjön más objektumoknak (observer/megfigyelő) az állapotának változásairól. Mindezt anélkül, hogy az alany függene a megfigyelők konkrét típusától.
- További példák gyakorlaton

Observer

- Előnyök
 - > A rendszer könnyen kiterjeszthető új view osztályokkal. Sem a document (subject), sem a többi view (observer) osztályt nem kell ehhez módosítani.
 - Próbáljuk ki, vezessünk be egy új nézet osztályt!
 - > Egy egyszerű mechanizmust kaptunk arra, hogy az összes view konzisztens nézetét mutassa az adatoknak.
 - > A document (subject) kódjában csak egy *View* (observer) lista van, így a modell független az egyes *View-t* implementáló osztályoktól. A document (subject) újrafelhasználható!
- Általánosítva, elvonatkoztatva a document-view esettől
 - > A fenti megközelítés lehetővé teszi, hogy egy objektum megváltozása esetén értesíteni tudjon tetszőleges más objektumokat anélkül, hogy bármit is tudna róluk
 - > Ez az ún. *observer* minta lényege

Observer osztálydiagram

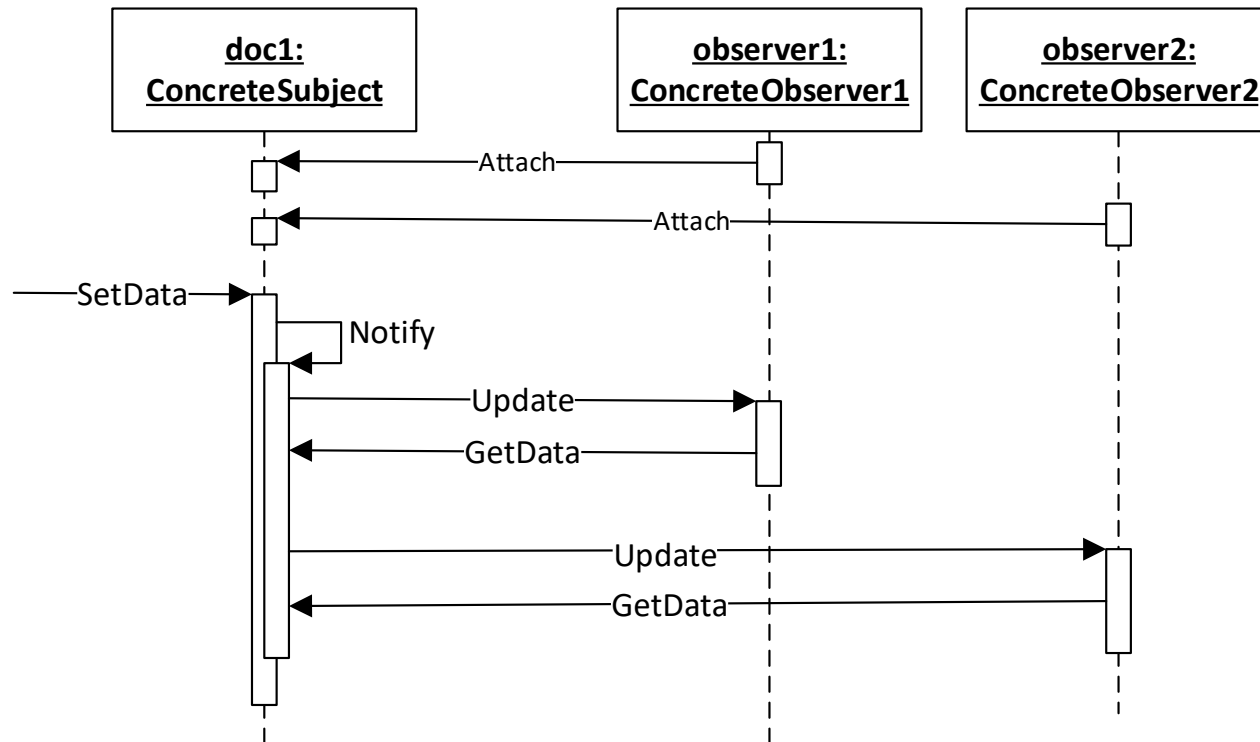


Olyan mint egy YouTube csatorna: te (**Subject**) feltöltesz egy videót → automatikusan megy az értesítés minden feliratkozónak (**Observer**). A csatorna nem tudja ki nézi majd, a néző nem tudja mikor jön a következő videó.

Observer - szereplők

- **Subject**
 - > Tárolja a beregisztrált Observer-eket
 - > Lehetőséget ad Observer-ek be- és kiregisztrálására valamint értesítésére. Ezáltal „tartalmazza” a különböző ConcreteObserver-ek közös kódját (közös ős).
 - > A gyakorlatban nem jelenik meg mindig
- **Observer**
 - > Interfészt definiál azon objektumok számára, amelyek értesülni szeretnének a Subject-ben bekövetkezett változásról (Update művelet)
- **ConcreteSubject**
 - > Az observer-ek számára érdekes állapotot tárol
 - > Értesíti a beregisztrált Observer-eket, amikor az állapota megváltozik
- **ConcreteObserver**
 - > Referenciát tárol a megfigyelt ConcreteSubject objektumra
 - > Olyan állapotot tárol, amit a megfigyelt ConcreteSubject állapotával konzisztensen kell tartani
 - > Implementálja az Observer interfészt (Update művelet), ez az, amit a Subject meghív, amikor a ConcreteSubject állapota megváltozik. Ebben frissíti a saját állapotát a megfigyelt ConcreteSubject állapotának megfelelően

Observer dinamikus nézet



- A subject állapotváltozását (**SetData hívás**) nem csak observer válthatja ki! Ezért az ábrán ezt egy kívülről jövő művelettel jelöltük.
- Megjegyzés: „Sajnos” az UML szekvencia diagram csak objektumokat ábrázol, nem képes kifejezni, hogy milyen interfészen keresztül hivatkoznak egymásra az objektumok, így a függetlenséget nem tudja kifejezni

Observer

- Használjuk, ha
 - > Amikor egy objektum megváltoztatása maga után vonja más objektumok megváltoztatását, és nem tudjuk, hogy hány objektumról van szó
 - > Amikor egy objektumnak értesítenie kell más objektumokat az értesítendő objektum szerkezetére vonatkozó feltételezések nélkül

Observer – záró gondolatok

- Gyakran használt minta
- Laza csatolás megvalósításával segíti a könnyű bővíthetőséget
 - > Az observerek nem direktben kommunikálnak egymással, egymásról nem is tudnak, csak a subject-et ismerik
 - > A subject nem függ az egyes observerek konkrét típusától (csak egy observer interfésztől)
 - > Új observer bevezetésekor nem kell módosítani sem a subject-et, sem a többi observert, csak implementálni kell az observer interfészt
- A .NET esemény (event) is ennek egy speciális megjelenése: egy osztály minden event tagja egy „subject”, amire a += operátorral iratkoznak fel az observerek

Observer – záró gondolatok

- A példáinkban változás esetén a subject nem adja át az observernek az adatot. Csak a változás tényét jelzi, az observer egy plusz lépésben kéri le az adatot.
 - > Teljesen érvényes megoldás lehet, ha a subject elküldi a változott adatot, paraméterben átadva az observernek (különösen, ha kisebb méretű az adat)
 - > Esetfüggő, melyik megközelítés a praktikusabb

LÉTREHOZÁSI MINTÁK

Dependency Injection (DI)

Abstract factory

Singleton

(Factory Method – nem tananyag)

Létrehozási minták

- Objektumok létrehozásával kapcsolatos
- A new-val való közvetlen objektum létrehozás sokszor rugalmatlan
 - > Emlékezzünk a korábbi alapelvünkre: zárjuk egységbe a kód azon részeit, melyeknél arra számítunk, változni/bővülni fognak
 - > Ezek a részek gyakran az objektumok létrehozásával kapcsolatosak
 - Pl. nem akarjuk beégetni, milyen típusú objektumot hozzon létre egy adott kódsor (a new operátor esetében ez be van égetve, ez maga a típus, mely a new után áll, pl. *new TextDocument()* esetén TextDocument típus)
 - > Rövidesen nézünk példákat ...

Létrehozási minták

- Megoldási lehetőségek

- > Dependency Injection -DI
- > Factory Method
- > Abstract Factory
- > Singleton – ez egy kicsit speciális célokat szolgál
- > ...



Dependency Injection, DI (függőséginjektálás)

Dependency Injection

- Ez nem egy klasszikus tervezési minta, nincs a GoF könyvben, de logikailag ide passzol
- Egy példán keresztül világítjuk meg
 - > A célunk egy olyan **SecurityService** osztály megírása, mely felhasználókhoz kapcsolódó biztonsági szolgáltatásokat biztosít, úgymint jelszó megváltoztatása, login, stb.
 - > A felhasználókat valamilyen adatbázisban tároljuk.

SecurityService példa – kezdeti verzió

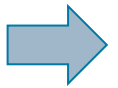
- Kód: lásd DesPattCode.DependencyInjection.Initial mappa

```
class SecurityService
{
    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        var userRepository = new UserRepository();
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid)
            return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
    ...
}
```

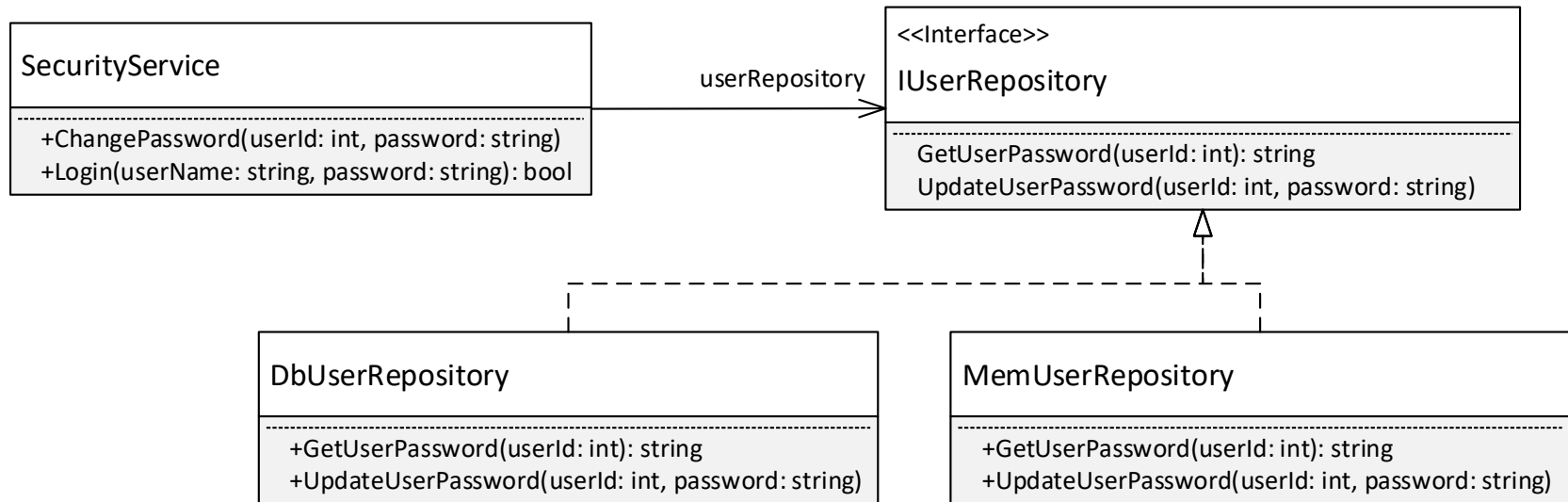


Magyarázat

SecurityService példa – kezdeti verzió

- Arra figyeltünk, hogy a felhasználók adatainak lekérdezése, módosítása (perzisztencia) le van választva egy **UserRepository** osztályba, hiszen az egy másik felelősségi kör (SRP elv).
- Probléma1: a **ChangePassword**-be a **UserRepository** osztály alkalmazása **be van égetve, csak ezzel tud dolgozni. Rugalmatlan**, csak adott adatbázis alapú felhasználó tárolással tud dolgozni. Ha más implementációt szeretnénk, a `var userRepository = new UserRepository()` sorokat mind át kell írni, ráadásul számos helyen.
- Probléma2: Az előző problémából adódóan a **ChangePassword** nem unit tesztelhető. A **ChangePassword** tesztelésénél NEM akarjuk tesztelni az adatbázis alapú perzisztenciát végző kódot, hanem csak a **SecurityService**-ben levő logikát. Ezen túl az adatbázis alapú perzisztencia nagyban feleslegesen lassítja a unit teszt futását.
- Megoldás: **alkalmazzuk a Strategy pattern**t, vezessünk be egy **IUserRepository** interfészt több lehetséges implementációval (adatbázis alapú az éles működéshez, memória alapú a unit teszteléshez), a **SecurityService** pedig interfészként hivatkozzon rá. Lásd WithStrategy mappa...

SecurityService példa – Strategy-vel



- Csak alkalmaztuk/gyakoroltuk a Strategy mintát
 - > **IUserRepository** – interfész
 - > **DbUserRepository** – egy implementáció, mely adatbázisba dolgozik (éles környezethez)
 - > **MemUserRepository** – egy implementáció, mely a memóriába dolgozik (így gyors) unit teszteléshez

SecurityService példa – Strategy-vel

- Lásd DesPattCode.DependencyInjection.WithStrategy mappa

```
class SecurityService
{
    // Interfészként hivatkozunk a user repository implementációra
    IUserRepository userRepository;

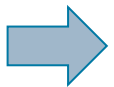
    // Egy központi helyen, a konstruktorban példányosít egy implementációt
    public SecurityService() { userRepository = new DbUserRepository(); }

    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid)
            return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
}
```



Magyarázat

SecurityService példa – Strategy-vel

- Az eredeti(Initial) **SecurityService** továbbfejlesztése.
- Interfészként hivatkozik a **UserRepository** implementációra: Ha más implementációt szeretnénk, akkor a műveleteket, pl. a **ChangePassword**-öt nem kell megváltoztatni:
- Még mindig maradt egy probléma, a konstruktorban a new-val a példányosítás! **A new után csak konkrét implementációt adhatunk meg. így ha át szeretnénk térni egy másik implementációra, akkor a SecurityService kódját még mindig módosítani kell** (igaz, már csak egy helyen, a konstruktorban), ez sérti az Open/Closed elvet (meg kell nyitni az osztályt módosításra).
- Megoldás: **Dependency Injection(DI)** alkalmazása, konstruktor, vagy esetleg metódus paraméterben adjuk át az implementációt a **SecurityService** osztálynak. Lásd WithStrategyAndDI mappa.

Dependency Injection (DI) alkalmazása a Strategy mellett)

- Lásd DesPattCode.DependencyInjection.WithStrategyAndDI mappa

```
class SecurityService
{
    // Interfészként hívkozunk a user repository implementációra
    IUserRepository userRepository;

    // Konstruktor paraméterként kap egy IUserRepository implementációt
    public SecurityService(IUserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid) return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
}
```

Dependency Injection (DI) alkalmazása a Strategy mellett)

- Folytatás, **DemoSecurityService** osztályban **SecurityService** használatra példák:

```
// Éles alkalmazásban a DbUserRepository-val használjuk a SecurityService-t
var securityService = new SecurityService( new DbUserRepository() );
securityService.ChangePassword(111, "abrakadabra");

// Unit teszt esetében a gyors MemUserRepository-val használjuk a
// SecurityService-t
var securityService2 = new SecurityService( new MemUserRepository() );
// ... teszt előkészítés ...
// tesztelendő kód futtatás
bool wasValid = securityService2.ChangePassword(111, "abrakadabra");
// ... teszt validálás ..
```

Magyarázat

Minden pont fontos!

- A **SecurityService** Interfészként hivatkozik a **UserRepository** implementációra, és az implementációt is kívülről kapja meg (dependency injection) jelen példában konstruktor paraméterként.
- Minden korábbi problémától megszabadultunk, sehol nem szerepel a **SecurityService** osztályban egyik **IuserRepository** implementáció sem (próbáljuk rákeresni a fájlban a DbUserRepository és MemUserRepository szövegekre – nem lesz találat).
- A **SecurityService-et** így bármelyik **IuserRepository** implementációval is akarjuk használni, a **SecurityService** kódját nem kell módosítani (lásd **DemoSecurityService** osztály)

További DI gondolatok

- A DI-t már előző előadáson is ösztönösen használtuk, amikor a **DataProcessor** osztálynak a **ICompressionStrategy** és **ICancellationStrategy** implementációkat adtunk át!
- Általában a konstruktor paraméterként való megadást preferáljuk, de ha futás közben szükség van az implementáció megváltoztatására, akkor egy setter metódust használunk, pl. a **SecurityService**-ben.:

```
public void SetUserRepository(IUserRepository
    userRepository)
{
    this.userRepository = userRepository;
}
```

- A gyakorlatban sokszor a keretrendszer extra szolgáltatásokat (ún. DI containert) nyújt ahhoz, hogy a DI-t igazán kényelmesen tudjuk használni, pl. .NET, Java Spring. Ez a tárgy keretében nem szerepel.

További DI gondolatok

- Miért hívják függőséginjektálásnak?
 - > Az osztály a függőségeit (osztályok, melyekre épít, a példánkban a UserRepository) nem maga példányosítja a new operátorral, hanem ezek példányait kívülről kapja meg.
- Bizonyos esetekben a DI nem használható, mert a feladat olyan jellegű, hogy az osztálynak kell adott pontban – még ha közvetve is – példányosítani az objektumokat.
 - > Ez esetben a Factory Method és az Abstract Factory minták használhatók,
 - > Illetve a Prototype és a Builder minták, de ezeket nem tanuljuk

private konstruktor + static instance

Pl: Logger - fölösleges mindegyik osztályba példányosítani!

Singleton (Egyke)

Singleton

- Célja
 - > Biztosítja, hogy egy osztályból csak egy példányt lehessen létrehozni, és ehhez az egy példányhoz globális hozzáférést biztosít
- Globális hozzáférés
 - > Az egy példány kódban bárholnán kényelmesen elérhető, anélkül, hogy függvényparaméterben kellene átpasszolni
- Példa
 - > Pl. egy központi ablakkezelő (pl. WindowManager) vagy Application objektum
- Megoldás
 - > Singleton
 - Legyen az osztály felelőssége, hogy csak egy példányt lehessen belőle létrehozni
 - Maga az osztály biztosítson globális hozzáférést ehhez az egy példányhoz
 - > A programozási nyelvek segítségét igényli

Singleton C# példa

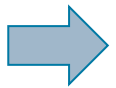
```
class WindowManager
{
    // Tárolja az egyetlen példányt, kezdetben null.
    private static WindowManager instance;

    // Globális hozzáférést biztosító statikus művelet
    public static WindowManager GetInstance()
    {
        // Az egy példányt az első hozzáférés alkalmával hozzuk létre
        if (instance == null)
            instance = new WindowManager();
        return instance;
    }

    // Védett konstruktor
    protected WindowManager() { }

    // Az osztály egyik művelete
    public void DoSomething() { /*...*/ }
}

...
// Valahol a kódban hozzáférünk hozzá a WindowManager singleton példányhoz
WindowManager.GetInstance().DoSomething();
```



Magyarázat

Singleton C# példa magyarázat

- Az egyetlen példányt maga az osztály tárolja egy statikus, pl. **instance** nevű védett (private) változóban.
- Az egyetlen példányt a többi osztály egy statikus, pl. **GetInstance** nevű metódussal éri el
 - > Ez az első hívás alkalmával hozza létre a példányt és el is tárolja a statikus instance változóban
 - > A későbbi hívások során ugyanezt a példányt adja vissza
- Az osztály konstruktora védett (private vagy protected), így más osztályok nem tudják a new operátorral példányosítani. Ez garantálja, hogy más osztály nem tudja megkerülni a **GetInstance** használatát, így további példányokat nem tud létrehozni.
- Mivel a **GetInstance** statikus, a kódban a **WindowManager.GetInstance()** hívással bárhol el tudjuk érni az egyetlen **WindowManager** objektumot.

Singleton

- C#-ban a hozzáférő statikus művelet helyett gyakori a statikus property használata (az alábbi példa csak a különbséget mutatja):

```
class WindowManager
{
    ...
    // Globális hozzáférést biztosító statikus művelet
    public static WindowManager Instance
    {
        get
        {
            // Az egy példányt az első hozzáférés alkalmával hozzuk létre
            if (instance == null)
                instance = new WindowManagerP();
            return instance;
        }
    }
    ...
}

...
// Valahol a kódban hozzáférünk hozzá a WindowManager singleton példányhoz
WindowManager.Instance.DoSomething();
```

Szálbiztos C# implementáció *

- **Nem kell tudni!**
- A korábbi megoldásunk nem volt szálbiztos, vagyis több szálú környezetben kis valószínűséggel, de előfordulhat, hogy több objektum jön létre
- A statikus tagváltozók inicializálása az osztály első használatakor történik
- A C# fordító olyan kódot generál az inicializáláshoz, ehhez, ami szálbiztos.

```
class WindowManager
{
    // Tárolja az egyetlen példányt
    // Itt inicializáljuk, ez szálbiztos
    private static WindowManager instance
        = new WindowManager();

    // Globális hozzáférést biztosító statikus property
    public static WindowManager Instance
    {
        get
        {
            return instance;
        }
    }
    // Védett konstruktor
    protected WindowManager() { }

    // Az osztály egyik művelete
    public void DoSomething() { /*...*/ }
}
```


Singleton

- Használatát nem szabad túlzásba vinni
- Sok esetben használata ellenjavallt (anti-pattern), mert a **GetInstance** mindig az osztály típusával tér vissza, nem tudunk egy alternatív „dummy”/fake/mock implementációval visszatérni, ami pedig a unit tesztelésnél sokszor alapelvárás.
 - > A Dependency Injection általában egy sokkal jobb tervezési minta!

Szoftvertchnológia és -technikák

8. Előadás – Benedek Zoltán
Tervezési minták 3



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták

- > Command
- > Command Processor
- > Memento
- > Adapter
- > Composite

Adapter (Illesztő)

(Wrapper)

Adapter

- Cél

➤ Egy osztály interfészét olyan interfésszé konvertálja, amilyent a kliens vár. Lehetővé teszi olyan osztályok együttműködését, melyek egyébként az inkompatibilis interfészeik miatt nem tudnának együttműködni.

- Alternatív név: Wrapper

- Elve



?



Nem megfelelő interfész



Megoldás: adapter

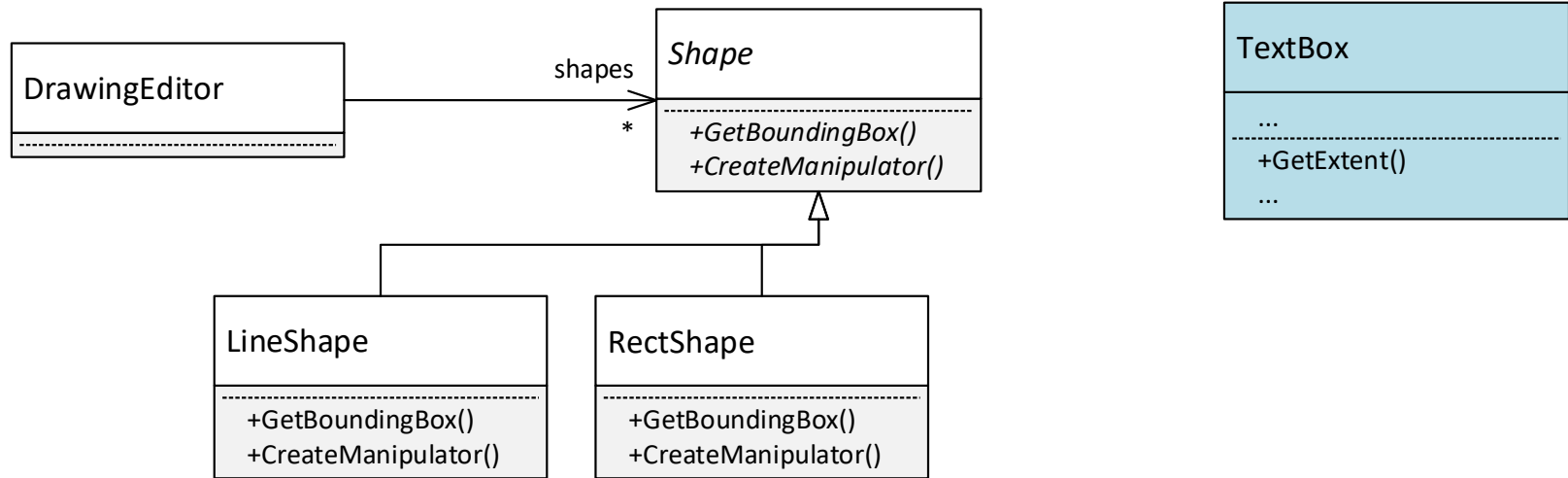
Adapter - példa

- A feladatunk egy vektorgrafikus alkalmazás elkészítése
 - > A felhasználó különböző grafikus alakzatokat tud elhelyezni a felületen. Támogatni kell a vonal, téglalap, stb. alakzatot, valamint a **szerkeszthető szövegdobozt** is (amibe lehet gépelni futás közben)
 - > Bevezetünk egy **Shape** **őosztályt**, hogy az **alakzatokat egységesen szeretnék kezelni** (esetünkben **egy heterogén kollekcióban tudjuk tárolni**). A grafikus alakzatokat ebből a Shape osztályból származtatjuk. Pl. LineShape, RectShape, EditableTextShape.
 - > **Probléma:** az EditableTextShape (egy szerkeszthető szövegdoboz teljes logikájának) megvalósítása nagyon-nagyon nehéz, beláthatatlan mennyiségű munka.

Adapter – példa folytatás

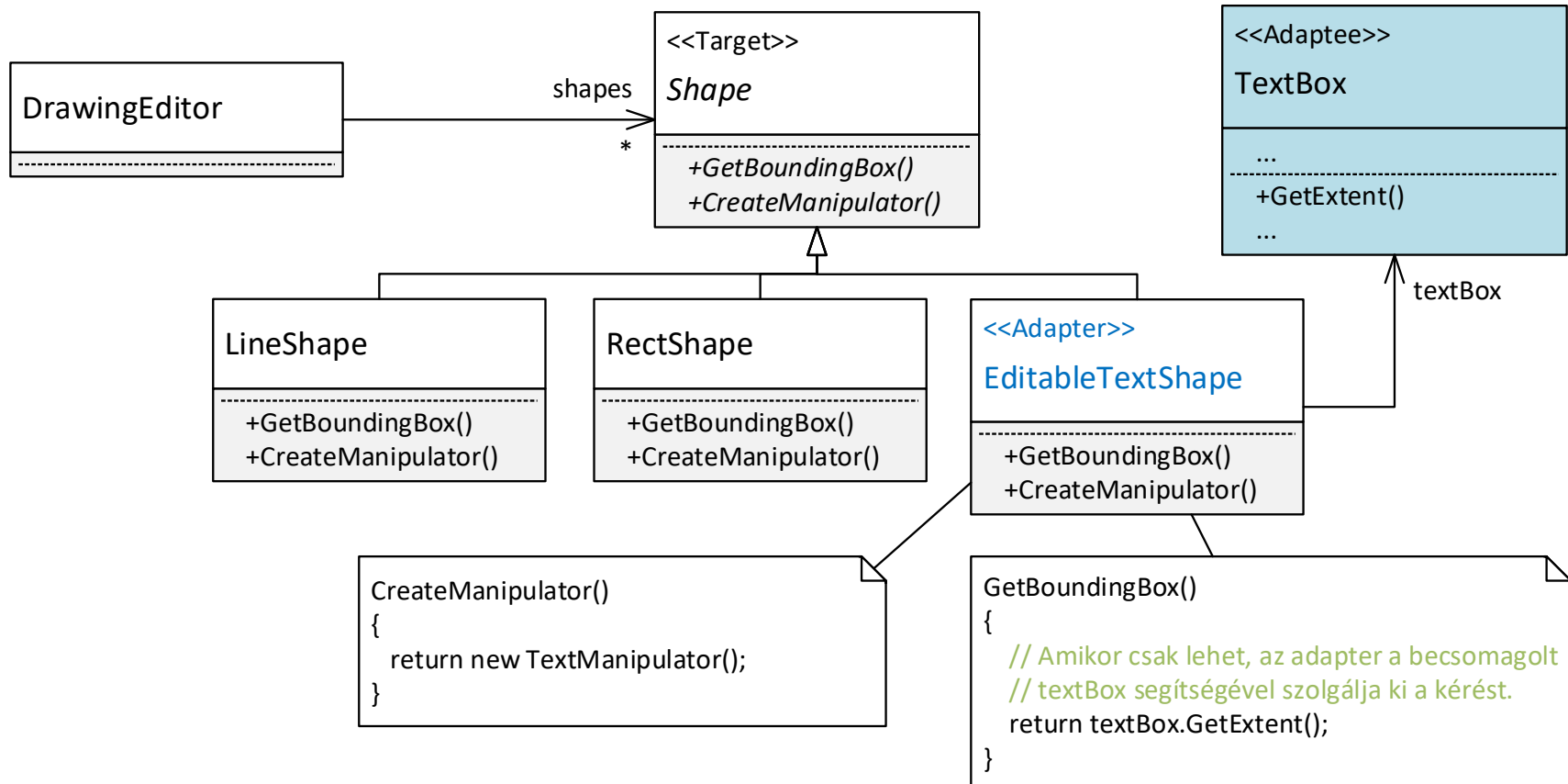
- > T.f.h. az alkalmazást egy adott keretrendszerre (pl. UWP) építve írjuk, amiben **már van egy beépített szerkeszthető szövegdoboz** (TextBox osztály), ami tudja mindazt, amit az EditableTextShape-től elvárunk. Jó lenne ezt felhasználni.
- > Probléma: a **beépített TextBox osztályt nem tudjuk közvetlenül felhasználni, mert nem megfelelő az interfésze**, ugyanis nem a Shape osztályból származik (emiattnem tudjuk a többi Shape-el együtt egységesen kezelni). Mivel egy „beépített” osztály, a forráskódját nem is tudjuk módosítani (hogya Shape osztályból származzon)
- > Probléma: nem tudjuk újrafelhasználni a meglévő osztályt!
- > Megoldás: Adapter minta használata (Object Adapter vagy Class Adapter)

Kiindulás

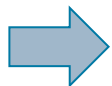


- A **DrawingEditor** egy **Shape** listát tárol (heterogén kollekció), a **TextBox** nem tehető bele
- A **GetBoundingBox** művelet egy befoglaló téglalapot ad vissza
- A **CreateManipulator** egy „manipulátor” objektumot, amivel az adott típusú alakzat szerkeszthető (pl. átméreteztető, stb., nincs jelentősége a példában)

Példa megoldás Object Adapterrel



Magyarázat



Példa megoldás Object Adapterrel

- Az eredeti tervünknek megfelelően leszarmaztatunk az `Shape` osztályból (vagy, ha a `Shape` interfész, implementáljuk azt) → Ez lesz az `EditableTextShape` osztály. Így az `EditableTextShape`-nek jó lesz az interfésze, a `DrawingEditor` tudja ezt is kezelni.
- Az `EditableTextShape` nem maga valósítja meg a műveletek többségét. Helyette:
 - > Példányosít és becsomagol egy `TextBox` osztályt
 - > A műveletek többségénél továbbhív a `TextBox` osztályba, delegálja a kéréseket. A példában a `GetBoundingBox` művelet a `TextBox` `GetExtent`-et hívja.
- Így a `EditableTextShape` fel tudja használni a már meglévő komplex logikát (`TextBox` osztály)
- Megjegyzés: a példában a `Shape` egy absztrakt osztály, lehetne interfész is, nincs jelentősége.

Példa megoldás Object Adapterrel

- Kód: lásd DesPattCode/Adapter mappa
- A legfontosabb rész maga az adapter

```
class EditableTextShape : Shape
{
    // A becsomagolt/adaptálandó osztály
    TextBox textBox;

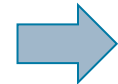
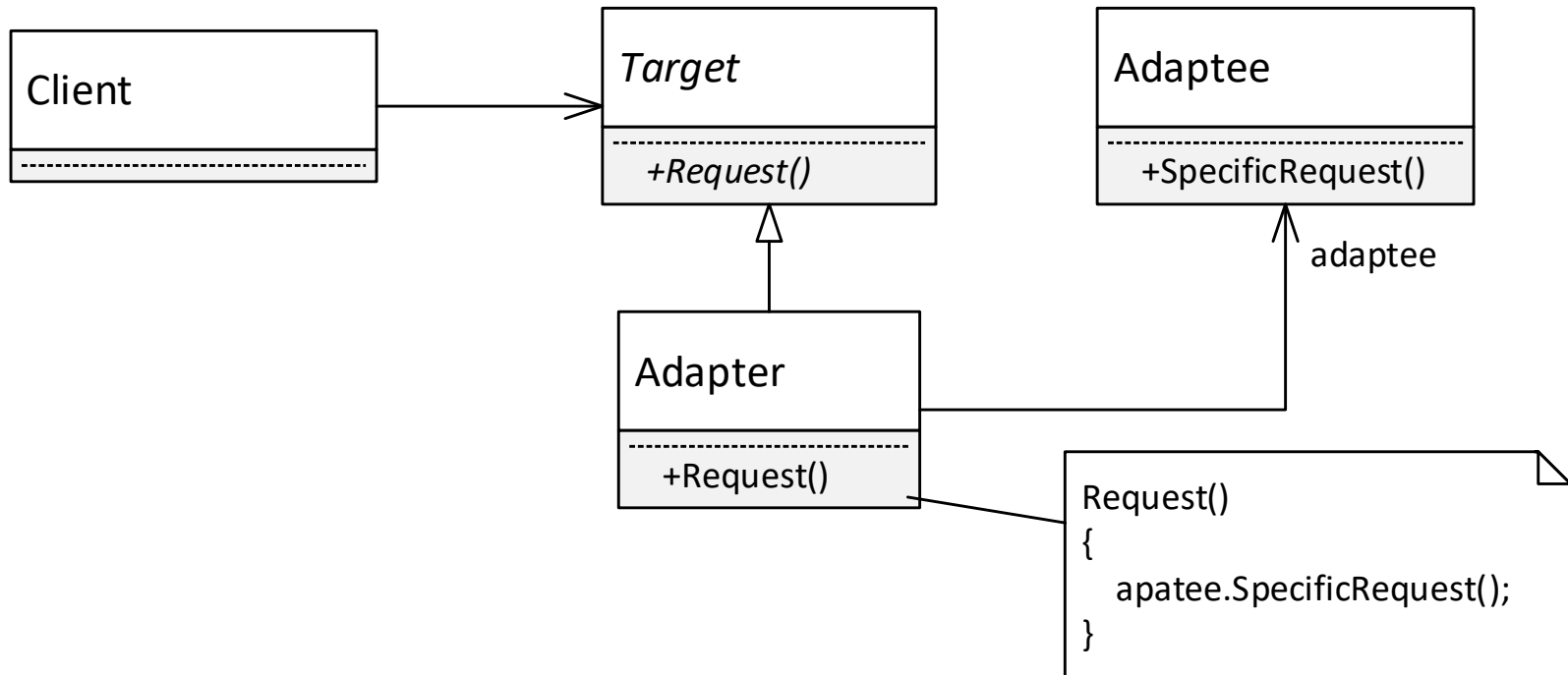
    // ...

    protected override Rect GetBoundingBox()
    {
        // Ez a lelke, ahol csak lehet, továbbítjuk a kérést
        // a becsomagolt/adaptálandó osztálynak, felhasználjuk
        // a kódját
        return textBox.GetExtent();
    }
}
```

Adapter

- Adapter minta általánosságában
 - Két változat
 - > Object Adapter
 - > Class Adapter
- Object Adapter → kompozícióval (has-a)
Class Adapter → örökléssel (is-a)

Object Adapter struktúra

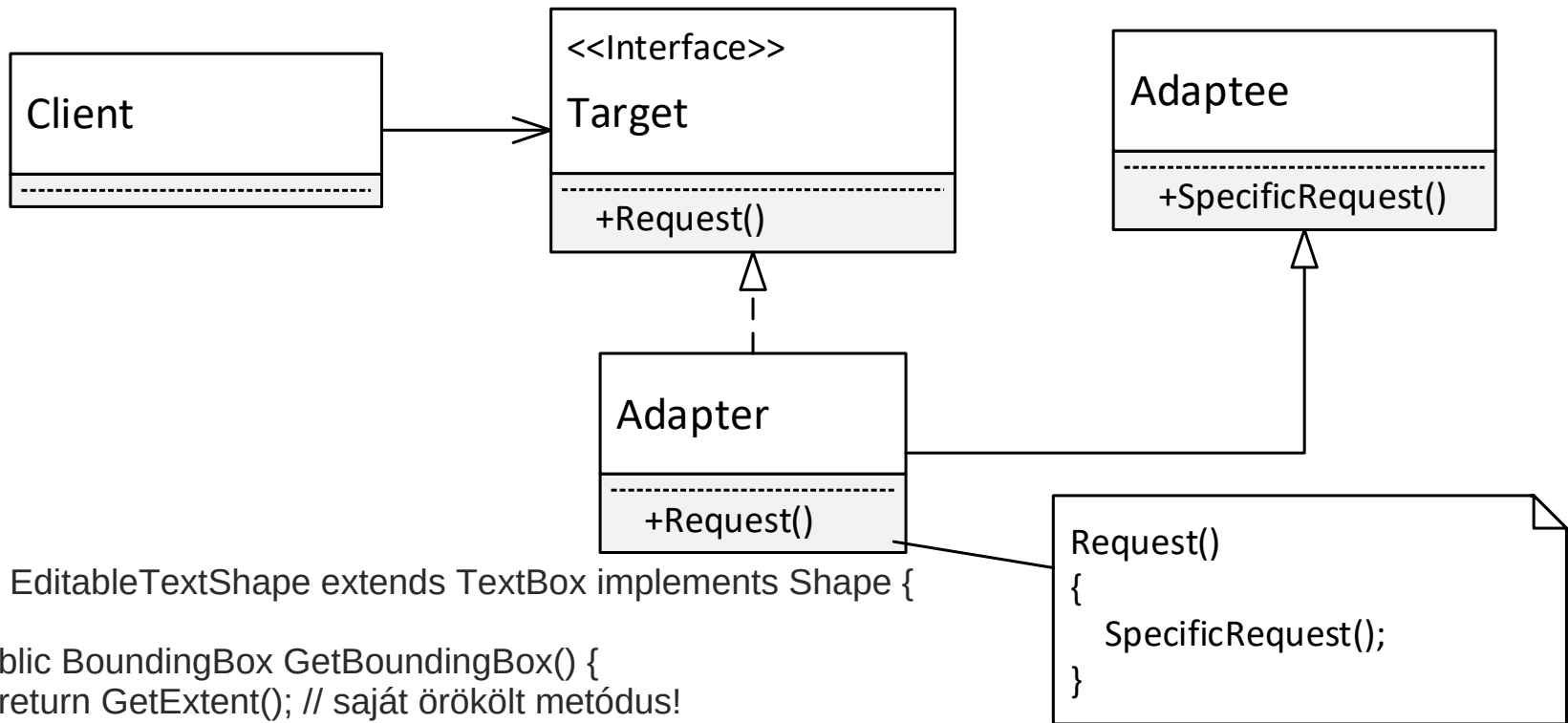


Magyarázat

Object Adapter magyarázat

- **Adaptee**: Az adaptálandó osztály, melynek nem megfelelő az interfésze (emiat nem tudjuk az adott környezetben triviális egyszerűséggel felhasználni).
- **Adapter** (*EditableTextShape*) : Illesztő, az Adaptee interfészt a Target interfésszé „konvertálja”. Tartalmaz egy hivatkozást egy adaptee objektumra (becsomagol egyet). A műveletei, ahol csak tehetik, a becsomagolt adaptee műveleteit hívják.
- **Target** (*Shape*): Interfész, amit a kliens használ. A gyakorlatban lehet interfész vagy absztrakt osztály is.

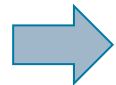
Class adapter struktúra



class EditableTextShape extends TextBox implements Shape {

```
    public BoundingBox GetBoundingBox() {
        return GetExtent(); // saját örökölt metódus!
    }
}
```

Alapja: tartalmazás (becsomagolás)
helyett leszármaztatás.



Magyarázat

Class Adapter magyarázat

- A szerepek alapvetően megegyeznek az Object Adapterével, ami különbség:
 - > Az Adapter nem becsomagolással használja fel az Adaptee kódját, hanem leszármaztatással. Az Adapter műveletei, ahol csak tehetik, az ősz Adaptee műveleteit hívják.
 - > Mivel a legtöbb nyelv (pl. Java, C#) nem támogatja a többszörös öröklést, a Target itt csak interfész lehet, őszosztály nem!
 - > Kevésbé rugalmas, mint az Object Adapter, általában az Object Adapter változatot preferáljuk!

Adapter összefoglaló

- Használjuk: ha nem tudunk valamilyen osztályt újrafelhasználni egy környezetben, mert nem jó az interfésze
- Megjegyzések
 - > (Az Object Adapter esetében nem minden esetben szükséges tagváltozóként tárolni az Adaptee-t, ritkább esetben az Adapter művelete maga példányosítja majd el is dobja az Adaptee objektumot. Ekkor nem asszociáció, hanem csak függőség van az Adapter és az Adaptee között.)

Composite (Összetett)

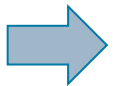
Composite

- Célja

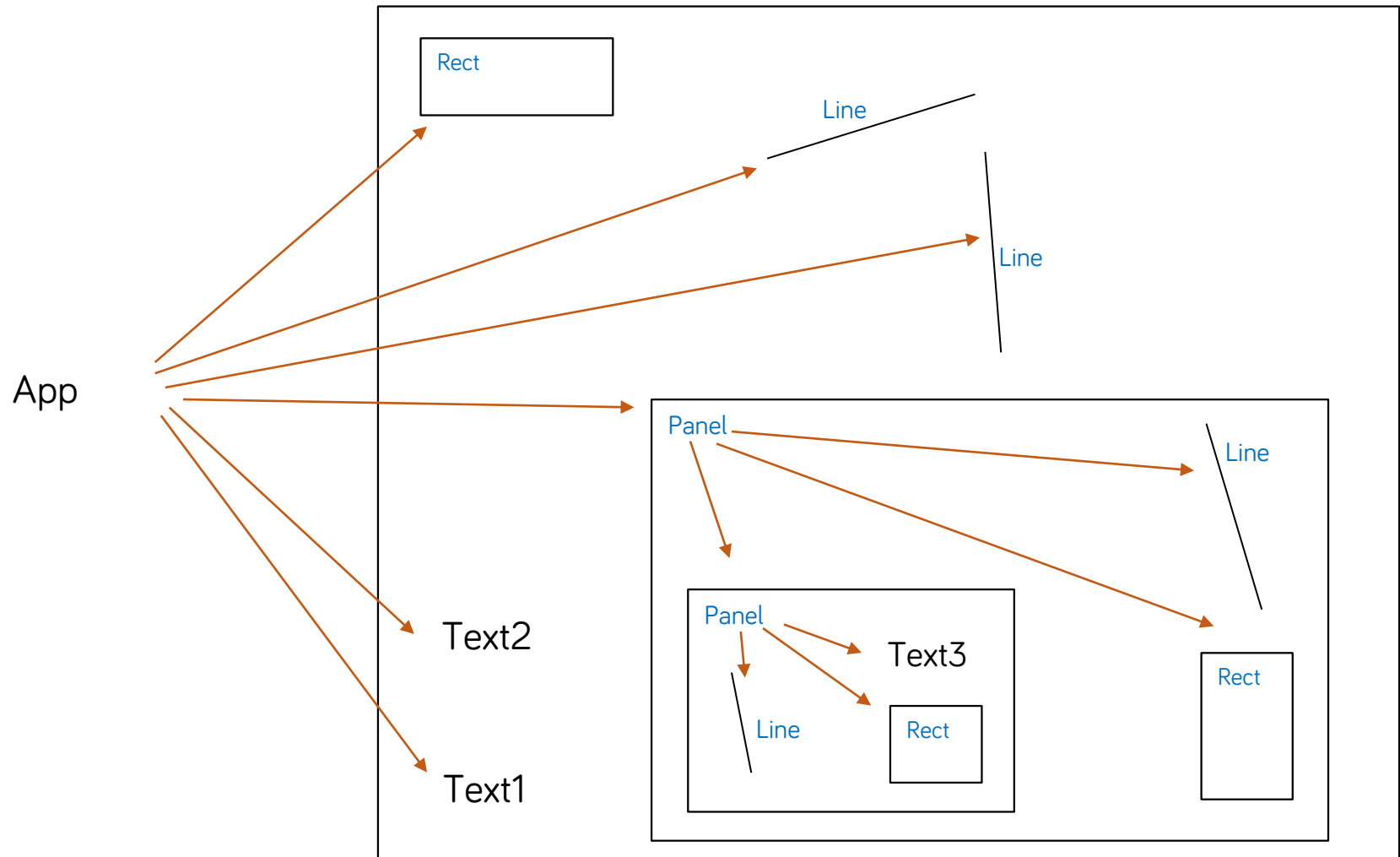
- > Rész-egész viszonyban álló objektumokat fastruktúrába rendezi
- > A kliensek számára lehetővé teszi, hogy az egyszerű és összetett (kompozit) objektumokat egységesen kezelje

- Példa

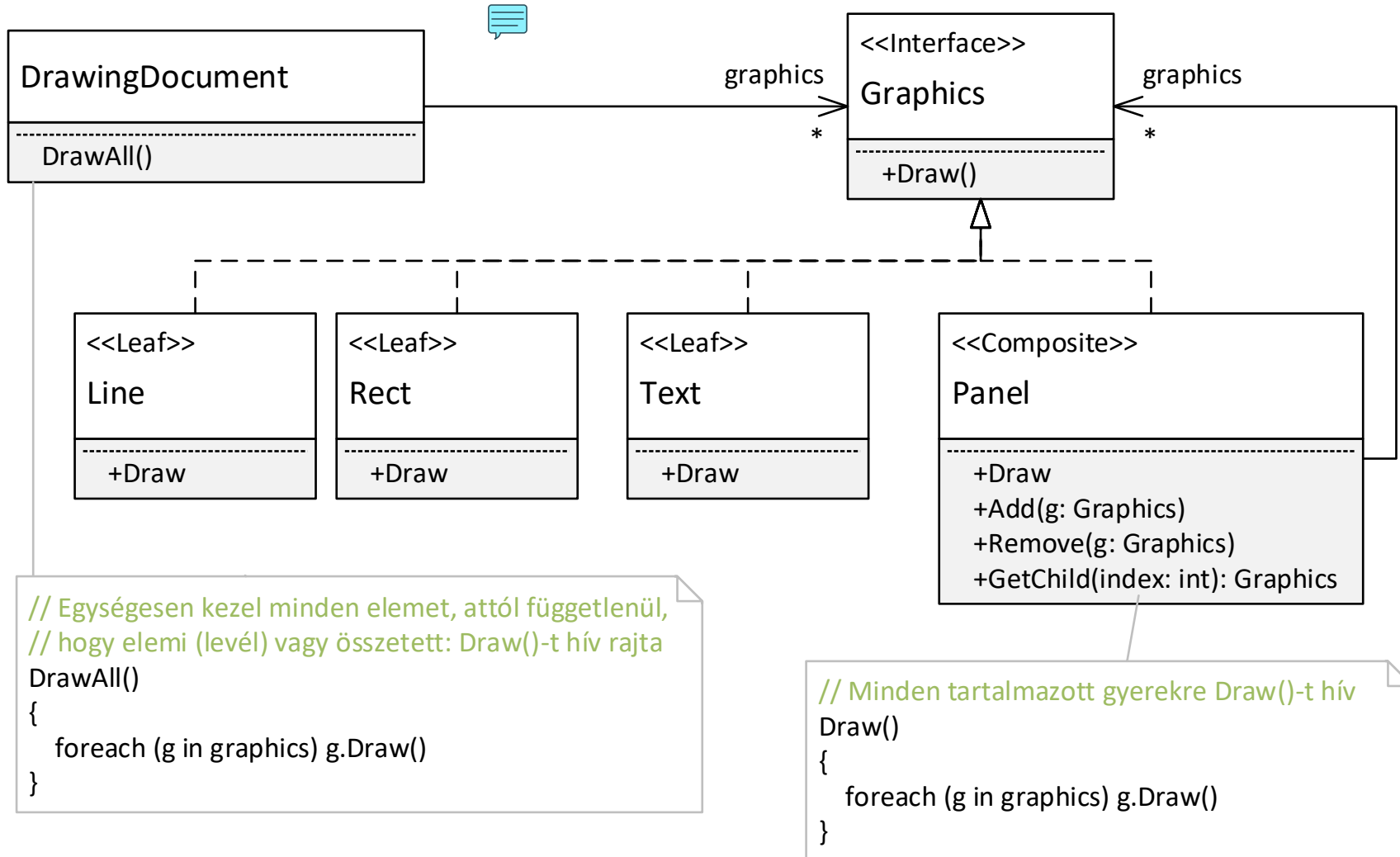
- > Olyan grafikus alkalmazás, amely lehetővé teszi elemi és összetett grafikus objektumokat tartalmazó rajzok létrehozását
- > A Panel egy olyan összetett grafikus elem, mely grafikus alakzatokat tartalmaz (tetszőlegesenek, akár más Panel objektumok is lehetnek rajta!)



Composite példa



Composite példa



Magyarázat



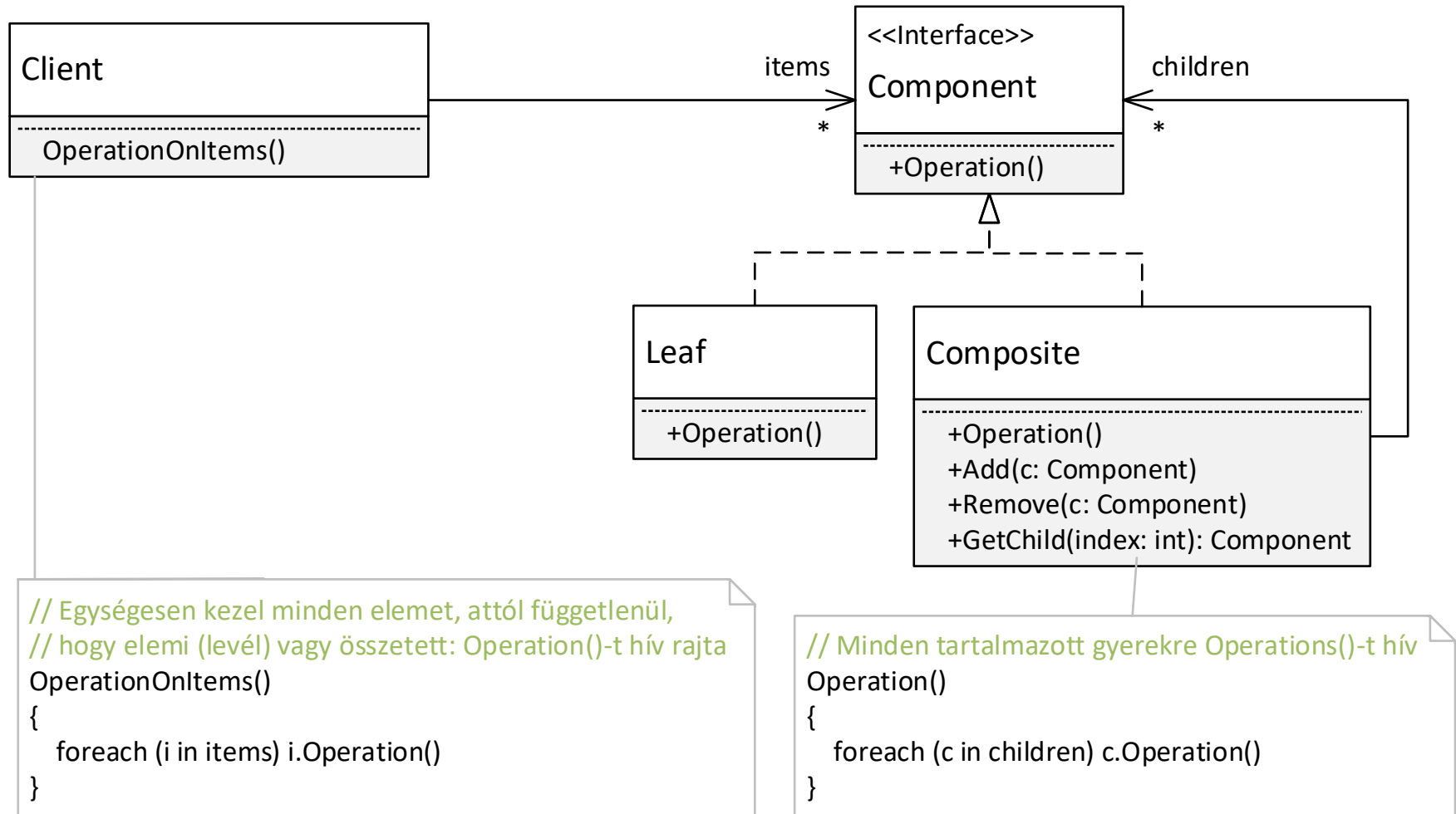
Composite példa magyarázat

- **DrawingDocument:** egy olyan dokumentumot reprezentál, mely különböző grafikus alakzatokat tárol és kezel
- Ezen grafikus alakzatok lehetnek
 - > **Elemiek** (levél, leaf), mint pl. Line, Rect, Text.
 - > **Összetettek**, mint pl. a Panel (mely maga is tartalmazhat elemi és összetett alakzatokat).
- **Graphics:** a grafikus alakzatok közös interfésze/őse, attól függetlenül, hogy elemi vagy összetett az alakzat

Composite példa magyarázat

- A Composite minta egyik alapelve, hogy **egységesen kezeli az elemi (Line, Rect, Text) és az összetett (Panel) objektumokat**. Ez a példában két pontban is megjelenik:
 - > A DrawingDocument **közös gyűjteményben tárolja az elemi és összetett objektumokat** (ez a graphics tag). Megteheti mert az összetett Panel is implementálja a Graphic interfészt.
 - > A DrawAll műveletben a kirajzoló kód **nem különböztetni meg – vagyis egységesen kezeli - az elemi és összetett objektumokat** (nincsenek típus szerint leválogatva): mindre egységesen a Draw() műveletet hívja.
- A összetett Panel Draw művelete az általa tartalmazott alakzatokat rajzolja ki, pont ez volt a célunk!
- Kód: lásd DesPattCode/Composite mappa

Composite általánosságában



Composite

- Használjuk, ha
 - > Objektumok rész-egész viszonyát szeretnénk kezelni (jellemzően fastruktúrában)
 - > A kliensek számára el akarjuk rejtetni, hogy egy objektum elemi objektum vagy kompozit objektum: bizonyos műveletek szempontjából egységesen szeretnénk kezelni őket

Composite megjegyzések

- A gyerekek kezelése nem tud egységes lenni!
 - > Hiszen az csak az összetett (példánkban a Panel) osztály objektumaira értelmezett (Add, Remove, GetChild)
 - > A kliens pl. típusellenőrzéssel tudja megnézni, hogy az adott elem összetett-e (pl. C#-ban az „is”, Java-ban az „instanceof” operátorral), pl.:
 - if (item is Composite) ...
 - > Az Add, Remove, GetChild műveletek hívásához a kliensnek a kompozit osztályra/interfészre kell castolnia a hivatkozását

Tervezési minták összefoglaló

További tervezési minták

- A klasszikus „GoF” minták közül is kimaradt pár:
 - > Prototype, Builder, Bridge, Mediator, Chain of Responsibility, Visitor, Decorator, Iterator, State, stb.
- Számos nem általános tervezési minta létezik, pl.
 - > Elosztott, konkurens rendszerekre jellemző minták
 - Szolgáltatás hozzáférés
 - Konfiguráció
 - Esemény kezelés
 - Szinkronizáció
 - Konkurencia
 - > Valós idejű rendszerekre jellemző minták
 - > Vállalati rendszerekre jellemző minták

Összefoglalás

- Tapasztalati tudást hordoznak
 - > Mi is rá tudunk jönni, de
 - Jó sokáig tart
 - Nem jövünk rá
 - Miért ne tanuljunk mások tapasztalataiból
 - > Értékes tudás!
- Célunk
 - > Ismerjünk meg minél több mintát
 - > Hosszútávon legalább arra emlékezzünk, hogy egy adott problémakörben mely mintákat lehet jól használni

Szoftvertchnológia és - technikák

11. Előadás - A vízesés és a RUP módszertan



Szoftverfejlesztési módszerek

- Egy mérnöki diszciplína, amely fókuszában a szoftver-alapú rendszerek fejlesztésének aspektusai szerepelnek.



Szoftverfejlesztési módszerek

- Vízesés
- Spirális
- Iteratív és inkrementális
- Agilis



A szerződés: megállapodás

- Mit adok – mit kapok
 - > Keretek: idő, minőség, ...

Két alap modell

1. Projekt alapú
2. Költség alapú

1. Projekt alapú

- A szerződésben rögzített témák
 - > „specifikáció”, feature set
 - > Ellenérték
 - > Egyéb paraméterek...
- Kinél van a kockázat?
 - > A fejlesztőnél:
 - > Ha a projekt mérföldköveinél szállítja ami benne van a szerződésben, kifizetik, de addig ...
- Van-e mozgástere a megrendelőnek?
 - > Nincs: ha a végén elkészül amit kért, ki kell fizetni



2. Költség alapú

- A szerződésben rögzített témák
 - > Fejlesztői/... erőforrások időarányos ára
 - > Minőségi és egyéb paraméterek
- Kinél van a kockázat?
 - > A megrendelőnél
 - > Bármilyen is lesz a projektből, a ráfordított időt ki kell fizetnie
- Van-e mozgásteret a megrendelőnek?
 - > Igen: bármit kérhet bármikor, úgyis kifizeti



Mi az alapvető probléma?

- Minden szervezet stratégiai célokban és költség keretekben gondolkodik
- Stratégia: jól definiált jövőbeni célok
- A pontos részletekre bontás nehéz
- Az elkészült projekt nem mindig juttatja el a céget a kitűzött stratégiai célhoz
 - Az igazi probléma gyakran a megrendelő oldali folyamatokban, struktúrában stb. rejlik
- **Az informatika kiszolgáló terület!**
Alkalmazkodnunk kell!



Egy projekt „megnyerése”

- Szabályozott környezet
 - > Például előre rögzített pontszámítási módszer, ...
- Szabad környezet
 - > Gyakran kevésbé publikált elbírálási módszer
- Mindkét esetben nagy tere van a lobbinak



Leggyakoribb kiválasztási módszer

- Projekt alapú
 - Fix költségvetés
 - Fix feature set
 - Ár alapú döntés
- Kockázat a fejlesztőnél



A szabályrendszerekről

- Minden szabályrendszer motiválja a résztvevőket valamilyen irányba
 - > Lehet a szabályalkotó tudatos választása
 - > Váratlan „mellékhatások”
- Ezekkel érdemes tisztában lenni
 - > Kihasztnálni
 - > A veszélyeket időben felismerni

Mire motivál a projekt alapú szerződés?

- Fejlesztők

- Feature freeze: semmi változtatás az eredeti specifikáción, merev ellenállás, „bármilyen áron”
 - Vagy olyan változás, amit a megrendelő kifizet...

- Megrendelők

- Minél több/gazdagabb funkció fejlesztése
- Előzetes felmérés: minden belekerüljön amire szükség van/lehet
 - Költség tervezés, megtérülés vizsgálat, ...



A projekt alapú megközelítés veszélyei

- Az előzetes specifikáció (ajánlati kiírás) sosem lehet teljes
 - > Viszont ennek alapján készül az ajánlat
- A későbbi változtatások nehézkesek
- Okai:
 - > A megrendelő nem tudja, nem veszi a fáradságot
 - > Nem az írja a specet mint aki használni fogja
 - > Változó gazdasági, technológiai körülmények
 - > Az ördög mindig a részletekben rejlik

A kockázatok mérséklése

- Ajánlatba beépített puffer a fejlesztői oldalon
 - > Verseny környezetben kockázatos
- Rugalmasság, mindkét oldalon
 - > Barter megállapodások: nem módosul a teljes projekt összeg
- Lépcsőzetes kidolgozás és ajánlatok
 - > Részletes, kifizetett tervek
 - > Hátránya: lassú átfutás, sok döntési pont

Mire motiválhat a költség alapú szerződés?

- Fejlesztők

- > Hosszútávú projekt

- > A megrendelő folyamatosan kapjon értéket

- > Minél több munka, változtatások

- Megrendelők

- > Alul specifikálás: nem kell előre specifikálni, hiszen közben bármikor módosítható a funkció halmaz

A költség alapú megközelítés veszélyei

- Előzetes specifikáció/végig gondolás hiányában rengeteg, nagy költségű változtatás
 - > A változtatások gyakran architekturálisak
- Nem készül el a kívánt alap funkcionalitás sem
 - > Folyamatos változtatások
- A költségek irreálisan nagyokká válnak
 - > Különösen összevetve az elkészült terméket a rá költött összeggel

A kockázatok mérséklése

- Minden agilis módszertan nagy hangsúlyt helyez a tervezésre, közös vízióra!
 - > A termék elhelyezése a megrendelő stratégiai koncepciójában, életében
- Előzetes (nem kötelező érvényű) költség becslések

A rugalmasságról - kérdések

- „Miért nem úgy van elkészítve az alkalmazás, hogy ezt egy nap legyen beletenni?”
- „Ez miért nem paraméterezhető, miért van beledrótozva?”
- „Igaz, hogy nem kértük az ajánlatban kifejezetten, de minden minőségi informatikai projektben ezt paraméterezhetően csinálnák meg...”
- „Ez miért nem egy adatbázis mező amit mi tudunk módosítani, miért kell ehhez fejlesztés?”

A rugalmasságról - válaszok

- A rugalmasság sosincs ingyen
 - Ami igen, azt tessék belefejleszteni! 😊
- Előre nem tudni, hol lesz rá szükség!
- Az alkalmazást minden irányú bővítésre felkészíteni tipikus hiba
 - Költségek: fejlesztés és karbantartás
 - Overarchitecting, KISS/Yagni elveknek ellentmond stb.
 - Könnyen inkonzisztenciát okozhat
- Ezért nem feltétlenül hiba, ha valamit nem lehet paraméterezni pedig meg lehetett volna úgy is írni

Projekt alapú megközelítés elvárásai

- Fel lehessen használni a készen lévő „specifikációt”
- Ellenőrizhető, követhető dokumentáció
- Lehessen tervezni
 - > Tervező eszközökkel megközelíthető architektúra
 - > Esetleg kódgenerálás
 - > A terveket az implementációs fázisban ideálisan tudjuk felhasználni, ne legyen kétszeri munka
- Stabil technológiák, eszközök használata
 - > Tartani kell a határidőket

Költség alapú megközelítés elvárásai

- „Változtatás tűrés”
- Refaktorálás regresszió nélkül
- A kódban történt változtatások gyors ellenőrzése
- Jól unit tesztelhető alkalmazás
 - > A jó unit tesztek jellemzői
- Rugalmas, bővíthető architektúra

Az üzleti modell és a módszertanok

- Az üzleti modell / szerződés típus erősen befolyásolja az alkalmazott módszertant
 - > Fix költség – vízesés
 - > Költség alapú – evolúciós
- De vannak egyéb – üzleti – tényezők
 - > Például: a szállító hosszú távon tartja karban az alkalmazást, jó minőségű, olcsó javításokat, fejlesztéseket szeretne adni hosszú távon -> evolúciós módszertant választ az esetlegesen nagyobb kezdeti költségek ellenére

Összefoglalás

- Minden fejlesztési modellnek megvannak a maga erősségei és gyengeségei
 - > A kiválasztás soktényezős
- Néhány évente újabb és újabb fejlesztési módszerek jelennek meg
 - > Érdeemes követni az új megoldások fejlődését
- Mindig legyünk tisztában a környezettel amiben dolgozunk!

Vízesés modell



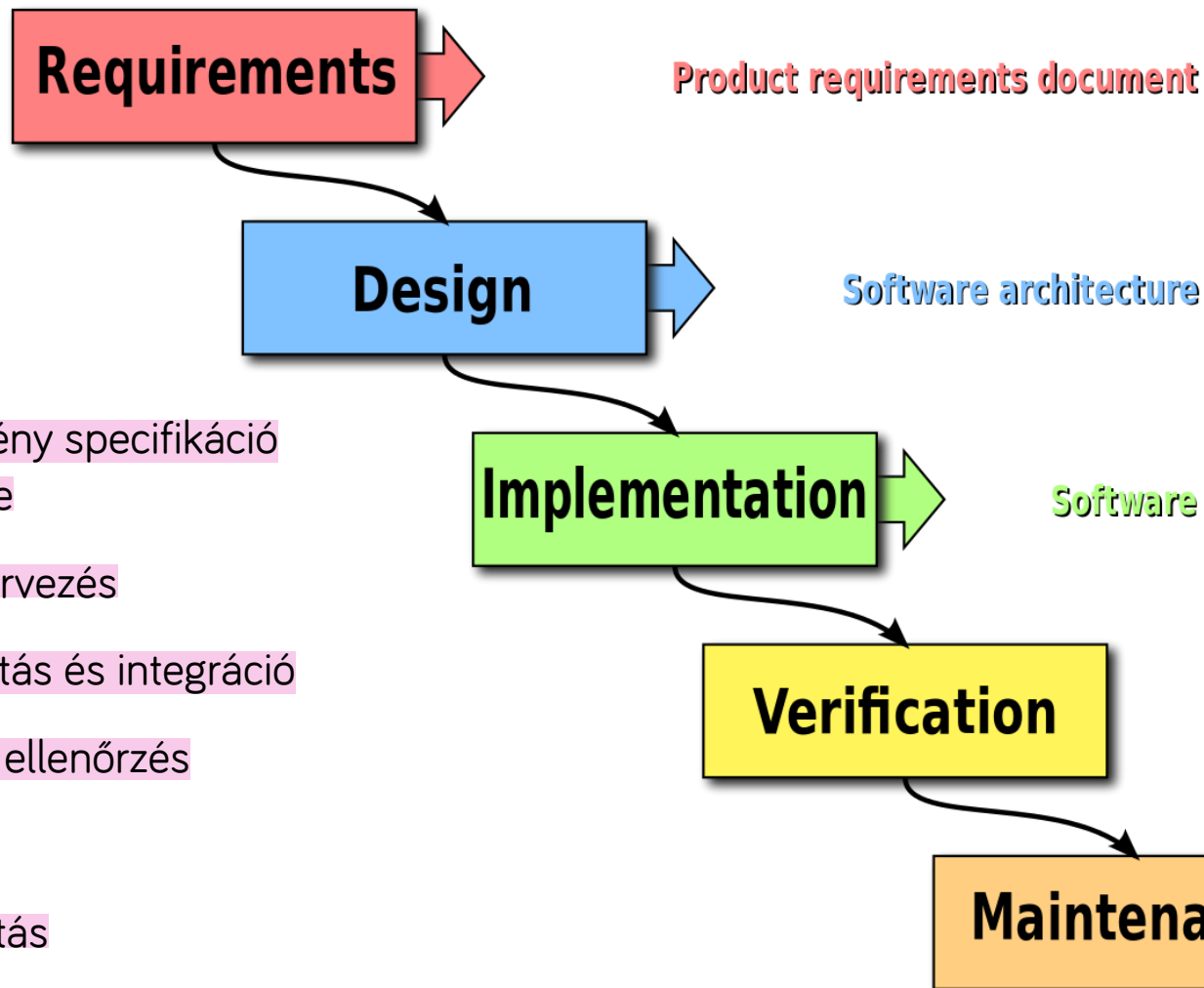
Vízesés modell

- **Példa:** A BKV megbízza a szoftverfejlesztő csapatunkat egy elektronikus jegyrendszer fejlesztésével.
- Jól rögzített határideje van:
 - az átadást 2022-re tűzik ki.
- Alphától omegáig minden ránk van bízva:
 - Mindent nekünk kell kitalálni, megvalósítani, telepíteni, átadni, karbantartani.
- A megrendelővel a projekt elején és végén találkozunk.

Vízesés modell

- Az elmúlt 50 év leggyakrabban használt modellje
- Ma is nagyon gyakori, főleg hazánkban
- Lineáris modell
 - > Egymást követik a lépések
 - > Akkor kezdünk el egy új fázist, ha az előző befejeződött

Vízesés modell lépései



1. Követelmény specifikáció elkészítése
2. Szoftvertervezés
3. Megvalósítás és integráció
4. Tesztelés, ellenőrzés
5. Telepítés
6. Karbantartás

1. lépés: Követelmények

- Részletes, közös képet alakít ki a Megrendelő és a Szállító között az elkészítendő termékről
- Akkor lépünk tovább, amikor elfogadásra került a specifikáció



1. lépés: Követelmények

- Egy specifikáció elemei:
 - > A projekt célja, helye a vállalatnál
 - > Terminológia, definíciók
 - > Felelősök
 - > Felhasználói csoportok
 - > Feltételezések, kényszerek, elő-követelmények, függőségek
 - > Funkcionális követelmények – Folyamatok, interfészek stb. tételes felsorolása és részletezése
 - > Nem funkcionális követelmények – Teljesítmény, biztonság, bővíthetőség, formátum, ...
 - > Becsült határidők
 - > Átadás / átvételi tesztek

2. lépés: Tervezés

- Konceptcionális tervek:
 - > Architektúra, felhasznált keretrendszer
 - > Interfészek, modul határok
 - > Részletes adatbázisterv
 - > Folyamatok leírása
 - > Képernyőtervek, navigációs térkép, stb.
 - > Használati minták
 - > Biztonsági, tesztelési tervek
- Prototipizálás:
 - > Iterációk a felhasználói visszajelzések alapján

2. lépés: Tervezés

- A tervezés lépése után a további módosítások már költségesek!
- A vízesés modell alapja a jó terv.
 - > Megrendelő által validálható tervek – például képernyő képek, folyamat ábrák!
 - > Megrendelői oldalon valódi ellenőrzés a munkát végzők által (kulcsfelhasználók)!
 - > Nehézséget és kockázatot jelent, ha a megrendelő közben egy transzformációban van benne.

3. lépés: Megvalósítás és Integráció

- Ebben a lépésben indul el a fejlesztés.
- Munka dekomponálása, fejlesztési tervek
- Többnyire a leghosszabb idő a teljes folyamatban

4. lépés: Tesztelés

- A minőség ellenőrzők megvizsgálják a szoftvert, hogy a specifikáció szerint működik-e?
- A hibák visszajutnak a fejlesztőkhöz és a tesztelés előlről indul
- Gyakran használunk átmeneti állapotot: béta verzió, meghívott felhasználók stb.

5. lépés: Telepítés

- Telepítés, terjesztés
- Pontosán ugyanaz a verzió menjen ki, mint amit a tesztelők jóváhagytak!

6. lépés: Karbantartás

- Karbantartási / továbbfejlesztési fázis
 - > Hibajavítás, tesztelés, ...
- A karbantartás a legdrágább fázis
 - > A hibajavítás kiemelten költséges lehet

A vízesés modell előnyei

- A megrendelők és a fejlesztők hamar (a projekt elején) megállapodnak, tisztázzák a fejlesztés fázisait.
- A projekt fejlődése könnyen mérhető és nyomonkövethető
 - > Melyik fázisban vagyunk éppen
 - > Tudjuk (jó becslésünk van arról, hogy mennyi van hátra

A vízesés modell előnyei

- Nem kell mindenkinek folyamatosan a projekten dolgoznia
 - > Pl. A tesztelőknek a tesztelés fázisában kell dolgozniuk, előbb nem
- A megrendelő (folyamatos) jelenléte nem szükséges
- Ha a tervezés jó, akkor előre becsülhető, mikorra lesz kész a fejlesztés és melyik fázis mennyi időt vesz igénybe

A vízesés modell kritikája

- Nem az készül el, amire szükség van végül
 - > Gyakran nem kapunk visszajelzéseket a termékről addig, amíg az el nem készült
- A végül megjelenő új követelmények koncepcionális változásokat indukálhatnak, ami újratervezést igényelhet
 - > Jelentőssé válhat a kidobott idő, energia
- A tervezők gyakran nincsenek tisztában az implementációs részletekkel
 - > Néha egyszerű tervváltoztatás helyett bonyolult implementáció készül el

A vízesés modell kritikája

- A tervező eszközök nem integrálódnak jól az implementációt segítő eszközökkel
- Túl hosszú lehet a teljes fejlesztési fázis, nincs részfelhasználás

Mikor használjuk?

- Fix projekt alapú a megállapodásunk
- A megrendelő csak a mérföldköveknél van jelen
- Pontos határidővel dolgozunk
 - > Például törvényi előírások
- Nem változnak a követelmények útközben

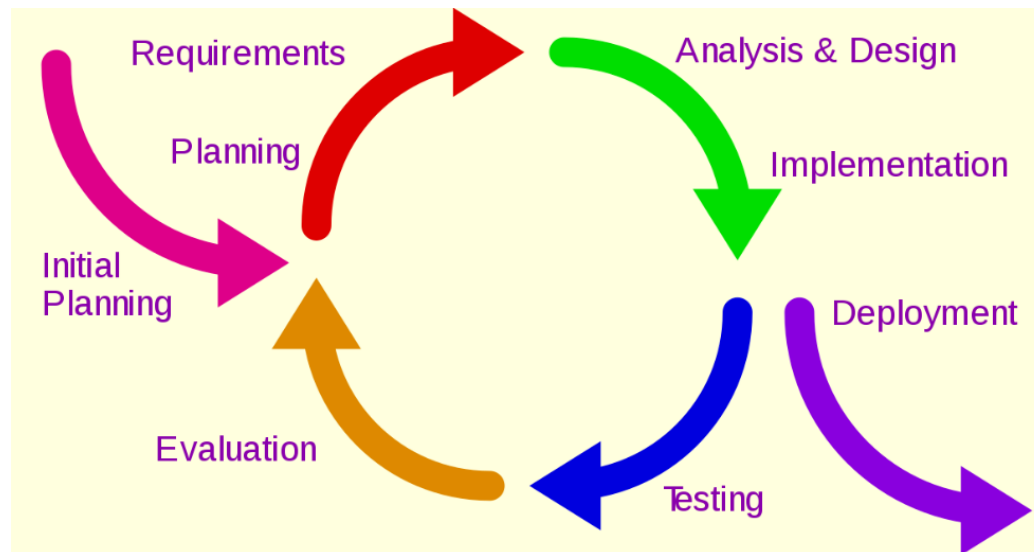
Esettanulmány (olvasmány)

- <https://medium.com/@kawomersley/would-you-rather-be-late-or-wrong-a-strategic-case-for-waterfall-development-35d81911022c>



Iteratív és inkrementális

- Részegységekre osztjuk a feladatokat
- A rendszer ismétlődő ciklusok (iteratív) és egy időben kisebb részek kidolgozásával (inkrementálisan) fejlődik
 - > Minden iterációban frissül a terv és új funkciók kerülnek kidolgozásra
 - > Folyamatos finomítás alatt van a fejlesztés, egymással párhuzamosan is futnak a lépések



Szoftvertchnológia és -technikák

12. Előadás - Agilis fejlesztés, Scrum



Mik a kihívások a fejlesztésben?

- Követelmények megértése
- Követelmények változása
- Pontatlan becslések / keretek tartása
- Adminisztrációs/kommunikációs overhead
- A vízésés modell nem kezeli jól ezeket a kihívásokat, ezért születtek meg az agilis módszertanok



Agilis módszertanok

- Eleve figyelembe veszik a fenti nehézségeket
- Iteratívan, mindig a prioritásokra fókuszálunk
- Transzparencia: mindig legyen látható, hol tartunk
- Vizsgálódás: rendszeresen vizsgáljuk meg, hogy jó-e az irány
- Adaptálás: szükség esetén változtassunk



Az Agile Manifesto

- Az agilis mozgalom fő dokumentuma
- 2001-ben írta le több neves szoftverguru
- A fontosabbak:
 - > Kent Beck
 - > Alistair Cockburn
 - > Martin Fowler
 - > Robert C. Martin
 - > Ken Schwaber és Jeff Sutherland
- Értékeket és 12 irányelvet rögzítettek.
- <https://agilemanifesto.org/>

Agilis kiáltvány

- 2001. február 11-13. The Lodge at the Snowbird ski resort
- Célravezetőbb alternatívák a szoftverfejlesztés támogatására
- **Értékek:** !!!!!
 - > Egyének és interakció inkább, mint folyamatok és eszközök
 - > Működő szoftver inkább, mint jól dokumentált szoftver
 - > Felhasználó bevonása inkább, mint szerződések tárgyalása
 - > Reagálás a változásokra inkább, mint egy terv követése



Az agilis fejlesztés 12 pontja

1. A legfontosabb dolog a megrendelő kielégítése úgy, hogy a kezdetektől folyamatosan értékes szoftvert szolgáltatunk.
2. Üdvözljük a változtatási kérelmeket, még a fejlesztés késői szakaszában is. Az agilis folyamatok a megrendelő versenyelőnye érdekében hasznosítják a változtatásokat.
3. Működő szoftver gyakori kiadása (pár hét – pár hónap) minél rövidebb időszakokkal.
4. Az üzleti megrendelőnek és a fejlesztőknek napi szinten együtt kell működniük.



Az agilis fejlesztés 12 pontja

5. A projekteket jól motivált egyénekre kell építeni. Adjuk meg nekik a környezetet és a támogatást, és bízunk bennük, hogy a rájuk bízott feladatot elvégzik.
6. A leghatékonyabb és leghatásosabb módja az információáramlásnak a csapaton belül és azon kívül a személyes beszélgetés.
7. A haladás mértéke a működő szoftver.
8. Az agilis folyamatok a fenntartható fejlesztést támogatják. A megrendelők, a fejlesztők és a végfelhasználók egy állandó sebességet tudnak fenntartani gyakorlatilag végtelen ideig.



Az agilis fejlesztés 12 pontja

9. A technikai tökéletességre és a jó dizájnra való törekvés növeli az agilitást.
10. Az egyszerűség – az el nem végzett munka maximalizálása – alapkövetelmény.
11. A legjobb architektúrát, követelményt és dizájnt önszerveződő csoportok tudják kitermelni.
12. Bizonyos időközönként a csapat megvitatja, hogyan lehetne hatékonyabb és ennek megfelelően hangolja a viselkedését.



Értékek - Kommunikáció

- „What matters most in team software development is communication”
- Minden probléma visszavezethető a kommunikációra
 - > Programozó – Programozó
 - > Programozó – Ügyfél
 - > Manager – Programozó
- Egy cél: minden fejlesztő ugyanúgy lássa a rendszert, ahogy a majdani felhasználók is látni fogják.



Értékek - Egyszerűség

- „What is simplest thing that could work?”
- Az agilis módszer arra fogad, hogy...
- YAGNI
- Az egyszerűség segíti a kommunikációt

Értékek – Visszacsatolás

- „Az optimizmus szakmai ártalom a programozóknál, és a visszajelzés rá a gyógyír.”
- „Elsőre jól?”
 - > Nem tudjuk hogyan
 - > Ha tudjuk, holnap már lehet a jó rossz
 - > Sok és gyors visszacsatolással tudunk közel kerülni a jó megoldáshoz.



Értékek – Bátorság

- Tenni valamit a félelem ellenére
 - > Betartani a YAGNI-t
 - > Refaktorálni
 - > Revertálni
- Vagy nem tenni
- Főleg a többi értéket támogatja
 - > Kimondani jót és rosszat
 - > Eldobni a rossz megoldásokat
 - > Igazi konkrét válaszokat keresni

Értékek – Respect

- Ha nem törődünk egymással, akkor...
- Ha nem törődünk a projekttel, akkor...
- „I am important and so are you”

Elvek

- Humanity
 - > Business and personal needs
- Economics
 - > Somebody has to pay for all this.
- Mutual Benefit
 - > Every activity should benefit all concerned
- Self-Similarity
 - > Copying a structure of a solution to new context
- Improvement
 - > In software development „perfect” is a verb, not an adjective
- Diversity
 - > Conflict comes with diversity
- Reflection
 - > Good teams don't just do their work, they think about “how” and “why”
- Flow
 - > Steady flow of valuable sw
- Opportunity
 - > Problems as opportunity for change
- Redundancy
 - > Redundancy of practices
- Failure
 - > If you can't succeed, fail
- Quality
 - > Quality is not a control variable
- Baby Steps
 - > Many small steps rapidly looks like a leap
- Accepted Responsibility
 - > Responsibility cannot be assigned

Scrum

- Egy konkrét agilis keretrendszer
- Előír szerepköröket és eseményeket, de további technikák is belevehetők, amelyek nem mondanak ellent az alapvetéseinek
- Időkeretes eseményekkel dolgozik, maximális időt ad meg, ez alatt be kell fejeződniük

Követelmények megértése

- Sok résztvevő
 - > Fejlesztő: fejlesztéshez ért, az üzleti területhez nem
 - > Megrendelő: üzletileg érdekelt a szoftverben, de nem feltétlen ért se a szoftverhez, se az üzleti területhez
 - > Domain expert, felhasználó: elvileg ért a doménhez, talán tudja, mi segítené a munkáját, de nem mindig kérdezik meg elégszer, illetve nem is feltétlen tudja elmondani
 - > Megrendelő $\neq$ felhasználó: sokszor nem azonosak, pl. kórházigazgató és ápolónő

Követelmények megértése

- Rossz irányba ment fejlesztés
 - > Frusztráció a fejlesztőknek
 - > Pazarlás és időbeli csúszás a megrendelőnek
 - > Üzleti bukás
- Iteratív fejlesztés
 - > Gyakran validálni: „ez az, amit szeretnél volna?”
 - > Nem kerüli el a félreértéseket, de kordában tarthatóak
 - > Más megközelítést igényel (nem lehet rétegenként haladni, horizontálisan kell felosztani a feladatot)

A követelmények változása

- Még ha meg is értjük egymást, sok minden változhat
 - > Jogi környezet (ld. Uber, AirBnb)
 - > Piaci verseny helyzete
 - > Technológiai fejlődés
- Megrendelőnek kell tudnia, hogy versenyképes-e, amit szeretne
- Nekünk felkészülni az esetleges változásokra
 - > Lehet holnap már mást kér tőlünk
- Erre is az **iteratív** fejlesztés a megoldás

Verifikáció és Validáció

- Verifikáció: a specifikációval összhangban implementálom-e a rendszert?
 - > Jól építem-e meg?
 - > Klasszikusan ezt teszteljük
- Validáció: az üzleti igényeket kielégítő rendszert építek-e?
 - > A jó rendszert építem meg?
 - > Jól lettek definiálva a követelmények?
 - > Felhasználó előtti demóval vagy speciális tesztekkel végezzük

Pontatlan becslések

- A szoftverfejlesztés kreatív alkotómunka
 - > Szobafestés vs. költészet
- Tapasztalat kell a becsléshez
- Így is lehetnek nem várt kihívások, érdemes ráhagyással
- Elnagyolt, változó, félreértett követelmények tovább nehezítik a helyzetet

Pontatlan becslések

- Nehezen tartható keretek!
- Következmények
 - > Több idő = több költség
 - > Ha kifutunk az időből, nem biztos, hogy továbbra is lesznek anyagi erőforrások a fejlesztés fedezésére
 - > Csúszással nemcsak többet kell a megrendelőnek költenie, hanem a korábbi piacra lépésből realizálható haszontól is elesik

Priorizálás

- A fenti problémák priorizálással mérsékelhetők
- Mindig a legfontosabb elemeken dolgozunk
 - > Ami elkészül, az legyen jó és használható...
- Ezek minél előbb elkészülnek, ha valamire végül nem jut pénz/idő, az kisebb jelentőségű dolog
- Kevésbé kerül veszélybe a piacra lépés
- Hogyan?
 - > Azt teljesítsük, amit kérnek, ne próbáljuk túlteljesíteni
 - > Gondolkodjunk előre, de ne túlzottan; inkább hagyjuk nyitva a bővítés lehetőségét, de ne tegyünk sok energiát túl előre tervezésre

A Scrum csapat

- **Product Owner (PO):** megrendelőt képviseli
 - > Átmenet üzleti és szoftveres világ közt
 - > Érti a követelményeket és közvetíti a fejlesztőknek
 - > Csapat része
- **Scrum Master (SM):** a Scrum elvek betartását segíti
 - > Nem klasszikus PM
 - > Nem a csapat vezetője
 - > A Scrumban képzett
- **Development Team:** a fejlesztők szigorú szerepkörök nélkül

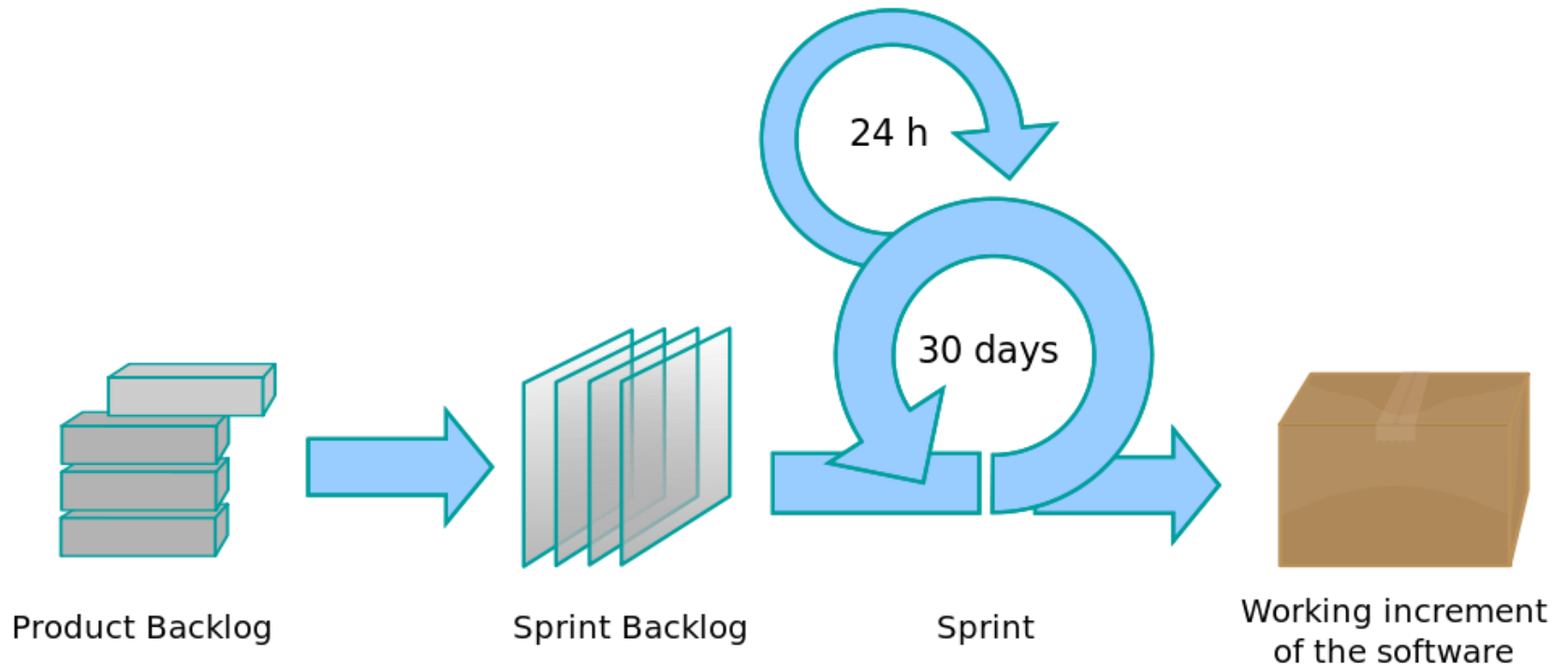
Hajó analógia

- PO: kapitány, kijelöli az irányt
- SM: motivál, koordinál, segít kikerülni az akadályokat
- DT: evezősök
- <https://www.advancedproductdelivery.com/blog/2017/06/20/roles-in-scrum/>

Önszerveződő csapat

- A DT csapat önszerveződő
- Ők tudják a legjobban, hogyan tudnak hatékonyan dolgozni
- Minden taszkért közösen felelnek
- SM, PO nem szól bele

Scrum folyamat



A Scrum események

- Spring Planning (≤ 8 óra)
- Sprint (≤ 1 hó)
- Daily Scrum (≤ 15 perc)
- Sprint Review (≤ 4 óra)
- Sprint Retrospective (≤ 3 óra)

Projektindítás

- Első sprint előtti tervezés
- Product Vision: nagyon magas szintű vízió arról, hogy mi fog készülni
 - > „I would have written you a shorter letter, but I didn't have the time.” (Blaise Pascal)
 - > **Üzleti célok: milyen értéket képvisel a megrendelőnek**
 - > (Gyakorlati célok: fejlesztőként mit kell megcsinálnunk)
- NOT list: egyelőre nem kell vagy bizonytalan
 - > Ez is a fókuszálást, priorizálást szolgálja

Projektindítás

- Definition of Done: fektessük le előre, hogy mikor tekintünk valamit elkészültnek?
 - > Implementáltam
 - > Lefordul
 - > Tesztek futnak
 - > Gitre pusholtam
 - > Többiek átnézték
 - > Többi komponenssel jól integrálódik
 - > Master branchbe merge-elve lett
 - > PO elfogadta
 - > Stb.

Product Backlog

- Definiálni kell az elkészítendő user story-kat
- Hagyományos specifikáció helyett ezeket használjuk
- Prioritás szerint csökkenő sorrendben, ez a backlog
- Minden sprint elején a tetejéről veszünk le annyit, amennyit a sprint alatt elkészítünk → Sprint Backlog

User story

Egy rövid, követelmény-leírás, ami egy felhasználó szemszögéből írja le, mit szeretne a szoftvertől.

pl: "As a customer, I want to filter products by price, so that I can find items within my budget."

- A követelmények megfogalmazásának eszköze, egy kicsi, tovább nem bontható feature
 - Adott formátuma van
 - > Cím
 - > Leírás
 - > Elfogadási kritériumok
 - > NOT lista
 - > Nemtriviális tesztesetek
 - > Alfeladatok
 - > Story pontok -> mennyire NAGY/BONYOLULT a feladat
 - > Becsült idő
 - > Elköltött idő
-
- PO írja meg
- Fejlesztőkre bízva

User story leírása

As [ROLE], I was to achieve [GOAL] for a [REASON]

- Három fő dolog mindig benne van
 - > **Role**: ki számára értékes
 - > **Goal**: funkcionálisan mit tudjon a szoftver (gyakorlati cél)
 - > **Reason**: miért képvisel ez értéket (üzleti cél)

Elfogadási kritériumok

- Nem technológiai, hanem felhasználóközpontú
- De objektíven mérhető kritérium, pl.
 - > A formon szerepel egy név mező
 - > Ebben csak betűk szerepelhetnek, különben hibaüzenet jelenik meg
 - > Van alatta egy mentés gomb
 - > A gombon való kattintás elmenti az adatot az adatbázisban

NOT Lista

Egy lista arról, mi NEM tartozik bele az adott user story-ba – tehát mit NE csináljon meg a fejlesztő, még ha kapcsolódna is hozzá.

- Ami jelenleg nem a scope része a user story-ban
 - > Lehet későbbi user story része
 - > Vagy még nem eldöntött, hogy szükséges-e
- Pl.:
 - > A bemenetet még nem validáljuk
 - > A jelszóemlékeztető funkciónak még nem kell működnie

Nemtriviális tesztesetek

- Emlékeztető, hogy milyen speciális eseteket kellene vizsgálni
- Pl.:
 - > Határidő nem lehet jövőbeli dátum
 - > Kategória nélkül nem menthető el termék

Alfeladatok

- A megrendelőnek lényegtelen
- Az önszerveződő agilis csapat használja
 - > Eldöntik, hogy lehet lépésekre bontani a feladatot,...
 - > ... és felosztják a lépéseket maguk közt
- Teljesen a csapatra van bízva
 - > Lehet alfeladatokat kiosztani, de teljes story-kat is
- Segíti a becslést

User Story példák

- The accounting software must run on a dedicated server
- As an administrator, I want the accounting software not to put extra load of critical systems
- Accounting data must be visualized fast
- As an end-user, I want the data of clients to be accessible in 1 second

A sprintek menete

- Ha a kezdeti tervezés kész, sprinteket kell terveznünk
- A sprintek menete
 1. Sprint planning (≤ 8 óra)
 2. Tényleges fejlesztés napi munkával
 3. Sprint review (≤ 4 óra)
 4. Retrospektív (≤ 3 óra)

Sprint tervezés

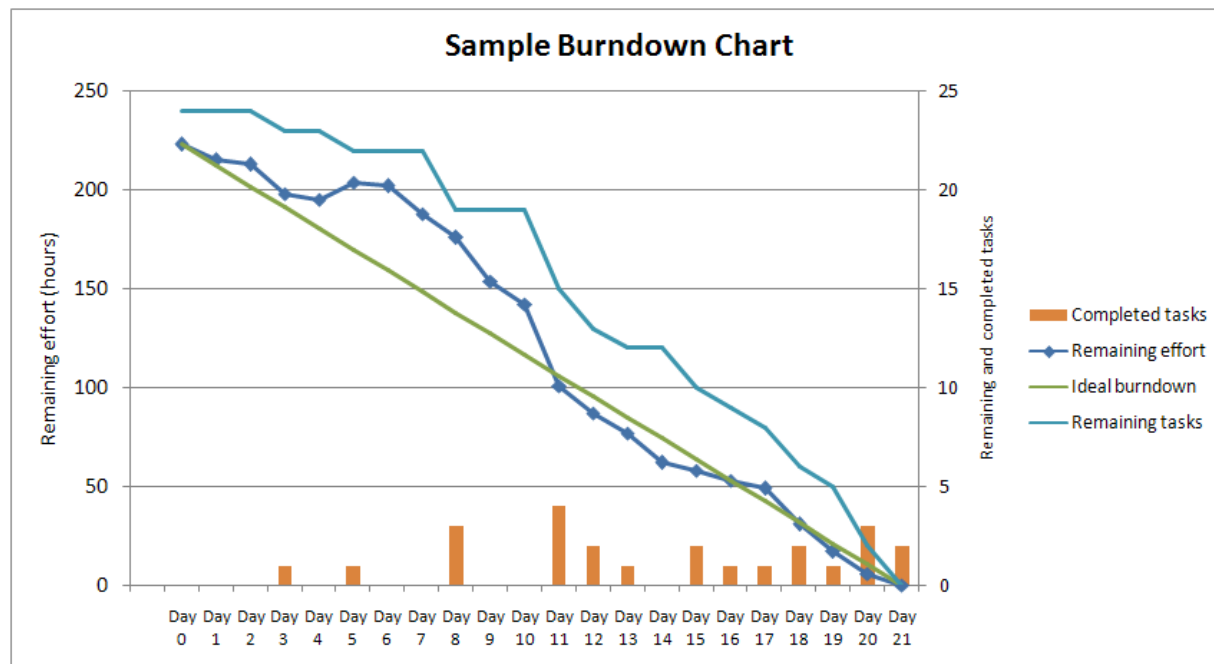
- Párórás meeting a sprint elején
 - A sprint tipikusan 2-4 hét, erre tervezünk, ideális napokkal és hetekkel (napi 8 óra, heti 5 nap)
1. A user story-kon megyünk prioritási sorban
 2. Beazonosítjuk az alfeladatokat
 3. Hozzárendelünk egy story pointot
 4. Adunk egy időbecslést
 5. Amikor elég story-t „beáraztunk”, vállalást teszünk

Story Points

- A user story-kat relatív nehézségi sorrend szerint rendezzük, és a közepes ér 5 pontot
- A közepes nehézségre adunk becslést
 - > A többi ehhez relatívan becsüljük, pl. kétszer olyan nehéz, dupla idő
 - > Korábbi tapasztalat segít a becsléseinket egyre pontosabbá tenni
 - > Átlagos story point / sprint teljesítmény kalkulálható
 - > Ez segít a hosszabb távú becslésben

Scrum artifacts

- **Product Backlog:** Termék teendőlistája prioritás szerint rendezve
- **Sprint Backlog:** Sprint teendőlistája
- **Burn down chart:** napi eredmény-kimutatás



Use Case vs. User Story

! -> Gondoljunk itt a Use Case diagramra!

- Hasonlóság
 - > Szerep/aktor, cél, elfogadási kritériumok
- Különbségek
 - > Use case általában részletesebb
 - > User story inkább beszélgetés indító
- “A story card is a promise for a conversation.”
- User story ~ feature – eredmény központú
 - > „a marker for what is to be built, fine-grained enough to fit into modern iteration/sprint periods” (A. C.)
- Use case: hogyan reagál/mit csinál a rendszer
 - > a contextual view of what is to be built

Daily Scrum

- Napi szinten is tervezünk a nap elején
- Max. 15 perc, **ténylegesen felállva!**
 - > Nem fotelben elterülve
 - > Nem laptopba bújva
- Mindenki megválaszolja:
 - > Mit végeztem el tegnap?
 - > Mit fogok ma csinálni?
 - > Van-e valami, ami akadályozza a munkámat?

A sprint terv módosulása

- Scope levágható
- Az időbeli hosszt sosem változtatjuk!
- Nincs félkész user story
 - > Kettébontás
 - > Rollback és a következő sprintre marad
- Sürgős user story bejöhet, PO dönti el
- Sprint csak akkor szakítható meg, ha a célja elavulttá vált

“Kész, kész” (done, done)

- Megtervezett
- Lekódolt
- Integrált
- Tesztelt
- A build lefut
- A termék installálódik
- Migrálódik
- A végfelhasználó ellenőrizte és elfogadta
- Megfixált

Demo

- Annak áttekintése, hogy mely munkák készültek el és melyek nem.
- Az elkészült munka bemutatása a terméktulajdonos és a fejlesztésben érdekeltk részére (demo).

Retrospektív

- Cél a munkafolyamataink folyamatos javítása
 - > Mit csináltunk jól, mit kell megtartani?
 - > Mit kéne másképp próbálnunk?
 - > Nem bűnbakok kereséséről szól!
- Kimenet: **végrehajtható akciók**
 - > Legalább egyet fel kell venni a következő sprint backlogjába
 - > **Teszteljünk többet**
 - > Csak getter, setter és konstruktor maradhat teszteletlen

Mit csinál pontosan a SM?

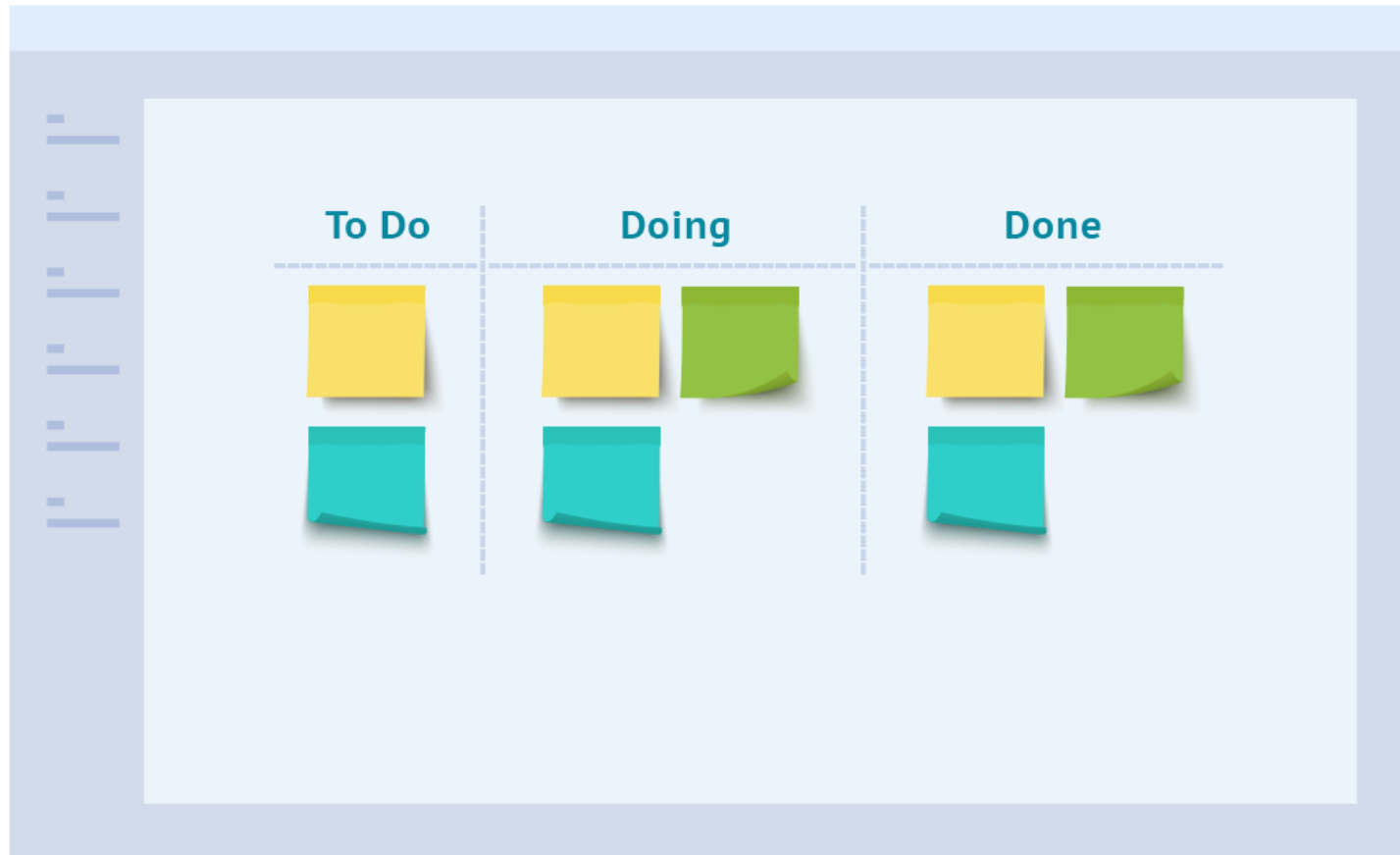
- „Szolgálva vezető”
- Facilitátor
- Coach
- Menedzser
- Mentor
- Oktató
- Problémamegoldó
- Változást segítő

<https://www.scrum.org/resources/blog/8-stances-scrum-master>

Kanban tábla

- Nem a Scrum része, külön technika, de szokták vele használni
- Fizikai tábla oszlopokkal, a sprint alatt a user story-kat és esetleg más taszkokat ezen vezetjük
- Az állapotokat a csapat dönti el, tipikusan a Definition of Done lépései alapján

Kanban



Az eXtreme Programming

- A Scrum mellett egy másik agilis módszer
- Nagy különbség nincs
- Az XP több gyakorlatot tesz kötelezővé
 - > Pair Programming
 - > Test-Driven Development (TDD)
 - > Continuous Integration (CI)
 - > Collective Code Ownership

Tényleg működik ez?

CHAOS RESOLUTION BY AGILE VERSUS WATERFALL

SIZE	METHOD	SUCCESSFUL	CHALLENGED	FAILED
All Size Projects	Agile	39%	52%	9%
	Waterfall	11%	60%	29%
Large Size Projects	Agile	18%	59%	23%
	Waterfall	3%	55%	42%
Medium Size Projects	Agile	27%	62%	11%
	Waterfall	7%	68%	25%
Small Size Projects	Agile	58%	38%	4%
	Waterfall	44%	45%	11%

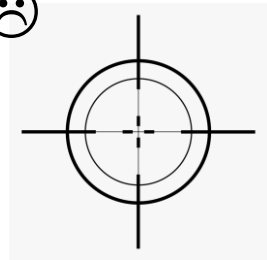
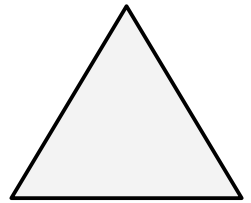
The resolution of all software projects from FY2011-2015 within the new CHAOS database, segmented by the agile process and waterfall method. The total number of software projects is over 10,000

CHAOS konklúzió

- Korábban a sikeres projekt:
 - > büdzsén belül
 - > időben
 - > Tervezett funkcionalitás
- “We found that both **satisfaction** and **value** are greater when the features and functions delivered are much less than originally specified and only meet obvious needs.”
 - 2015 Chaos Report, Standish Group

Összevetés

- Mindegyik módszertannak megvan a maga erőssége, ki van valamire „hegyezve”!
 - > Egyiket sem „könnyű” jól csinálni
 - > Máshol a kockázat... - más a
- Vízesés:
 - > Előre rögzített keretek tartása, tervezés, beavatkozás 😊
 - > Nem biztos, hogy az készül el, amire szükség van ☹️
- Agilis
 - > Biztos, hogy olyan készül el, amire szükség van 😊
 - > Előre nem tudjuk biztosan, hogy mikorra, mennyiért, és azt sem, hogy pontosan mi lesz az ☹️



Köszönöm a figyelmet!